SE

SOFTWARE ENGINEERING & SDLC

Software Engineering

> SDLC, Agile, testing pyramid, CI/CD, code review – code రాయడం కాకుండా, ఒక team lo software ని ఎలా నడపాలి.

CS fundamentals – self-taught / non-CS background నుంచి వచ్చినవారికి SSE interview lo అడిగే లోతు వరకు. ప్రతి concept ని MERN / JavaScript ప్రపంచంతో ముడిపెట్టి.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – Process (సాఫ్ట్‌వేర్ ఎలా తయారవుతుంది)

1. Software Engineering అంటే ఏమిటి, ఎందుకు (coding vs engineering, scale దగ్గర process/discipline ఎందుకు ముఖ్యం)
2. SDLC – Software Development Life Cycle (phases: requirements → design → implement → test → deploy → maintain)
3. SDLC Models (Waterfall vs Iterative vs Agile vs Spiral – ఎప్పుడు ఏది, comparison table)
4. Agile & Scrum – deep (manifesto, sprints, roles PO/SM/team, ceremonies, backlog, user stories, story points, velocity)
5. Kanban vs Scrum (flow vs cadence, ఎప్పుడు ఏది)

Part 2 – Building well (బాగా కట్టడం)

6. Requirements & Design (functional vs non-functional, ఎలా gather చేయాలి, design docs, HLD/LLD links)
7. Version Control Workflows (feature-branch, gitflow, trunk-based; Pull Requests + code review culture)
8. Clean Code & Code Quality (readability, naming, functions, comments, DRY/KISS/YAGNI, tech debt, refactoring, SOLID recap)

Part 3 – Quality (నాణ్యత)

9. Testing – deep (ఎందుకు test, unit/integration/e2e, test pyramid, TDD red-green-refactor, mocking/stubbing, coverage, Jest example)
10. Code Review (ఏం చూడాలి, feedback ఇవ్వడం/తీసుకోవడం, PR etiquette)

Part 4 – Ship & operate (deploy చేసి నడపడం)

11. CI/CD (continuous integration/delivery/deployment, pipeline stages, GitHub Actions example, deployment strategies blue-green/canary/rolling)
12. DevOps basics (containers/Docker, orchestration/Kubernetes, IaC, monitoring/logging/alerting, "you build it you run it")
13. Documentation, Estimation & Collaboration (READMEs, API docs, work estimation, teamwork, incident/postmortem culture)

Part 5 – Reference (రిఫరెన్స్)

14. Interview Q&A (process + behavioral) + Memory Tips + Common Mistakes

Part 1 – Process (సాఫ్ట్‌వేర్ ఎలా తయారవుతుంది)

> Code రాయడం వేరు; software *engineer* చేయడం వేరు. ఈ Part లో software engineering అంటే ఏమిటి, ఒక idea నుండి production వరకు ఏ phases దాటుతుంది (SDLC), ఆ phases ని నడిపే models (Waterfall/Agile/...), మరియు ప్రపంచంలో ఎక్కువ teams వాడే Agile/Scrum/Kanban – ఇవి నేర్చుకుంటాం. ఇది నీ mental model. ఇది clear అయితే, ఏ company join అయినా "వీళ్ళు ఎలా పని చేస్తున్నారు" అని instant గా అర్థమవుతుంది.

---

1. Software Engineering అంటే ఏమిటి, ఎందుకు

వివరణ

చాలామంది self-taught devs కి ఉన్న అపోహ: **"నాకు React, Node వచ్చు కదా, అంటే నేను software engineer."** కాదు. అది **coding** వచ్చు అని అర్థం. Coding అనేది software engineering లో ఒక్క భాగం మాత్రమే – దాదాపు 20-30%. మిగతా 70% ఏమిటంటే: సరైన problem ని అర్థం చేసుకోవడం, దాన్ని maintainable గా design చేయడం, team తో కలిసి build చేయడం, test చేయడం, safely deploy చేయడం, break అయితే fix చేయడం, మరియు 2 సంవత్సరాల తర్వాత కూడా ఆ code ని ఇంకో engineer చదివి అర్థం చేసుకోగలగడం.

Definition: Software Engineering = *"multiple people, over a long time, building software that works reliably"* అనే problem ని solve చేసే **discipline.** ఇది Fred Brooks అనే engineer చెప్పిన idea కి దగ్గర: **"a program"** (నువ్వు ఒక్కడివి రాసి, ఒక్కసారి run చేసేది) కి, **"a software product/system"** (చాలామంది కలిసి రాసి, సంవత్సరాల పాటు వేలమంది వాడేది) కి మధ్య భారీ తేడా ఉంది. ఆ తేడాను handle చేయడమే engineering.

Google దీన్ని ఒక్క వాక్యంలో చెప్తుంది: **"Software engineering is programming integrated over time."** అంటే – *time* అనే dimension add అయినప్పుడు programming, engineering అవుతుంది. నువ్వు ఇవాళ రాసిన code రేపు మారాలి, పక్కవాడు దాన్ని touch చేయాలి, ఒక సంవత్సరం తర్వాత కూడా అది పని చేయాలి.

మూడు forces ఒక program ని engineering గా మారుస్తాయి:

1. **Time (కాలం):** Code ఒక్కసారి రాసి పడేసేది కాదు. అది నెలలు/సంవత్సరాల పాటు బతుకుతుంది, మారుతూ ఉంటుంది. "ఇది తర్వాత maintain చేయాల్సి వస్తుంది" అనే ఆలోచన ప్రతి decision ని మారుస్తుంది.
2. **Scale (పరిమాణం):** ఒక్క దేవుడు కాదు – 5, 50, 500 engineers ఒకే codebase మీద పని చేస్తారు. అప్పుడు coordination, standards, communication తప్పనిసరి.
3. **Trade-offs (రాజీలు):** Engineering లో "సరైన" answer ఉండదు; "ఈ context కి best" answer ఉంటుంది. Fast vs cheap vs reliable – మూడూ ఒకేసారి పొందలేవు. దేన్ని ఎంచుకోవాలో నిర్ణయించడమే

engineering.

Real-life Scenario

> **Coding = ఇంట్లో నీకోసం వంట చేయడం. Software Engineering = 500 మందికి రోజూ వడ్డించే restaurant kitchen నడపడం.**
>
> ఇంట్లో నీకోసం వండేటప్పుడు – recipe కాగితం మీద రాయక్కర్లేదు, ఉప్పు కొంచెం ఎక్కువైనా నువ్వే తింటావు, పాత్రలు తర్వాత కడుగుతా అనుకోవచ్చు, రేపు వేరేలా వండినా ఎవరికీ తెలియదు. అది **coding** – ఒక్కడివి, ఒక్కసారి, నీకోసం.
>
> కానీ restaurant kitchen లో? Recipe ప్రతి dish కి **standardized** (design doc) గా ఉండాలి – లేకపోతే ప్రతి chef వేరేలా చేస్తాడు. Order రావడం, cook చేయడం, quality check, serve చేయడం – ఒక **process** (SDLC) ఉంటుంది. కొత్త chef వచ్చినా ఆ recipe చూసి అదే dish చేయగలగాలి (readable code + docs). ఒక dish లో సమస్య వస్తే మిగతా kitchen ఆగకూడదు (isolation, testing). Rush hour లో 5 గురు chefs coordinate అవ్వాలి (teamwork, version control). ఒక్క పొరపాటు 500 మంది customers కి పోతుంది (scale + reliability).
>
> **అదే వంట – కానీ context మారగానే, "ఎలా వండాలి" అనే skill కంటే "kitchen ని ఎలా నడపాలి" అనే skill ముఖ్యమవుతుంది. అదే software engineering.** SSE అంటే మంచి chef కాదు – kitchen ని నడపగలిగే head chef.

Coding vs Engineering – అసలు తేడా

అంశం	Coding (programmer)	Software Engineering (engineer)
దృష్టి	"ఇది పని చేస్తుందా?"	"ఇది reliable గా, maintainable గా, team తో కలిసి పని చేస్తుందా?"
కాల పరిధి	ఇప్పుడు (now)	సంవత్సరాలు (code lifetime)
ఎవరు చదువుతారు	నేను	నేను + 10 మంది + future నేను
Success అంటే	Output correct	Correct + tested + reviewed + deployed + observable + documented
ఏం value	Cleverness (smart trick)	Clarity + simplicity (boring is good)
Failure handling	Crash అయితే మళ్ళీ run	Graceful handling, monitoring, rollback
Change	Rewrite from scratch	Extend safely without breaking existing
Decision basis	"నాకు నచ్చింది"	Trade-offs + data + team agreement

గుర్తుంచుకో: **Interview లో "smart code" కంటే "maintainable, well-reasoned code" ఎక్కువ marks తెస్తుంది.** SSE level లో వాళ్ళు నీ cleverness ని కాదు, నీ *judgment* ని test చేస్తారు.

Process/discipline scale దగ్గర ఎందుకు అంత ముఖ్యం?

ఒక్కడివి weekend project చేసేటప్పుడు process అనవసరం అనిపిస్తుంది – "నేను నేరుగా code రాసేస్తా, git commit చేస్తా, deploy చేస్తా." కానీ 8-10 మంది team లో అదే attitude ఉంటే chaos:

- ఇద్దరు ఒకే file మార్చారు → conflict, ఎవరి change పోతుందో తెలియదు.
- ఒకడు test చేయకుండా deploy చేశాడు → production down, 10,000 users affected.
- ఒక feature ఎందుకు అలా రాశారో ఎవరికీ తెలియదు (no docs) → ఆ person leave అయితే knowledge పోయింది (**bus factor = 1**).
- ఒకడు రాసిన code ఇంకొకడు అర్థం చేసుకోలేడు → ప్రతి change slow, buggy.

Process అంటే bureaucracy కాదు – అది "అందరూ ఒకే lane లో, ఒకే వేగంతో, గుద్దుకోకుండా drive చేయడానికి traffic rules." ఒక్కడివి empty road మీద rules అనవసరం. కానీ 8-lane highway మీద rules లేకపోతే accidents guaranteed. Team size పెరిగే కొద్దీ process యొక్క value exponentially పెరుగుతుంది.

ఒక సంఖ్యాపరమైన intuition: **communication paths** = n·(n-1)/2. 2 గురు ఉంటే 1 path; 5 గురు ఉంటే 10 paths; 10 మంది ఉంటే 45 paths! అందుకే team పెరిగే కొద్దీ "ఎవరు ఏం చేస్తున్నారు" track చేయడానికి, sync అవ్వడానికి process అవసరమవుతుంది. Process లేకపోతే, add చేసిన ప్రతి కొత్త engineer team ని *slow* చేస్తాడు (Brooks's Law: "adding manpower to a late software project makes it later").

The hidden truth: code write కంటే read ఎక్కువ

Engineers తమ time లో **code రాయడానికి 10%, code చదవడానికి/అర్థం చేసుకోవడానికి/debug చేయడానికి 90%** ఖర్చు చేస్తారు. అందుకే:

- **Readable > clever.** రేపు ఇంకొకడు (లేదా మతిమరుపు నువ్వే) దీన్ని చదవాలి.
- **Boring > exciting.** Predictable code = fewer bugs.
- **Simple > complete.** అవసరం లేని feature రాయకు (YAGNI – Topic 8).

ఈ ఒక్క insight – "code is read far more than it is written" – clean code, testing, reviews, documentation ఎందుకు అన్నిటినీ justify చేస్తుంది. మొత్తం software engineering ఈ ఒక్క నిజం చుట్టూ తిరుగుతుంది.

Junior → Senior → SSE: అసలు ఏం మారుతుంది

Self-taught dev గా నీ లక్ష్యం SSE. కానీ SSE అంటే "ఎక్కువ code రాయడం" కాదు. Levels మధ్య తేడా *scope of impact* – నీ నిర్ణయాలు ఎంత దూరం, ఎంత కాలం ప్రభావితం చేస్తాయి:

అంశం	Junior	Senior / SSE
దృష్టి	ఇచ్చిన task ని code చేయడం	సరైన task ఏమిటో నిర్ణయించడం
Scope	ఒక function/file	whole feature/system/team
ప్రశ్న	"ఎలా చేయాలి?"	"అసలు చేయాలా? ఏ trade-offs?"
Help	help అడుగుతాడు	ఇతరులని unblock చేస్తాడు
Code	పని చేస్తే చాలు	maintainable, tested, observable

అంశం	Junior	Senior / SSE
Failure	"ఎవరు fix చేస్తారు?"	"నేను own చేస్తా, prevent చేస్తా"
Ambiguity	frozen, clear specs కావాలి	ambiguity ని navigate చేస్తాడు
Value	తన output	team output (reviews, mentorship, docs)

కీలక mindset shift: junior "నా code ఎలా ఉంది?" అనుకుంటాడు; senior **"నా నిర్ణయాలు 6 నెలల తర్వాత team కి ఎలా ఉంటాయి?"** అనుకుంటాడు. ఈ document మొత్తం నిన్ను ఆ రెండో ఆలోచనా విధానానికి తీసుకెళ్తుంది – code నుండి *judgment* కి.

Real MERN team లో ఇది ఎలా కనిపిస్తుంది

నువ్వు solo గా MERN app చేసినప్పుడు: idea → code → `git push` → deploy. అంతే.

అదే ఒక company లో MERN team లో:

1. Product team ఒక feature idea తెస్తుంది → **requirements** clarify (Topic 6).
2. Team అది sprint కి plan చేస్తుంది → **Agile/Scrum** (Topic 4).
3. నువ్వు feature branch తీసి code రాస్తావు → **version control** (Topic 7).
4. Unit tests రాస్తావు → **testing** (Topic 9).
5. PR raise చేస్తావు, ఇద్దరు seniors review చేస్తారు → **code review** (Topic 10).
6. Merge అవగానే CI pipeline build + test run చేస్తుంది → **CI/CD** (Topic 11).
7. Staging → production కి canary deploy → **deployment strategies** (Topic 11).
8. Prod లో error rate పెరిగితే alert వస్తుంది, నువ్వే fix చేస్తావు → **DevOps/on-call** (Topic 12).
9. Incident అయితే postmortem రాస్తావు → **collaboration** (Topic 13).

ఈ మొత్తం loop నే ఈ document నేర్పుతుంది. ఒక్క feature కూడా "code రాసి push చేయడం" మాత్రమే కాదు – అది ఒక **disciplined pipeline** గుండా వెళ్తుంది.

Key Points

- **Coding ≠ Software Engineering.** Coding = program రాయడం; engineering = *time* + *scale* + *trade-offs* కలిసినప్పుడు software ని reliable గా build & maintain చేయడం.
- Google definition: **"programming integrated over time."** Time అనే dimension programming ని engineering గా మారుస్తుంది.
- Scale దగ్గర process = traffic rules. Team పెరిగే కొద్దీ communication paths n^2గా పెరుగుతాయి → discipline తప్పనిసరి.
- **Code is read 10x more than written** → readable, simple, boring, tested code యే asset. ఇదే మిగతా అన్ని practices కి మూలం.
- SSE అంటే fastest coder కాదు – **best judgment + team enabler + long-term thinker.**

Interview దృష్టి

Q: మీరు software engineer నా, programmer నా? తేడా ఏమిటి? A: Programmer code రాస్తాడు; engineer software system ని time + scale దృష్ట్యా build & maintain చేస్తాడు. నేను ఒక feature

రాసేటప్పుడు "ఇది పని చేస్తుందా?" మాత్రమే కాదు – "6 నెలల తర్వాత ఇంకొకడు దీన్ని safely మార్చగలడా? Tests ఉన్నాయా? Prod లో break అయితే తెలుస్తుందా?" అని ఆలోచిస్తాను. ఆ long-term, team-oriented thinking యే నన్ను engineer చేస్తుంది.

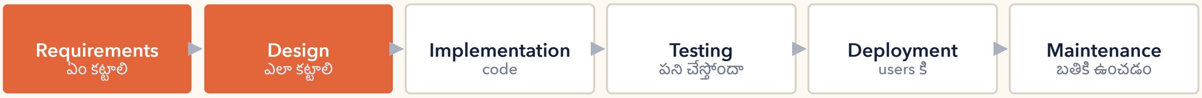
Q: మీ startup లో ఒక్కరే dev ఉన్నప్పుడు కూడా process అవసరమా? Overhead కాదా? A: Process ని team size కి తగ్గట్టు scale చేయాలి. Solo project లో heavy Scrum ceremonies waste. కానీ version control, basic testing, CI, README ఇవి solo లో కూడా viable – ఎందుకంటే future నేనే ఆ code చదవాలి. Process అనేది "అందరికీ same heavy rules" కాదు; "ఈ context కి తగిన discipline" – అదే engineering judgment.

Q: Non-CS background నుండి వచ్చారు కదా, gap ఎలా పూడ్చుకున్నారు? A: Coding నేను bootcamp/self-study తో నేర్చుకున్నా, కానీ *engineering process* (SDLC, Agile, testing, CI/CD, code review, on-call) ని consciously study చేసి, ప్రతి practice ఎందుకు ఉందో అర్థం చేసుకున్నా. ఇప్పుడు నేను code మాత్రమే కాదు – ఒక feature ని idea నుండి production monitoring వరకు తీసుకెళ్లగలను. (ఈ answer నిన్ను చాలా mature గా చూపిస్తుంది.)

2. SDLC – Software Development Life Cycle

SDLC · ఆరు దశలు

SDLC – ఆరు దశలు



Bug ఖరీదు దశ ప్రకారం పెరుగుతుంది

Requirements lo పట్టుకుంటే – ఒక వాక్యం మార్చడం.
Production lo పట్టుకుంటే – code మార్పు + test + deploy + బహుశా data fix + customer trust.
అందుకే review, testing "నెమ్మది" కాదు – అవి తర్వాతి ఖర్చుని తగ్గించే పెట్టుబడి.

వివరణ

SDLC = Software Development Life Cycle. ఒక software idea (mind లో) నుండి running product (users చేతిలో) వరకు ప్రయాణించే **standard phases** యొక్క పేరు ఇది. ఇది ఏ particular framework కాదు – ఇది ఒక *mental checklist.* ప్రపంచంలో ఏ team అయినా (Waterfall అయినా, Agile అయినా) ఈ phases దాటాల్సిందే; తేడా ఏమిటంటే – వాటిని **ఏ order లో, ఎంత తరచుగా** దాటుతారు అనేదే.

ఆరు core phases:

```
 ┌──────────────┐   ┌──────────┐   ┌─────────────┐   ┌─────────┐   ┌───────────┐
┌─────────────┐
│ 1. Require-  │──▶│ 2. Design│──▶│ 3. Implement│──▶│ 4. Test│──▶│ 5. Deploy│──▶│ 6.
Maintain│
│    ments     │   │ (HLD/LLD)│   │   (code)    │   │         │   │           │   │
│
└──────────────┘   └──────────┘   └─────────────┘   └─────────┘   └───────────┘
└─────────────┘
     ఏం?              ఎలా?            రాయి            సరిగ్గా       users కి
బతికుండగా
   (what)           (how)          (build)         పనిచేస్తుందా?    ఇవ్వు            చూసుకో
```

```
                                                         (verify)      (ship)
(operate)
       ▲
|
       └──────────────────── feedback / కొత్త requirements ◄──────────────────┘
```

చివరి arrow గమనించు: maintain phase లో వచ్చే feedback తిరిగి requirements గా మారుతుంది – అందుకే దీన్ని **cycle** (life *cycle*) అంటారు, straight line కాదు. Software ఎప్పటికీ "పూర్తి" అవ్వదు; అది ఎప్పుడూ evolve అవుతూనే ఉంటుంది.

Real-life Scenario

> **SDLC = ఒక ఇల్లు కట్టడం.** నువ్వు నేరుగా ఇటుకలు పేర్చడం మొదలుపెట్టవు కదా?
>
> 1. **Requirements** – "ఎన్ని గదులు కావాలి? Budget ఎంత? ఎంతమంది ఉంటారు?" (owner తో మాట్లాడటం). ఇది skip చేస్తే – 2 bedrooms కావాలంటే 4 కట్టేస్తావు.
> 2. **Design** – architect blueprint గీస్తాడు (ఏ గది ఎక్కడ, plumbing, wiring). Paper మీద mistake సరిచేయడం చవక; concrete లో సరిచేయడం ఖరీదు.
> 3. **Implement** – masons ఇల్లు కడతారు (actual coding).
> 4. **Test** – inspector wiring, leaks, strength check చేస్తాడు (QA). Family move అవ్వక ముందే.
> 5. **Deploy** – family గృహప్రవేశం చేసి move అవుతుంది (release to users).
> 6. **Maintain** – leak వస్తే fix, కొత్త గది కావాలంటే add (bug fixes + new features).
>
> **ఏ phase skip చేసినా ప్రమాదం:** requirements లేకుండా కడితే తప్పు ఇల్లు; design లేకుండా కడితే గోడలు కూలుతాయి; test లేకుండా move అయితే wiring షార్ట్ అవుతుంది. Software లో కూడా అచ్చం అంతే – కాకపోతే bugs కనబడవు కాబట్టి మనం skip చేసినా ఇబ్బంది అనిపించదు... production లో break అయ్యేదాకా.

ప్రతి phase ని లోతుగా

Phase 1 – Requirements (ఏం build చేయాలి?)

- Stakeholders (product manager, client, users) ఏం కావాలో సేకరించడం.
- Functional requirements ("user login చేయగలగాలి") + Non-functional ("2 seconds లో load అవ్వాలి, 10k concurrent users") – Topic 6 లో deep.
- Output: **PRD** (Product Requirements Document), user stories, acceptance criteria.
- **అతి ముఖ్యం, ఎందుకంటే:** ఇక్కడ తప్పు అర్థం చేసుకుంటే, మిగతా అన్ని phases perfect గా చేసినా – తప్పు product build అవుతుంది. "సరైన software" కంటే "software సరిగ్గా" చేయడం ముఖ్యం అనుకుంటాం, కానీ ముందు సరైనది ఏమిటో తెలియాలి.

Phase 2 – Design (ఎలా build చేయాలి?)

- **HLD (High-Level Design):** system architecture – ఏ services, ఏ database, ఎలా communicate. (`HLD_Telugu.md`, `HLD_Go_Telugu.md`)

- **LLD (Low-Level Design):** class/module structure, API contracts, schemas, design patterns. (`LLD_Telugu.md`)
- Output: architecture diagrams, API specs, DB schema, tech decisions (ADR – Architecture Decision Records).
- MERN లో ఉదా: "MongoDB schema ఎలా? REST vs GraphQL? Auth JWT vs session? Frontend state Redux vs Context?"

Phase 3 – Implementation (build)

- Actual coding. Version control లో feature branches (Topic 7), clean code (Topic 8), unit tests (Topic 9) రాయడం.
- Output: working code, merged PRs.

Phase 4 – Testing (verify)

- Unit → integration → e2e → manual QA → UAT (User Acceptance Testing). Test pyramid (Topic 9).
- Output: passing test suite, bug reports, QA sign-off.
- **గుర్తుంచుకో:** modern teams లో testing separate phase కాదు – implementation తోనే జరుగుతుంది (shift-left testing). కానీ conceptually ఇది ఒక distinct activity.

Phase 5 – Deployment (ship)

- Code ని users చేరేలా production కి తీసుకెళ్లడం. CI/CD pipelines (Topic 11), deployment strategies (blue-green/canary/rolling).
- Output: live software, release notes.

Phase 6 – Maintenance (operate)

- Bug fixes, performance tuning, security patches, కొత్త features, monitoring, on-call (Topic 12).
- **అతి ఖరీదైన phase:** software total cost లో **60-80% maintenance లోనే** ఖర్చవుతుంది! అందుకే మనం "future maintainability" కోసం ఇప్పుడు clean code, tests రాస్తాం. Development అనేది మంచుకొండ యొక్క కనిపించే భాగం మాత్రమే; maintenance నీటి కింద ఉన్న పెద్ద భాగం.

Cost of change – ఎందుకు early phases అంత ముఖ్యం

Software లో ఒక అతి ముఖ్యమైన నిజం: **ఒక bug/తప్పు ఎంత late దొరుకుతే, fix అంత ఖరీదు.** దీన్ని Boehm's cost-of-change curve అంటారు:

```
Fix cost
(relative)
  1000x |                                         ●  Production
        |                                     /
   100x |                                ●  Testing
        |                            /
    10x |                       ●  Implementation
        |                   /
```

```
    1x │  ●────────●  Requirements/Design
       │
       └──────────────────────────────────────────▶ ఏ phase లో దొరికింది
```

Requirements phase లో "user email ద్వారా login చేస్తారా, phone ద్వారానా?" అని అడగడం – ఉచితం (ఒక్క మాట). అదే question production లో revisit చేయాల్సి వస్తే – database migration, code rewrite, downtime, user impact. అందుకే **"ముందు ఆలోచించు, తర్వాత రాయి"** అనే క్రమశిక్షణ. (కానీ Agile దీన్ని balance చేస్తుంది – అతిగా ముందుగానే ప్రతిదీ design చేస్తే over-engineering; Topic 3-4 లో చూద్దాం.)

SDLC ≠ ఒక్కసారి; ఇది iterate అవుతుంది

Waterfall లో ఈ 6 phases *ఒక్కసారి*, sequence లో జరుగుతాయి. Agile లో అవే 6 phases *ప్రతి sprint (2 వారాలు)* లో చిన్న scale లో పునరావృతం అవుతాయి. అంటే SDLC phases మారవు – వాటి **rhythm** మారుతుంది. ఇది అర్థం చేసుకుంటే, తర్వాతి Topic (SDLC models) చాలా సులభం.

V-Model – ప్రతి phase కి ఒక test level

SDLC phases ని అర్థం చేసుకోవడానికి ఒక అందమైన mental model – **V-Model.** ఇది Waterfall యొక్క variant, కానీ కీలక insight: **ప్రతి development phase కి ఒక corresponding testing level ఉంటుంది.** ఎడమవైపు కిందకి (design), కుడివైపు పైకి (test) – "V" ఆకారం:

```
Requirements ──────────────────▶ Acceptance Testing (UAT)
   │                                  ▲
   ▼                                  │
 HLD (Architecture) ─────────────▶ System / E2E Testing
   │                                  ▲
   ▼                                  │
 LLD (Detailed design) ──────────▶ Integration Testing
   │                                  ▲
   ▼                                  │
   Coding ───────────────────────▶ Unit Testing
        (ఎడమ = build; కుడి = verify; ప్రతి level ఎడమవైపు దానికి test)
```

అర్థం: unit tests **code** ని verify చేస్తాయి; integration tests **LLD** (modules కలిసి) ని; system/E2E tests **HLD** (whole architecture) ని; acceptance tests అసలు **requirements** ని. ఈ mapping చాలా powerful – ఒక test fail అయితే, ఏ phase లో సమస్య ఉందో చెప్తుంది. Topic 9 test pyramid కి ఇది conceptual foundation. (V-Model కూడా Waterfall లాంటిదే – rigid; కానీ ఈ "phase↔test" mapping idea Agile లో కూడా విలువైనది.)

Key Points

- SDLC = idea నుండి retire వరకు software దాటే **6 phases**: Requirements → Design → Implement → Test → Deploy → Maintain.
- ఇది framework కాదు, ఒక **universal checklist** – Waterfall/Agile అన్నీ ఈ phases నే వేరే rhythm లో నడుపుతాయి.
- **Cost of change** phases పెరిగే కొద్దీ 10x-1000x పెరుగుతుంది → early clarity (requirements/design) చాలా విలువైనది.
- **Maintenance = total cost లో 60-80%** → అందుకే clean code, tests, docs future కోసం investments.

- చివరి loop-back (feedback → requirements) వల్ల ఇది *cycle* – software ఎప్పటికీ "done" కాదు.

Interview దృష్టి

Q: SDLC అంటే ఏమిటి, phases చెప్పండి. A: SDLC = software ని idea నుండి maintenance వరకు తీసుకెళ్లే lifecycle. Six phases: requirements (ఏం build చేయాలి), design (ఎలా – HLD/LLD), implementation (coding), testing (verify), deployment (ship to users), maintenance (bug fixes + evolution). ఇది ఒక cycle – maintenance లో వచ్చే feedback కొత్త requirements గా మారుతుంది.

Q: ఏ phase అత్యంత ముఖ్యం? A: రెండు కారణాలకి requirements. మొదటిది – తప్పు requirement అంటే మిగతా అన్ని phases perfect గా చేసినా తప్పు product. రెండోది – cost of change: requirements phase లో దొరికిన issue fix చేయడం చవక, production లో అదే fix 100-1000x ఖరీదు. కానీ maintenance phase total cost లో 60-80% తీసుకుంటుంది, కాబట్టి maintainability కి design early చేయడం కూడా అంతే ముఖ్యం.

Q: Testing ఏ phase? అది develop అయ్యాక చేస్తారా? A: Traditionally deploy కి ముందు separate phase. కానీ modern teams "shift-left" చేస్తాయి – implementation తోనే tests రాస్తాయి (TDD/unit tests), CI లో ప్రతి commit కి run అవుతాయి. అంటే testing ఒక late gate కాదు, continuous activity. దీనివల్ల bugs early దొరుకుతాయి → చౌక fix.

---

3. SDLC Models – Waterfall vs Iterative vs Agile vs Spiral

వివరణ

Topic 2 లో SDLC యొక్క **6 phases** చూశాం. కానీ ఆ phases ని *ఏ order లో, ఎన్నిసార్లు, ఎంత risk తో* నడపాలి? ఆ "ఎలా నడపాలి" అనే నిర్ణయమే **SDLC Model** (aka process model). నాలుగు ప్రధాన models – ప్రతిదీ ఒక specific problem కి పరిష్కారంగా పుట్టింది. వీటిని software history లో ఒక evolution గా చూస్తే బాగా గుర్తుంటాయి.

Real-life Scenario

> **నలుగురు cook ఒకే feast తయారుచేసే విధానాలు:**
>
> - **Waterfall cook:** మొత్తం menu ముందే fix చేసి, అన్నీ ఒకేసారి వండి, చివర్లో ఒక్కసారే వడ్డిస్తాడు. Guests రుచి చూసేదాకా వాళ్లకి నచ్చిందో లేదో తెలియదు. నచ్చకపోతే – మొత్తం మళ్ళీ.
> - **Iterative cook:** మొదట ఒక simple version వండి guests కి ఇస్తాడు, feedback తీసుకుని మెరుగుపరుస్తూ మళ్ళీ వండుతాడు – ప్రతిసారి better.
> - **Agile cook:** ప్రతి 2 gantalaki ఒక చిన్న dish ready చేసి, guests తినగానే feedback అడిగి, తర్వాతి dish ని దానికి తగ్గట్టు మారుస్తాడు. Menu కూడా మధ్యలో మారొచ్చు.
> - **Spiral cook:** ప్రతి కొత్త dish కి ముందు "ఇది risky గా ఉందా? (అలర్జీ? ఖరీదు?) ఒక చిన్న sample వండి test చేద్దాం" అని risk ముందు తగ్గించుకుంటాడు – పెద్ద, ఖరీదైన feast కి ఇది సురక్షితం.
>
> అన్నీ అదే feast – కానీ **feedback ఎప్పుడు తీసుకుంటారు, risk ఎలా handle చేస్తారు** అనేదే తేడా.

Model 1 – Waterfall (జలపాతం)

అన్ని phases **strict sequence** లో, ఒక్కసారి, పైనుండి కిందకి జలపాతంలా. ఒక phase 100% పూర్తయ్యాకే తర్వాతిది. వెనక్కి వెళ్ళడం కష్టం/ఖరీదు.

```
Requirements ──▼
               Design ──▼
                        Implement ──▼
                                   Test ──▼
                                         Deploy ──▼
                                                 Maintain
(ప్రతి దశ ముందుది పూర్తయ్యాకే మొదలవుతుంది – వెనక్కి వెళ్ళడం లేదు)
```

- **మంచిది:** requirements ముందే స్పష్టంగా, మారని projects (govt contracts, hardware, medical devices, aerospace). Documentation heavy. Predictable timeline & cost.
- **చెడ్డది:** requirements మారితే విపత్తు. Working software చివర్లోనే కనబడుతుంది – అప్పటికే తప్పు అయితే చాలా late. Real world లో requirements ఎప్పుడూ మారతాయి → అందుకే software లో Waterfall చాలావరకు fail అయ్యింది.
- **ఎప్పుడు:** requirements frozen, regulation heavy, feedback loop అవసరం లేని చోట. Modern web/product teams లో అరుదు.

Model 2 – Iterative & Incremental

మొత్తం ఒక్కసారి కాకుండా, **చిన్న చిన్న iterations** లో build చేయడం. ప్రతి iteration ఒక working (కానీ incomplete) version ఇస్తుంది, దాన్ని feedback తో మెరుగుపరుస్తారు.

- **Iterative** = అదే thing ని మళ్ళీ మళ్ళీ refine చేయడం (Mona Lisa ని rough sketch → shading → detail).

- **Incremental** = feature by feature add చేయడం (car: మొదట chassis, తర్వాత engine, తర్వాత seats).
- Real projects రెంటినీ కలిపి వాడతాయి. Agile దీని మీదే built.
- **ఎప్పుడు:** requirements కొంత తెలుసు కానీ పూర్తిగా కాదు, feedback కావాలి. చాలా modern approaches కి పునాది.

Model 3 – Agile

Iterative idea ని ఒక **culture + set of values** గా మార్చింది Agile (2001 manifesto – Topic 4). చిన్న iterations (sprints, 1-4 వారాలు), constant customer feedback, changing requirements ని welcome చేయడం, working software ని docs కంటే ఎక్కువ value చేయడం. **నేటి 70%+ software teams Agile (Scrum/Kanban).**

- **మంచిది:** requirements uncertain/evolving (అంటే almost every product). Fast feedback, early value delivery, adapt to change.
- **చెడ్డది:** predictability తక్కువ (exact timeline/cost చెప్పడం కష్టం), discipline లేకపోతే chaos, heavy documentation-dependent projects కి సరిపోదు.
- Topic 4-5 లో deep dive.

Model 4 – Spiral

Waterfall + Iterative + **risk analysis** కలయిక. ప్రతి loop (spiral) లో నాలుగు quadrants: (1) objectives set, (2) **risk analyze & prototype**, (3) develop & test, (4) plan next iteration. అతి ముఖ్యం – ప్రతి iteration ముందు **risk ని identify & reduce** చేస్తారు.

```
          ┌─────────────┐
          │  1. Plan    │   ← objectives, alternatives
          └──────┬──────┘
                 │
     ┌───────────┴───────────┐
     │                       │
┌────▼─────┐          ┌──────▼───────┐
│ 4. Next  │          │ 2. Risk      │   ← prototype, analyze risk
│ iteration│          │  analysis    │       (అతి ముఖ్యం)
└──────────┘          └──────┬───────┘
     │                       │
     └───────────┬───────────┘
          ┌──────▼──────┐
          │ 3. Build+Test│
          └─────────────┘
     (ప్రతి spiral పెద్దదవుతూ, risk తగ్గుతూ)
```

- **మంచిది:** పెద్ద, ఖరీదైన, high-risk projects (కొత్త technology, mission-critical). Risk ముందే తగ్గించడం.
- **చెడ్డది:** overhead ఎక్కువ (risk analysis expertise కావాలి), చిన్న projects కి overkill, ఖరీదు.
- **ఎప్పుడు:** R&D-heavy, భారీ budget, failure ఖరీదైన systems.

పోలిక – ఏ Model ఎప్పుడు (అతి ముఖ్యమైన table)

అంశం	Waterfall	Iterative	Agile	Spiral
Flow	Linear, ఒక్కసారి	Repeat + refine	Short sprints, adaptive	Risk-driven loops
Requirements	ముందే frozen	కొంత తెలుసు	Evolving, welcome change	పెరుగుతూ clarify
Feedback	చివర్లో	ప్రతి iteration	ప్రతి sprint (fast)	ప్రతి spiral
Working software ఎప్పుడు	చివర్లో మాత్రమే	early + growing	ప్రతి sprint చివర	ప్రతి spiral చివర
Risk handling	Weak (late discovery)	Medium	Continuous, small	**Explicit, upfront**
Documentation	భారీ	Medium	Light (working software > docs)	భారీ
Predictability (cost/time)	**High**	Medium	Low-Medium	Medium
Change ఖరీదు	చాలా ఎక్కువ	తక్కువ	**చాలా తక్కువ**	Medium
Best for	Fixed-scope, regulated (aerospace, govt)	Requirements partly known	Products, startups, web (MERN)	Large, high-risk, R&D
Team size	పెద్ద, siloed	Medium	చిన్న cross-functional	పెద్ద
Real-world లో	తగ్గుతోంది	పునాది	**Dominant (70%+)**	Niche (aerospace, defense)

ఏది ఎంచుకోవాలి – ఒక్క line decision guide

- Requirements **frozen + regulated** (medical/defense/govt) → **Waterfall.**
- Requirements **evolving, fast feedback కావాలి** (startup, web product, MERN app) → **Agile.** (99% of your career ఇక్కడే.)
- **భారీ risk + budget** (కొత్త tech, satellite) → **Spiral.**
- **Iterative** అనేది standalone కంటే Agile/Spiral కి underlying philosophy.

ముఖ్యమైన insight: "ఏ model best?" అనేది తప్పు ప్రశ్న. సరైనది – "ఈ project యొక్క uncertainty & risk కి ఏ model సరిపోతుంది?" Requirements uncertain అయితే Waterfall ఆత్మహత్య; requirements frozen అయి regulation ఉంటే Agile chaos. **Context ప్రకారం model.**

Real MERN team లో

99% MERN product teams **Agile** (Scrum లేదా Kanban) వాడతాయి. కారణం: product requirements ఎప్పుడూ మారతాయి (user feedback, market, A/B tests), fast ship చేయాలి, competitor ముందు

వెళ్ళాలి. కానీ ఒక enterprise MERN app లో payment/compliance module కి కొంచెం Waterfall-ish rigor (upfront design, heavy testing, sign-offs) కలపొచ్చు – దీన్ని **"Water-Scrum-Fall"** అని కూడా అంటారు (hybrid). Real teams pure model అరుదుగా వాడతాయి; అవి pragmatic గా mix చేస్తాయి.

Key Points

- **SDLC Model = SDLC phases ని ఏ order/rhythm/risk-approach తో నడపాలి** అనే నిర్ణయం.
- **Waterfall** = linear, ఒక్కసారి, frozen requirements కి; change ఖరీదు, feedback late.
- **Iterative/Incremental** = చిన్న chunks లో build + refine; Agile కి పునాది.
- **Agile** = short sprints + constant feedback + welcome change; నేటి dominant model (products/web/MERN).
- **Spiral** = risk-first loops; పెద్ద, ఖరీదైన, high-risk projects కి.
- సరైన model = project యొక్క **uncertainty + risk + regulation** ని బట్టి; pure కంటే hybrids common.

Interview దృష్టి

Q: Waterfall vs Agile – తేడా, ఎప్పుడు ఏది? A: Waterfall linear – అన్ని requirements ముందే fix, phases sequence లో, feedback చివర్లో; frozen-scope, regulated projects కి (aerospace, medical). Agile iterative – short sprints, changing requirements ని welcome, ప్రతి sprint చివర working software + feedback; evolving requirements ఉన్న products/startups కి. Web/MERN product almost always Agile ఎందుకంటే requirements constantly మారతాయి, మనం fast feedback ప్రకారం adapt అవ్వాలి.

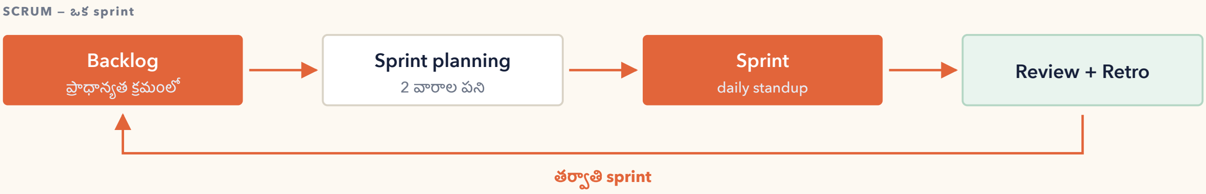
Q: Agile అన్నిటికీ best అని అనొచ్చా? A: లేదు – అది common tell. Agile fast-changing, uncertain products కి great, కానీ regulation heavy లేదా fixed-scope contract projects లో దాని low predictability సమస్య. మంచి engineer model ని project context (uncertainty, risk, compliance) కి match చేస్తాడు, blindly Agile అని చెప్పడు. చాలా real teams hybrid (Water-Scrum-Fall) వాడతాయి.

Q: Spiral model ఎప్పుడు? A: పెద్ద, ఖరీదైన, high-risk (కొత్త, unproven technology; failure చాలా ఖరీదు) projects లో – ఎందుకంటే ప్రతి iteration ముందు explicit risk analysis + prototyping చేసి risk ముందే తగ్గిస్తుంది. Web CRUD app కి overkill, కానీ satellite software కి ideal.

---

4. Agile & Scrum – Deep Dive

SCRUM · SPRINT యొక్క చక్రం



Retro యొక్క ఉద్దేశం

ఏం బాగా జరిగింది · ఏం జరగలేదు · ఏం మారుస్తాం
ఒక్క actionable item అయినా తీసుకోవాలి
లేకపోతే అది కేవలం ఫిర్యాదుల సమావేశం

Standup ఎలా చెడిపోతుంది

15 నిమిషాలు దాటడం · status report అవడం
Manager కి నివేదిక కాదు – team కి sync
Blocker చెప్పడమే అసలు ఉద్దేశం

వివరణ

నీ SSE career లో 99% సమయం నువ్వు **Scrum** లో పని చేస్తావు. Interview లో "మీ team ఎలా పని చేస్తుంది?" అని అడిగినప్పుడు fluent గా Scrum vocabulary మాట్లాడగలగాలి. అందుకే ఈ topic అతి ముఖ్యం. దీన్ని రెండు layers గా అర్థం చేసుకుందాం: **Agile** (philosophy/values) మరియు **Scrum** (ఆ values ని అమలు చేసే specific framework).

Agile అనేది 2001 లో 17 engineers ఒక ski resort లో కలిసి రాసిన **Agile Manifesto** నుండి పుట్టిన ఒక mindset. అప్పటి Waterfall-heavy industry లో projects fail అవుతున్నాయి – years పట్టి, డబ్బు తిని, చివర్లో users కి అక్కర్లేని software ఇచ్చేవి. దానికి పరిష్కారంగా Agile.

Agile Manifesto – 4 values (ఎడమవైపు కంటే వేటికి ఎక్కువ value ఇస్తామో):

ఎక్కువ value	కంటే	తక్కువ (కానీ value లేకపోలేదు)
Individuals & interactions	over	processes & tools
Working software	over	comprehensive documentation
Customer collaboration	over	contract negotiation
Responding to change	over	following a plan

గమనిక: కుడివైపు వాటికి value లేదని కాదు – ఎడమవైపు వాటికి ఎక్కువ value అని. Documentation చెడ్డది కాదు; కానీ working software కంటే ముఖ్యం కాదు. దీనితో పాటు **12 principles** ఉన్నాయి (early & continuous delivery, welcome change, deliver frequently, business+devs together daily, motivated individuals, face-to-face, working software = progress measure, sustainable pace, technical excellence, simplicity, self-organizing teams, regular reflection).

Real-life Scenario

> **Agile = ఒక పెళ్లి tailor, Waterfall = ready-made shop.**
>
> Ready-made shop (Waterfall) – నీ measurements ఒక్కసారి తీసుకుని, నెల తర్వాత పూర్తి dress ఇస్తారు. అప్పటికి నీ బరువు మారిందా, design నచ్చలేదా – bad luck, మళ్ళీ మొదటి నుండి.
>
> Agile tailor – వారానికోసారి నిన్ను పిలిచి, half-stitched dress try చేయిస్తాడు. "ఇక్కడ tight గా ఉందా? ఈ color బాగుందా?" అని అడుగుతాడు. నువ్వు "sleeve పొడవు తగ్గించు" అంటే వెంటనే మారుస్తాడు. చివర్లో నీకు సరిగ్గా కావాల్సిన dress వస్తుంది, ఎందుకంటే ప్రతి అడుగులో నీ feedback తీసుకున్నాడు.
>
> **Software లో users కి ఏం కావాలో వాళ్ళకే స్పష్టంగా తెలియదు – ready-made చూసేదాకా.** అందుకే చిన్న చిన్న deliveries, constant feedback. అదే Agile యొక్క గుండె.

Scrum అంటే ఏమిటి

Scrum = Agile values ని అమలు చేసే అత్యంత popular framework. Rugby నుండి వచ్చిన పేరు (team కలిసి ball ముందుకు తోసే formation). Core idea: పనిని **Sprints** అనే fixed-length timeboxes (సాధారణంగా 2 వారాలు) గా విభజించి, ప్రతి sprint చివర ఒక **potentially shippable increment** (వాడగలిగే working software) deliver చేయడం.

Scrum కి మూడు స్తంభాలు (pillars): **Transparency** (అందరికీ ఏం జరుగుతోందో కనిపించాలి), **Inspection** (regular గా progress చూడాలి), **Adaptation** (చూసినదాన్ని బట్టి మారాలి). ఈ మూడూ empirical process control – "plan అంతా ముందే చేయలేం, చేస్తూ నేర్చుకుందాం" అనే philosophy.

Scrum Roles – ఎవరు ఏం చేస్తారు

Scrum లో మూడు roles మాత్రమే (titles కాదు, responsibilities):

Role	ఏం చేస్తారు	Analogy	ఎవరు కాదు
Product Owner (PO)	ఏం build చేయాలో నిర్ణయిస్తారు. Backlog ని own & prioritize చేస్తారు. "Voice of customer." Value maximize చేస్తారు.	Movie director – ఏ scene ముఖ్యం చెప్తాడు	ఎలా build చేయాలో చెప్పే boss కాదు
Scrum Master (SM)	Process ని facilitate చేస్తారు, blockers (impediments) తొలగిస్తారు, team ని protect చేస్తారు. "Servant leader."	Referee/coach – game rules చూస్తాడు, ఆడడు	Manager/boss కాదు, team ని command చేయడు
Development Team	Actual software build చేసే engineers (devs, QA, designers). **Self-organizing** – ఎలా build చేయాలో వాళ్ళే నిర్ణయిస్తారు. Cross-functional.	Actors + crew – సినిమా తీస్తారు	PO/SM కి "కింది" వాళ్ళు కాదు; peers

అతి ముఖ్యమైన distinction: PO = **what/why** (ఏం, ఎందుకు). Dev team = **how** (ఎలా). SM = process ని smooth గా నడిపే గ్రీజ. PO team కి "ఎలా code రాయాలో" చెప్పడు; team PO కి "ఏ feature ముఖ్యమో" చెప్పదు. ఈ boundary respect చేయడం healthy Scrum కి కీలకం.

Scrum Artifacts – మూడు documents/lists

1. **Product Backlog:** చేయాల్సిన అన్ని పనుల (features, bugs, tech debt) master list, PO priority ప్రకారం sort చేసిన. Living document – ఎప్పుడూ మారుతుంది. పైన ఉన్నవి refined & ready; కింద ఉన్నవి vague.
2. **Sprint Backlog:** ఈ sprint కి team commit అయిన items (product backlog నుండి top items). Sprint మధ్యలో ఇది freeze – కొత్తవి add చేయరు (scope protection).
3. **Increment:** ఈ sprint లో పూర్తయిన working software, "Definition of Done" (DoD) కి match అయ్యి, potentially shippable.

Scrum Ceremonies – 4 meetings

Scrum నాలుగు fixed events (ceremonies) చుట్టూ తిరుగుతుంది. ప్రతి sprint లో ఇవి జరుగుతాయి:

1. Sprint Planning (sprint మొదట్లో, ~2-4 గంటలు)

- "ఈ sprint లో ఏం చేస్తాం?" Team product backlog నుండి top items తీసుకుని, capacity ప్రకారం sprint backlog కి commit చేస్తుంది.
- Output: **Sprint Goal** + Sprint Backlog. PO priorities చెప్తాడు, team ఎంత చేయగలదో commit చేస్తుంది.

2. Daily Standup / Daily Scrum (రోజూ, 15 నిమిషాలు, timeboxed)

- నిలబడి (అందుకే standup – short గా ఉంచడానికి) 3 questions: (a) నిన్న ఏం చేశా? (b) ఇవాళ ఏం చేస్తా? (c) ఏమైనా blockers ఉన్నాయా?
- **లక్ష్యం:** sync + blockers surface చేయడం. Status report to boss కాదు – team కి team చెప్పుకోవడం. Problems solve చేయడం ఇక్కడ కాదు (offline "parking lot" లో).
- **సాధారణ తప్పు:** ఇది 45-min discussion అవ్వడం. SM దీన్ని 15 min కి timebox చేయాలి.

3. Sprint Review / Demo (sprint చివర్లో, ~1-2 గంటలు)

- Team build చేసిన working software ని stakeholders (PO, users, managers) కి **demo** చేస్తుంది.
- Feedback తీసుకుంటారు → product backlog update అవుతుంది. "మనం సరైనది build చేస్తున్నామా?" (product-focused).

4. Sprint Retrospective / Retro (review తర్వాత, ~1 గంట)

- **Process** గురించి team ఆత్మపరిశీలన: ఏం బాగా జరిగింది? ఏం సరిగా జరగలేదు? తర్వాతి sprint లో ఏం మెరుగుపరచాలి?
- "మనం సరిగ్గా build చేస్తున్నామా?" (process-focused). Continuous improvement యొక్క engine ఇదే. Retro లేని Scrum = మెరుగవ్వని Scrum.

> **Review vs Retro గుర్తుంచుకోవడానికి:** Review = **product** ("ఏం build చేశాం" – stakeholders తో). Retro = **process** ("ఎలా build చేశాం" – team లోపల). Review బయటకి చూస్తుంది; Retro లోపలికి చూస్తుంది.

(Backlog Refinement/Grooming అనే 5వ activity కూడా ఉంటుంది – ongoing గా backlog items ని clarify, estimate, split చేయడం. ఇది official ceremony కాదు కానీ team రోజూ చేస్తుంది.)

```
          ┌────────────────── ఒక Sprint (2 వారాలు) ──────────────────┐
          │                                                           │
Product   │  Sprint     Daily    Daily    Daily    ...    Sprint    Sprint│
Backlog───┼─►Planning──►Standup─►Standup─►Standup ───►  Review──► Retro  │──► తర్వాతి
Sprint
   │      │  (ఏం చేస్తాం)  (15min)  (15min)  (15min)      (demo)   (improve)│
   │      │                                                           │
   └──────┴──── Sprint Backlog (ఈ sprint items, frozen) ──────────────┘
```

User Stories – requirement ని చెప్పే విధానం

Backlog items ని సాధారణంగా **user stories** గా రాస్తారు – feature ని *user దృష్టికోణం* నుండి చెప్పే చిన్న statement. Standard template:

```
As a [ఎవరు – role],
I want [ఏం – goal/action],
so that [ఎందుకు – benefit/value].

ఉదా (MERN e-commerce):
As a customer,
I want to save items to a wishlist,
so that I can buy them later without searching again.
```

ఎందుకు ఈ format? ఎందుకంటే ఇది **ఎవరు, ఏం, ఎందుకు** – మూడూ ఒక్క వాక్యంలో బంధిస్తుంది. "ఎందుకు" (so that) చాలా ముఖ్యం – అది లేకపోతే team blindly feature build చేస్తుంది, value అర్థం కాకుండా.

ప్రతి story కి **Acceptance Criteria** (AC) ఉంటుంది – "ఇది done అని ఎలా చెప్తాం?" చెప్పే testable conditions. తరచుగా **Given-When-Then** format:

```
Given నేను logged in customer ని,
When నేను ఒక product మీద "Add to Wishlist" click చేస్తే,
Then ఆ product నా wishlist లో కనిపించాలి, మరియు icon filled అవ్వాలి.
```

మంchi story కి **INVEST** గుణాలు: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable. Story చాలా పెద్దదైతే (ఒక sprint లో పట్టనంత) దాన్ని **Epic** అంటారు, దాన్ని చిన్న stories గా split చేస్తారు.

Estimation – Story Points, ఎందుకు hours కాదు

Team ఒక story ఎంత "పెద్దది" అని estimate చేయాలి – planning కోసం. కానీ **hours లో కాకుండా, Story Points** అనే relative unit లో. ఎందుకు?

- **Hours absolute + individual:** "ఇది 6 గంటలు" – ఎవరికి? Junior కి 12, senior కి 3. Estimate చెడిపోతుంది.
- **Story points relative + effort/complexity/risk:** "ఈ story ఆ story కి రెండింతలు complex" – ఇది team అంతటికీ stable. Absolute time కాదు, *relative size*.

సాధారణంగా **Fibonacci-like scale** (1, 2, 3, 5, 8, 13, 21...) వాడతారు. ఎందుకు Fibonacci? ఎందుకంటే story పెద్దదైన కొద్దీ uncertainty పెరుగుతుంది – 1 vs 2 తేడా చెప్పడం సులభం, కానీ 20 vs 21 అర్థరహితం. Gaps పెద్దవవ్వడం ఈ uncertainty ని reflect చేస్తుంది. 8+ points వచ్చిందంటే story ని split చేయమని signal.

Planning Poker: team అందరూ ఒకేసారి ఒక story కి తమ estimate (cards) చూపిస్తారు. తేడాలు ఉంటే – "నువ్వు 3 ఎందుకు, నేను 8 ఎందుకు?" అని discuss చేస్తారు (ఒకరికి తెలిసిన complexity ఇంకొకరికి తెలియకపోవచ్చు). Consensus వచ్చేదాకా re-vote. ఇది estimate కంటే **shared understanding** కోసం – అసలు value అదే.

Velocity – team ఎంత వేగంగా

Velocity = ఒక sprint లో team పూర్తి చేసే average story points. ఉదా: గత 3 sprints లో 20, 24, 22 points → velocity ≈ 22. దీన్ని ఎందుకు track చేస్తారు?

- **Forecasting:** backlog లో 220 points ఉంటే, ~10 sprints (20 వారాలు) పడుతుంది అని estimate చేయవచ్చు.
- **Planning:** తర్వాతి sprint లో ~22 points మాత్రమే commit చేయాలి – overcommit కాకుండా.

అతి ముఖ్యమైన warning: velocity ఒక **planning tool, performance metric కాదు.** Manager "team A velocity 30, team B velocity 20, కాబట్టి A better" అంటే – అది తప్పు, hazardous. Story points teams మధ్య compare చేయలేం (ప్రతి team scale వేరు). Velocity ని target చేస్తే teams points ని inflate చేస్తారు (Goodhart's Law: "a measure that becomes a target ceases to be a good measure"). Velocity = ఆ team యొక్క self-planning కి మాత్రమే.

Real MERN team లో ఒక sprint

సోమవారం sprint planning: PO backlog నుండి "wishlist feature (5 pts), fix cart bug (2 pts), search pagination (3 pts)" propose చేస్తాడు. Team capacity చూసి 22 points commit చేస్తుంది. రోజూ 10am standup – "నేను wishlist API పూర్తిచేశా, ఇవాళ frontend, MongoDB indexing లో blocker ఉంది." SM ఆ blocker ని DBA తో solve చేయిస్తాడు. రెండో శుక్రవారం review – PO + stakeholders కి live demo. Retro – "PR reviews slow అయ్యాయి, తర్వాతి sprint నుండి రోజుకో review slot పెట్టుకుందాం." ఇదే rhythm ప్రతి 2 వారాలకి పునరావృతం.

Definition of Done (DoD) & Definition of Ready (DoR)

Scrum లో రెండు అతి ముఖ్యమైన "quality contracts" – ఇవి interview లో తరచు అడుగుతారు:

- **Definition of Ready (DoR):** ఒక story ని sprint లోకి తీసుకోవడానికి ముందు ఏమేమి ready అయి ఉండాలి? (acceptance criteria clear, dependencies తెలుసు, estimable, mockups ఉన్నాయి). DoR లేని story sprint లోకి తీసుకుంటే – మధ్యలో "ఇది ఎలా చేయాలి?" అని ఇరుక్కుంటారు.
- **Definition of Done (DoD):** ఒక story "done" అని ఎప్పుడు చెప్పగలం? Team అంతటికీ common checklist – code written **+ tests pass + reviewed + merged + docs updated + deployed to staging.** "నా machine లో పని చేస్తోంది" = done కాదు.

> **DoD = "done" అనే మాటకి team యొక్క ఉమ్మడి నిర్వచనం.** ఇది లేకపోతే ప్రతి dev కి "done" వేరే అర్థం – ఒకడికి "code రాశా" done, ఇంకొకడికి "prod లో నడుస్తోంది" done. ఈ ambiguity వల్ల "90% done" stories నెలలు వేలాడతాయి ("last 10%" నరకం). DoD ఆ ambiguity ని చంపుతుంది.

అంశం	Definition of Ready	Definition of Done
ఎప్పుడు apply	Story ని sprint లోకి తీసుకునేముందు (entry gate)	Story ని done అనేముందు (exit gate)
ప్రశ్న	"ఇది మొదలుపెట్టడానికి సిద్ధమా?"	"ఇది పూర్తయ్యిందా?"
ఉదా	AC clear, estimated, no blockers	tested, reviewed, merged, deployed

Burndown Chart – sprint progress ని చూడటం

Burndown chart = sprint లో మిగిలిన work (story points) ని రోజురోజుకీ చూపే graph. Y-axis = remaining points, X-axis = sprint days. Ideal గా అది పైనుండి కిందకి (sprint చివర్లో 0) descend అవ్వాలి.

```
Points    |●\  ideal line (planned)
remaining|  \___
         |      \●  actual (planned కంటే పైన = behind schedule)
         |        \●___
         |            \___●
         |               \●___
       0 └───────────────────\●──▶ days
          Day1              Day10 (sprint చివర)
```

- Actual line ideal కంటే **పైన** = team behind (scope ఎక్కువ commit అయ్యింది, లేదా blockers).
- Flat గా ఉంటే (points తగ్గట్లేదు) = stuck, standup లో surface చేయాలి.
- ఇది **early warning system** – sprint చివరిదాకా ఆగకుండా, రోజూ "మనం track లో ఉన్నామా?" చూపిస్తుంది. (Release burndown = మొత్తం project కి similar.)

సాధారణ Scrum Anti-patterns (వీటిని avoid చేయి)

చాలా teams "Scrum చేస్తున్నాం" అంటాయి కానీ ఈ తప్పులతో దాని value కోల్పోతాయి:

- **Standup = status report to manager:** team కి team sync కాదు, boss కి report అయిపోతుంది. (Standup team ది, manager ది కాదు.)
- **Retro లో action items లేకుండా:** "ఏం బాగోలేదు" చెప్పి, ఏమీ మార్చకపోవడం – retro theatre. Action items + owners కావాలి.
- **Sprint మధ్యలో scope మార్చడం:** PO కొత్త "urgent" work push చేస్తూనే ఉంటే – team ఏదీ finish చేయలేదు. Sprint commitment ని respect చేయాలి.
- **Velocity ని whip గా వాడటం:** manager "velocity పెంచండి" అంటే → points inflation, burnout.
- **"Water-Scrum-Fall":** పైకి Scrum, లోపల Waterfall (అన్ని requirements ముందే frozen, feedback లేదు). Ceremonies ఉన్నా Agile mindset లేదు.
- **DoD లేకపోవడం:** "done" అంటే ఎవరికి వాళ్ళ definition → half-done work pile అవుతుంది.

Key Points

- **Agile = mindset (4 values, 12 principles)**; **Scrum = ఆ mindset ని అమలు చేసే framework.** Agile అంటే Scrum కాదు – Scrum అనేది Agile యొక్క ఒక flavor.
- **3 Roles:** PO (*what/why* – backlog owner), SM (*process facilitator*, servant leader), Dev Team (*how*, self-organizing).
- **3 Artifacts:** Product Backlog (all work), Sprint Backlog (this sprint, frozen), Increment (shippable output).
- **4 Ceremonies:** Planning (ఏం), Daily Standup (sync/blockers, 15min), Review (product demo), Retro (process improve).
- **User Story** = As a/I want/so that + Acceptance Criteria (Given-When-Then); INVEST properties; big = Epic → split.
- **Story Points** = relative effort/complexity (Fibonacci), hours కాదు. **Velocity** = sprint average points – planning కోసమే, performance metric కాదు.

Interview దృష్టి

Q: మీ team ఎలా పని చేస్తుంది? (almost guaranteed) A: మేము 2-week sprints తో Scrum follow చేస్తాం. Sprint planning లో PO backlog prioritize చేస్తాడు, మేము capacity ప్రకారం commit చేస్తాం. రోజూ 15-min standup లో progress + blockers sync చేస్తాం. Sprint చివర్లో stakeholders కి demo (review), తర్వాత retro లో process మెరుగుపరచుకుంటాం. Stories ని story points తో (planning poker) estimate చేస్తాం, velocity ని forecasting కి వాడతాం. (ఈ ఒక్క answer నిన్ను experienced గా చూపిస్తుంది.)

Q: Scrum Master vs Project Manager? A: PM traditionally command-control – ఏం, ఎప్పుడు చేయాలో directs, progress track చేస్తాడు. SM servant leader – team ని command చేయడు, blockers తొలగిస్తాడు, process ని protect చేస్తాడు, team self-organize అవ్వడానికి enable చేస్తాడు. Scrum లో "ఏం build చేయాలో" PO నిర్ణయిస్తాడు, "ఎలా" team; SM ఆ engine smooth గా నడిచేలా చూస్తాడు.

Q: Story points ని hours గా convert చేస్తారా? Velocity తో teams compare చేయవచ్చా? A: రెండూ anti-patterns. Story points relative complexity – hours కి map చేస్తే వాటి purpose (individual-independent estimation) పోతుంది. Velocity ఒక team యొక్క planning tool – teams మధ్య points scale వేరు కాబట్టి compare invalid, పైగా velocity ని target చేస్తే teams points inflate చేస్తారు (Goodhart's Law). ఈ mistakes గురించి అవగాహన maturity చూపిస్తుంది.

Q: Daily standup లో ఏం చెప్తారు? A: మూడు: నిన్న ఏం చేశా, ఇవాళ ఏం చేస్తా, ఏమైనా blockers. ఇది status report కాదు – team కి team sync + blockers surface చేయడం. Detailed problem-solving standup లో కాదు, తర్వాత offline (parking lot). 15 నిమిషాలకి timeboxed.

---

5. Kanban vs Scrum

వివరణ

Scrum ని చూశాం – sprints, ceremonies, roles. కానీ Agile ని అమలు చేసే ఇంకో popular framework ఉంది: **Kanban.** చాలా MERN teams (ముఖ్యంగా support/maintenance/ops-heavy teams) Kanban

వాడతాయి, లేదా Scrum + Kanban కలిపి **Scrumban** వాడతాయి. తేడా అర్థం చేసుకోవడం SSE కి ముఖ్యం.

Kanban (జపనీస్ = "visual card/signboard") Toyota manufacturing నుండి వచ్చింది. Core idea: పనిని ఒక **board** మీద visualize చేయడం, ఒకేసారి ఎంత work-in-progress (WIP) ఉండో **limit** చేయడం, మరియు work ని smooth **flow** గా నడపడం. Sprints లేవు, fixed roles లేవు, planning ceremonies compulsory కాదు – కేవలం continuous flow.

Kanban board typical structure:

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│   Backlog    │    To Do     │ In Progress  │   Review     │     Done     │
│              │              │  (WIP ≤ 3)   │  (WIP ≤ 2)   │              │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ [wishlist]   │ [cart bug]   │ [search API] │ [auth PR]    │ [signup]     │
│ [dark mode]  │ [pagination] │ [profile UI] │              │ [email fix]  │
│ [export CSV] │              │              │              │ [logout]     │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────┘
       cards ఎడమ నుండి కుడికి "flow" అవుతాయి. WIP limit దాటితే కొత్తది pull చేయవద్దు.
```

Real-life Scenario

> **Scrum = బస్సు; Kanban = auto/taxi.**
>
> **బస్సు (Scrum)** fixed schedule లో నడుస్తుంది – ప్రతి 15 నిమిషాలకి (sprint) ఒకటి బయలుదేరుతుంది. ఎంతమంది ఎక్కినా, ఆ time కి వెళ్తుంది. Predictable rhythm, కానీ నువ్వు schedule కి align అవ్వాలి. Batch గా passengers.
>
> **Auto (Kanban)** – ఎప్పుడు passenger వస్తే అప్పుడు బయలుదేరుతుంది, continuous flow. Schedule లేదు, కానీ auto ఒకేసారి 3 గురినే (WIP limit) ఎక్కించుకుంటుంది – overload అయితే service చెడిపోతుంది. Flexible, on-demand.
>
> Planned feature work (బస్సు లాంటి regular delivery) కి Scrum; unpredictable, on-demand work (support tickets, bugs – auto లాంటి "ఎప్పుడొస్తే అప్పుడు") కి Kanban.

Scrum vs Kanban – పూర్తి పోలిక

అంశం	Scrum	Kanban
Cadence (rhythm)	Fixed sprints (1-4 వారాలు)	Continuous flow, no fixed iterations
Roles	PO, SM, Dev Team (defined)	No prescribed roles
Commitment	Sprint కి scope commit	ఏ commitment లేదు, pull-based
Change మధ్యలో	Sprint freeze – కొత్తది add చేయరు	ఎప్పుడైనా priorities మార్చవచ్చు
Key metric	**Velocity** (points/sprint)	**Cycle time** (item start→done ఎంత time), **throughput**
WIP limit	Sprint backlog పరిమితం చేస్తుంది	Explicit **WIP limits** per column

అంశం	Scrum	Kanban
Ceremonies	Planning, standup, review, retro	Optional (standup + review helpful)
Board reset	ప్రతి sprint reset	ఎప్పుడూ persist (continuous)
Best for	Feature development, planned roadmap	Support, ops, bug-fixing, unpredictable flow
Change ఎంత సులభం	Sprint boundaries దగ్గర	ఎప్పుడైనా (అత్యంత flexible)
నేర్చుకోవడం	ఎక్కువ structure	తక్కువ structure, సులభం మొదలు

WIP Limits – Kanban యొక్క రహస్యం

Kanban యొక్క అతి ముఖ్యమైన idea: **Work In Progress (WIP) limit.** ప్రతి column కి "ఇక్కడ గరిష్ఠంగా n items మాత్రమే" అని limit. ఎందుకు ఇది powerful?

- **"Stop starting, start finishing."** మనం సహజంగా చాలా పనులు ఒకేసారి మొదలుపెడతాం, ఏదీ పూర్తవదు. WIP limit దీన్ని ఆపుతుంది – కొత్తది మొదలుపెట్టాలంటే, ముందుది finish చేయాలి.
- **Context switching తగ్గుతుంది.** 5 tasks మధ్య jump చేస్తే మెదడు అలసిపోతుంది; 2 మీద focus చేస్తే వేగం.
- **Bottlenecks కనిపిస్తాయి.** ఒక column ఎప్పుడూ నిండి ఉంటే – అక్కడ సమస్య ఉంది (ఉదా: "Review" column ఎప్పుడూ full అంటే reviewers చాలదు). Board visually ఈ bottleneck ని చూపిస్తుంది.

Little's Law (flow యొక్క math): `Cycle Time = WIP / Throughput`. అంటే WIP తగ్గిస్తే (తక్కువ పనులు ఒకేసారి), ప్రతి item వేగంగా పూర్తవుతుంది. Highway analogy: రోడ్డు మీద తక్కువ cars ఉంటే అందరూ వేగంగా వెళ్తారు; overload అయితే traffic jam, అందరూ slow.

Kanban metrics – Lead Time vs Cycle Time (తరచు confuse అవుతారు)

Kanban health ని ఈ metrics చెప్తాయి – interview లో తేడా అడగవచ్చు:

Metric	ఎప్పటి నుండి ఎప్పటి వరకు	ఎవరి దృష్టి
Lead Time	request వచ్చిన క్షణం → delivered	customer ("నేను అడిగాక ఎంత time?")
Cycle Time	team పని మొదలుపెట్టిన క్షణం → delivered	team ("మేము పట్టుకున్నాక ఎంత time?")
Throughput	ఒక period లో ఎన్ని items పూర్తయ్యాయి	capacity

Lead time ఎప్పుడూ cycle time కంటే ≥ (backlog లో wait time వల్ల). Customer కి lead time ముఖ్యం; team efficiency కి cycle time. వీటిని track చేసి తగ్గిస్తే – predictable, fast delivery. Scrum velocity కి Kanban equivalent ఇవి (points కాదు, time-based).

Scrumban – రెండింటి మేలు కలయిక

చాలా real teams pure Scrum లేదా pure Kanban కాదు – **Scrumban** వాడతాయి: Scrum యొక్క ceremonies (standup, retro, planning) + Kanban యొక్క board + WIP limits + continuous flow.

ఉదా: sprint planning చేస్తారు కానీ board మీద WIP limits పెడతారు, మధ్యలో urgent bug వస్తే pull చేస్తారు. Pragmatic – dogma కాదు.

ఏది ఎప్పుడు – decision guide

- **Planned feature roadmap, predictable delivery కావాలి** → Scrum. (Product teams)
- **Unpredictable, interrupt-driven work** (production support, bug queue, DevOps, ops) → Kanban. (ఎప్పుడు ఏది వస్తుందో తెలియదు)
- **Feature work + తరచు interruptions** → Scrumban.
- **Team కొత్తది, structure అవసరం** → Scrum (దాని guardrails నేర్పిస్తాయి). Mature team → Kanban సరిపోతుంది.

Real MERN team లో

ఒక product team కొత్త features build చేస్తూ ఉంటే → Scrum (sprints, roadmap). అదే company యొక్క **platform/on-call team** (production issues, DB migrations, urgent bugs handle చేసేది) → Kanban, ఎందుకంటే వాళ్ళ work ముందే plan చేయలేం – ఎప్పుడు ఏ incident వస్తుందో తెలియదు. చాలా MERN teams Jira/Linear/Trello board వాడతాయి – అది Kanban board యే (columns: Todo/In Progress/Review/Done). నువ్వు "Scrum team" అన్నా, రోజూ ఆ Kanban-style board నే చూస్తావు.

Key Points

- **Scrum = cadence-based** (fixed sprints, roles, ceremonies, velocity); planned feature work కి.
- **Kanban = flow-based** (continuous, WIP limits, cycle time, no fixed roles); unpredictable/support work కి.
- **WIP limits** = Kanban గుండె – "stop starting, start finishing," bottlenecks ని surface చేస్తాయి (Little's Law: WIP తగ్గిస్తే cycle time తగ్గుతుంది).
- **Scrumban** = practical hybrid (Scrum ceremonies + Kanban board/WIP), చాలా real teams ఇదే.
- సరైనది = work యొక్క nature (planned vs interrupt-driven) ని బట్టి; రెండూ Agile ని అమలు చేసేవే.

Interview దృష్టి

Q: Scrum vs Kanban – తేడా, ఎప్పుడు ఏది? A: Scrum cadence-based – fixed sprints, defined roles, ceremonies, velocity తో plan. Kanban flow-based – continuous, sprints లేవు, columns కి WIP limits, cycle time/throughput metrics. Scrum planned feature roadmap కి; Kanban unpredictable, interrupt-driven work (production support, bug queue) కి. Real teams తరచుగా Scrumban (రెండింటి కలయిక) వాడతాయి.

Q: WIP limit ఎందుకు? A: ఒకేసారి చాలా పనులు మొదలుపెడితే context switching పెరిగి ఏదీ పూర్తవదు. WIP limit "stop starting, start finishing" ని enforce చేస్తుంది – throughput పెంచుతుంది, bottlenecks ని visually surface చేస్తుంది (ఏ column ఎప్పుడూ full అయితే అక్కడే problem). Little's Law ప్రకారం WIP తగ్గిస్తే cycle time తగ్గుతుంది.

Q: మీ support team ఎలా పని చేస్తుంది? A: Support/on-call work unpredictable కాబట్టి Kanban board వాడతాం – WIP limits పెట్టి, cycle time track చేసి, urgent items ని pull చేస్తాం. Feature team మాత్రం Scrum sprints follow చేస్తుంది. Work యొక్క nature ప్రకారం framework ఎంచుకోవడం key.

Part 2 – Building well (బాగా కట్టడం)

> Process తెలిసింది; ఇప్పుడు బాగా build ఎలా చేయాలి? ఈ Part లో: సరైనది build చేస్తున్నామని ఎలా నిర్ధారించుకోవాలి (requirements & design), team గా code ని ఎలా manage చేయాలి (version control workflows), మరియు ఆ code ని ఎలా clean గా, maintainable గా రాయాలి (clean code, tech debt, SOLID). ఇవి "engineer" ని "senior engineer" నుండి వేరు చేసే skills.

---

6. Requirements & Design

వివరణ

Topic 1 లో చెప్పుకున్న నిజం: **"software సరిగ్గా build చేయడం" కంటే "సరైన software build చేయడం" ముఖ్యం.** ఎంత perfect గా code రాసినా, తప్పు feature build చేస్తే – వృథా. అందుకే ఏ line code రాసేముందు రెండు ప్రశ్నలకి జవాబు కావాలి: **(1) ఏం build చేయాలి? (requirements) (2) ఎలా build చేయాలి? (design).**

Non-CS self-taught devs ఎక్కువగా ఇక్కడే బలహీనం – వాళ్ళు "ఇచ్చిన task ని code చేయడం" మీద focus చేస్తారు, కానీ "అసలు ఈ task సరైనదా, పూర్తిగా అర్థమైందా, ఎలా design చేయాలి" అని ఆగరు. SSE ఈ ఆగడం చేస్తాడు – అదే junior/senior తేడా.

Real-life Scenario

> **Requirements లేకుండా code = address అడగకుండా బయలుదేరిన delivery boy.** ఎంత fast bike ఉన్నా, ఎంత బాగా drive చేసినా – తప్పు ఇంటికి వెళ్తే వృథా. ముందు "ఎక్కడికి?" (requirement) తెలియాలి, తర్వాతే "ఎలా వెళ్ళాలి?" (design – bike/route).
>
> **Design లేకుండా code = blueprint లేకుండా ఇల్లు కట్టడం.** Masons ఇటుకలు పేర్చడం మొదలుపెడతారు, కానీ bathroom ఎక్కడ, wiring ఎలా – ఏమీ plan లేదు. మధ్యలో "అయ్యో, ఇక్కడ కిటికీ ఉండాలి" అని గోడ పగలగొట్టాల్సి వస్తుంది. ఖరీదైన rework.

Functional vs Non-Functional Requirements (అతి ముఖ్యం)

రెండు రకాల requirements – ఇవి interview లో తరచు అడుగుతారు:

అంశం	Functional Requirements (FR)	Non-Functional Requirements (NFR)
అర్థం	System **ఏం చేయాలి** (features, behavior)	System **ఎలా ఉండాలి** (quality attributes)
ప్రశ్న	"What?"	"How well?"

అంశం	Functional Requirements (FR)	Non-Functional Requirements (NFR)
ఉదా (MERN)	"User login చేయగలగాలి", "cart కి item add చేయాలి", "order place చేయాలి"	"2 seconds లో load", "10k concurrent users", "99.9% uptime", "GDPR compliant"
Verify	Feature పని చేస్తోందా? (yes/no)	Metrics/thresholds తో measure
Miss అయితే	Feature లేదు	Feature ఉంది కానీ slow/insecure/unreliable

NFRs ని "-ilities" అని కూడా అంటారు: scalability, reliability, availability, security, maintainability, usability, performance, portability. Junior devs FRs మీద focus చేస్తారు; **senior engineers NFRs ని ముందే అడుగుతారు** – ఎందుకంటే NFRs architecture ని shape చేస్తాయి. "10 users కి vs 10 million users కి" design పూర్తిగా వేరు. (ఇది HLD యొక్క గుండె – `HLD_Telugu.md`, `HLD_Go_Telugu.md` లో deep.)

> **గుర్తుంచుకో:** FR = "car కి brakes ఉండాలి." NFR = "brakes 3 seconds లోపు 100kmph నుండి ఆపగలగాలి." రెండోది లేకపోతే car ఉంది కానీ ప్రమాదకరం.

Requirements ఎలా gather చేయాలి

Requirements ఆకాశం నుండి రావు – వాటిని *elicit* (సేకరించడం) చేయాలి. పద్ధతులు:

- **Stakeholder interviews:** PO, users, business team తో మాట్లాడటం.
- **Clarifying questions అడగడం** (అతి ముఖ్యమైన skill): "user email తో login చేస్తారా, phone తోనా? password reset ఉందా? multiple devices?" ఒక vague requirement ని అడిగి-అడిగి concrete చేయడం.
- **User stories + acceptance criteria** (Topic 4): "As a.../I want.../so that..." + Given-When-Then.
- **Prototypes/mockups:** Figma design చూపిస్తే users "ఇది కాదు, ఇలా కావాలి" అని early చెప్తారు (feedback చౌక).
- **Edge cases ముందే గుర్తించడం:** empty state, error state, concurrent access, huge input.

Interview golden rule: system design/coding interview లో ఏ requirement clarify చేయకుండా నేరుగా solve చేయడం మొదలుపెడితే – red flag. Senior candidate ఎప్పుడూ ముందు clarifying questions అడుగుతాడు. Real job లో కూడా అంతే – "ఈ ticket లో 'search add చేయి' అని ఉంది, కానీ fuzzy search? filters? pagination?" అని అడగడం seniority signal.

Requirements ని prioritize చేయడం – MoSCoW

అన్ని requirements ఒకే importance కాదు – వేటిని ముందు build చేయాలి? **MoSCoW** అనే popular technique:

అక్షరం	అర్థం	ఉదా (MVP e-commerce)
M – Must have	ఇది లేకపోతే product పనికిరాదు	login, add to cart, checkout, payment
S – Should have	ముఖ్యం కానీ launch కి critical కాదు	order history, wishlist

అక్షరం	అర్థం	ఉదా (MVP e-commerce)
C – Could have	ఉంటే బాగుంటుంది (nice-to-have)	dark mode, product reviews
W – Won't have (ఇప్పుడు)	ఈ release లో కాదు, తర్వాత	AI recommendations, loyalty points

ఇది **MVP (Minimum Viable Product)** ఆలోచనకి దగ్గర – ముందు "Must have" మాత్రమే ship చేసి, users feedback ప్రకారం మిగతావి జోడించడం. Junior "అన్నీ ఒకేసారి చేద్దాం" అంటాడు (over-scope, late); senior "ముందు core, తర్వాత iterate" అంటాడు. Scope ని prioritize చేయడం – engineering కంటే ముఖ్యమైన product skill.

Design Docs – ఎందుకు, ఏం ఉంటాయి

Non-trivial feature/system ముందు ఒక **design doc** (aka technical design/RFC/one-pager) రాయడం mature teams practice. ఇది "code రాసేముందు ఆలోచనని paper మీద పెట్టడం" – cost of change తక్కువగా ఉన్నప్పుడే (Topic 2 curve) mistakes కనుక్కోవడం.

Typical design doc sections:

- **Context/Problem:** ఏ problem solve చేస్తున్నాం, ఎందుకు.
- **Goals & Non-goals:** ఏం చేస్తాం, ఏం చేయం (scope boundary – over-engineering ఆపుతుంది).
- **Requirements:** FR + NFR.
- **Proposed design:** architecture (HLD), data model, API contracts (LLD), diagrams.
- **Alternatives considered:** ఇతర approaches ఎందుకు reject చేశాం (ఇది అతి ముఖ్యం – reviewers కి thinking కనిపిస్తుంది).
- **Trade-offs & risks:** ఏం compromise చేస్తున్నాం.
- **Rollout/testing plan:** ఎలా ship చేస్తాం, ఎలా verify.

ADR (Architecture Decision Record): ఒక ముఖ్యమైన decision (ఉదా "REST కాదు GraphQL ఎంచుకున్నాం") ని ఎందుకు అనే context తో record చేయడం. 2 సంవత్సరాల తర్వాత "ఎందుకు ఇలా చేశారు?" అని అడిగినప్పుడు answer ఉంటుంది.

HLD vs LLD – design యొక్క రెండు layers

అంశం	HLD (High-Level Design)	LLD (Low-Level Design)
Scope	మొత్తం system architecture	ఒక component/module లోపల
Focus	Services, databases, communication, scale	Classes, functions, schemas, patterns
ప్రశ్న	"Boxes & arrows ఎలా?"	"ఈ box లోపల code ఎలా?"
ఉదా	"React frontend → Node API → MongoDB, Redis cache, S3 for images"	"User class, auth middleware, MongoDB schema, JWT flow"
NFR link	Scalability, availability నిర్ణయిస్తుంది	Maintainability, extensibility నిర్ణయిస్తుంది

అంశం	HLD (High-Level Design)	LLD (Low-Level Design)
Deep dive	`HLD_Telugu.md`, `HLD_Go_Telugu.md`	`LLD_Telugu.md`, `LLD_Go_Telugu.md`

రెండూ ఒకదాన్ని ఒకటి పూర్తి చేస్తాయి: HLD "ఏ services, ఎలా connect" (bird's eye view); LLD "ప్రతి service లోపల classes/APIs ఎలా" (worm's eye view). SSE interview రెండూ అడుగుతుంది – system design round HLD, coding/design round LLD.

MERN team లో ఒక feature యొక్క requirements→design flow

Product team ticket: "users తమ orders track చేయగలగాలి."

1. **Clarify (FR):** ఏ statuses (placed/shipped/delivered)? real-time updates? notifications?
2. **Clarify (NFR):** ఎంతమంది concurrent? ఎంత fast? historical orders ఎన్ని?
3. **User story:** "As a customer, I want to see my order status, so that I know when it arrives." + AC.
4. **HLD:** order-service, MongoDB `orders` collection, WebSocket for real-time, notification-service.
5. **LLD:** `Order` schema (status enum, timestamps), `GET /orders/:id/status` API, status-transition state machine.
6. **Design doc + review** → అప్పుడే code.

ఈ 6 steps skip చేసి నేరుగా code చేసే junior, మధ్యలో "అయ్యో real-time కావాలా?" అని మొత్తం rewrite చేస్తాడు. Senior ముందే clarify చేసి, ఒకసారే సరిగ్గా చేస్తాడు.

Key Points

- **సరైన software > software సరిగ్గా.** Code కి ముందు: ఏం (requirements) + ఎలా (design).
- **FR = system ఏం చేయాలి** (features); **NFR = ఎంత బాగా** (performance, scale, security, "-ilities"). Seniors NFRs ముందే అడుగుతారు – అవి architecture ని నిర్ణయిస్తాయి.
- Requirements ని **clarifying questions + user stories + prototypes** తో elicit చేయాలి; vague → concrete.
- **Design docs** (context, goals/non-goals, design, alternatives, trade-offs) + **ADRs** = cheap phase లో mistakes పట్టుకోవడం.
- **HLD** = system-level (services/DB/scale); **LLD** = component-level (classes/APIs/schemas). రెండూ కావాలి.

Interview దృష్టి

Q: Functional vs non-functional requirements? A: Functional = system ఏం చేయాలి (login, add to cart, place order – behavior). Non-functional = ఎంత బాగా చేయాలి (2s response, 10k concurrent users, 99.9% uptime, security – quality attributes, "-ilities"). Functional features define చేస్తాయి; non-functional architecture ని define చేస్తాయి. అందుకే system design లో నేను ముందు scale, latency, availability వంటి NFRs అడుగుతాను – అవి design ని పూర్తిగా మారుస్తాయి.

Q: Requirements అస్పష్టంగా ఉంటే ఏం చేస్తారు? A: నేరుగా assumptions తో code చేయను – clarifying questions అడుగుతాను (ఎవరు users, edge cases, scale, error handling). అవసరమైతే mockup/prototype చేసి stakeholder feedback తీసుకుంటాను. తప్పు అర్థం చేసుకుని build చేయడం కంటే, ఒక గంట clarify చేయడం చాలా చౌక (cost-of-change curve). Unclear ticket ని clarify చేయడం junior కాదు, senior లక్షణం.

Q: Design doc ఎందుకు రాస్తారు? Code నే చెప్తుంది కదా? A: Code "ఏం చేస్తోందో" చెప్తుంది, కానీ "ఎందుకు ఇలా, ఏ alternatives reject చేశాం, ఏ trade-offs" చెప్పదు. Design doc ఆ reasoning ని capture చేస్తుంది – reviewers early feedback ఇవ్వగలరు (cheap phase లో), కొత్తవాళ్ళు context అర్థం చేసుకోగలరు, future లో "ఎందుకు ఇలా?" కి answer ఉంటుంది (ADR). పైగా రాసేటప్పుడే నా thinking లో gaps నాకే కనిపిస్తాయి.

---

7. Version Control Workflows

వివరణ

Git ఎలా వాడాలో నీకు తెలుసు అనుకుంటున్నా (`git add/commit/push` – basics కోసం companion `Linux_Git_Telugu.md` చూడు). కానీ ఒక *team* లో Git ని ఎలా వాడాలి? ఎప్పుడు branch తీయాలి, ఎప్పుడు merge చేయాలి, ఎవరు main ని touch చేయవచ్చు, releases ఎలా cut చేయాలి – ఈ *team-level* నియమాలనే **branching strategy / Git workflow** అంటారు. Solo project లో ఇది అనవసరం; 8-మంది team లో ఇది లేకపోతే chaos.

Version control ఎందుకు అసలు? మూడు కారణాలు: **(1) History** – ఏ change ఎప్పుడు, ఎవరు, ఎందుకు చేశారో record. **(2) Collaboration** – చాలామంది ఒకే codebase మీద గుద్దుకోకుండా పని చేయడం. **(3) Safety** – ఏదైనా break అయితే గత working version కి rollback.

Real-life Scenario

> **Branch = Google Docs లో "Suggesting mode" copy.** మీ team ఒక పెద్ద document (main codebase) మీద పని చేస్తోంది. అందరూ నేరుగా అదే master doc ని edit చేస్తే – ఒకరి మార్పు ఇంకొకరిది చెరిపేస్తుంది, chaos.
>
> బదులుగా, ప్రతివాడు తన మార్పు కోసం ఒక **copy (branch)** తీసుకుంటాడు, అందులో స్వేచ్ఛగా edit చేస్తాడు (ఎవరికీ ఇబ్బంది లేదు), పూర్తయ్యాక "ఈ మార్పులు master లో కలపండి" అని **request (Pull Request)** పెడతాడు. Editor (reviewer) చూసి, సరిగ్గా ఉంటే approve చేసి master లో merge చేస్తాడు. **అందరూ parallel గా, safely, review తో పని చేయగలరు.** అదే branching.

Workflow 1 – Feature Branch Workflow (అత్యంత common)

అత్యంత widely-used, సులభమైన strategy – ముఖ్యంగా MERN teams కి. Rules:

- `main` (aka master) ఎప్పుడూ **stable, deployable** గా ఉంటుంది. ఎవరూ నేరుగా main కి push చేయరు (branch protection).
- ప్రతి feature/bug కి **main నుండి కొత్త branch** (`feature/wishlist` , `fix/cart-bug`).

- ఆ branch లో పని చేసి, commits చేసి, push చేసి, **Pull Request (PR)** raise చేస్తారు.
- Review + CI pass అయ్యాక **main లోకి merge.** Branch delete.

```
main:     ●—●———————————●———————————●—→ (ఎప్పుడూ deployable)
           \               /           /
feature/A:  ●—●—●————————/ (PR merged)   /
                 \                      /
feature/B:        ●—●—●————————————/ (PR merged)
       (ప్రతి feature isolated branch → PR → review → merge)
```

- **మంచిది:** సులభం, isolated work, review built-in, CI ప్రతి PR మీద run అవుతుంది.
- **ఎప్పుడు:** దాదాపు అన్ని modern web/product teams. Default choice.

Workflow 2 – Gitflow

పెద్ద, scheduled releases ఉన్న projects కి designed. చాలా branches:

- `main` – production releases (tagged versions) మాత్రమే.
- `develop` – integration branch, next release కోసం అన్ని features ఇక్కడ కలుస్తాయి.
- `feature/*` – develop నుండి, develop కి తిరిగి.
- `release/*` – release prep (bug fixes, version bump).
- `hotfix/*` – production లో urgent fix (main నుండి నేరుగా).

```
main:     ●———————————————————●(v1.0)——————————●(v1.1)
           \                 /  \              /
release:    \               ●———\             ●
             \             /     \           /
develop:  ●—●—●—●—●┘       ●—●—●┘
           \   /  \  /
feature:    ●—●    ●-●
```

- **మంచిది:** versioned software (mobile apps, desktop, libraries), multiple environments, scheduled releases.
- **చెడ్డది:** complex, చాలా branches, merge overhead ఎక్కువ, continuous deployment కి సరిపోదు (too slow). **Web SaaS కి overkill** – original author కూడా "మీరు continuous delivery చేస్తే Gitflow వాడకండి" అన్నాడు.

Workflow 3 – Trunk-Based Development (TBD)

Modern high-velocity teams (Google, Facebook) వాడేది. Rules:

- అందరూ **ఒకే branch (trunk/main)** మీద పని చేస్తారు.
- Branches చాలా **short-lived** (కొన్ని గంటలు/రోజు), వెంటనే main లో merge.
- Incomplete features ని **feature flags** వెనుక దాచుతారు (code main లో ఉంది కానీ off).
- ప్రతి commit main కి → strong CI/CD + automated tests తప్పనిసరి.

```
main: ●━●━●━●━●━●━●━●━●━●━► (అందరూ చిన్న commits, రోజూ merge)
        ↑ ↑   ↑ ↑ ↑   ↑ ↑
    (short-lived branches, గంటల్లో merge; incomplete = feature flag వెనుక)
```

- **మంచిది:** fast integration, "merge hell" లేదు, continuous deployment కి perfect, feature flags తో safe.
- **చెడ్డది:** strong test automation + discipline + feature flags infra తప్పనిసరి; లేకపోతే main break అవుతుంది.
- **ఎప్పుడు:** mature teams, strong CI/CD, frequent deploys.

మూడింటి పోలిక

అంశం	Feature Branch	Gitflow	Trunk-Based
Branches	main + short feature branches	main, develop, feature, release, hotfix	main + very short branches
Branch lifetime	రోజులు	వారాలు	గంటలు
Complexity	తక్కువ	ఎక్కువ	తక్కువ (కానీ discipline ఎక్కువ)
Release style	Continuous / on-demand	Scheduled versions	Continuous deployment
Merge conflicts	Medium	ఎక్కువ (long branches)	తక్కువ (short branches)
CI/CD fit	మంచిది	సరిపోదు (slow)	**అద్భుతం**
Feature flags	optional	అరుదు	తప్పనిసరి
Best for	చాలా web/product teams	versioned/scheduled releases	high-velocity, strong CI teams

Pull Requests (PR) – team collaboration యొక్క గుండె

PR (GitHub) / Merge Request (GitLab) = "నా branch ని main లో merge చేయమని request." ఇది కేవలం merge button కాదు – ఇది team యొక్క **quality gate:**

- **Code review:** ఇద్దరు peers నీ code చదివి feedback ఇస్తారు (Topic 10 deep).
- **CI checks:** automated tests, lint, build ప్రతి PR మీద run (Topic 11).
- **Discussion:** ఎందుకు ఇలా చేశావ్, alternatives – thread లో.
- **Record:** ఈ change ఎందుకు జరిగింది అనేది forever documented.

మంచి PR లక్షణాలు:

- **చిన్నది (small):** 200-400 lines max. పెద్ద PR review చేయడం కష్టం, bugs miss అవుతాయి. "1 PR = 1 logical change."

- **మంచి description:** ఏం మార్చావ్, ఎందుకు, ఎలా test చేశావ్, screenshots (UI అయితే).
- **Green CI:** tests pass అయ్యాకే review అడుగు.
- **Self-review:** raise చేసేముందు నీ diff నువ్వే ఒకసారి చదువు – silly mistakes పట్టుకో.

Commit hygiene – చిన్నదైనా ముఖ్యం

- **Atomic commits:** ఒక commit = ఒక logical change. "fix stuff" కాదు; "Fix null check in cart total calculation."
- **మంచి commit messages:** imperative mood ("Add wishlist API" not "Added"), body లో ఎందుకు.
- **Conventional Commits** (many teams): `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:` prefixes – automated changelog/versioning కి.

Semantic Versioning (SemVer) – version numbers అర్థం

Software releases ని `MAJOR.MINOR.PATCH` (ఉదా `2.4.1`) గా version చేస్తారు – దీన్ని **SemVer** అంటారు. ప్రతి number ఒక అర్థం:

భాగం	ఎప్పుడు పెంచాలి	ఉదా
MAJOR (2.x.x)	**breaking change** – పాత code break అవుతుంది	API endpoint తీసేశారు → `3.0.0`
MINOR (x.4.x)	కొత్త feature, backward-compatible	కొత్త optional endpoint → `2.5.0`
PATCH (x.x.1)	bug fix, backward-compatible	ఒక bug fix → `2.4.2`

ఎందుకు ముఖ్యం? నీ MERN app npm packages మీద ఆధారపడుతుంది – `package.json` లో `^2.4.1` అంటే "2.x.x లో ఏదైనా OK, కానీ 3.0.0 (breaking) కాదు." SemVer వల్ల `npm install` safe గా updates తీసుకుంటుంది, break చేయకుండా. Conventional Commits (పైన) SemVer ని automate చేయగలవు – `feat:` → MINOR bump, `fix:` → PATCH, `BREAKING CHANGE:` → MAJOR.

(Git commands, rebase vs merge, conflict resolution deep dive → companion `Linux_Git_Telugu.md`. ఇక్కడ *team workflow* strategy focus.)

Real MERN team లో

చాలావరకు **Feature Branch + PR** workflow: main protected, ప్రతి Jira ticket కి `feature/PROJ-123-wishlist` branch, PR raise, 2 approvals + green CI అవసరం, squash-merge to main, auto-deploy to staging. కొన్ని advanced teams **Trunk-Based + feature flags** (LaunchDarkly వంటివి) వాడతాయి fast deploys కోసం. Gitflow మాత్రం web SaaS లో తగ్గిపోతోంది (mobile/versioned products లో ఇంకా ఉంది).

Key Points

- **Branching strategy = team level Git నియమాలు** – ఎప్పుడు branch/merge, ఎవరు main touch చేయవచ్చు.
- **Feature Branch** (main stable + short feature branches + PR) = default, చాలా web teams.

- **Gitflow** (main/develop/feature/release/hotfix) = scheduled/versioned releases; web SaaS కి overkill.
- **Trunk-Based** (అందరూ main మీద, short branches, feature flags) = high-velocity + strong CI teams.
- **PR = quality gate:** review + CI + discussion + record. మంచి PR = చిన్నది, well-described, green CI, self-reviewed.
- **Commit hygiene:** atomic commits, imperative messages, conventional commit prefixes.

Interview దృష్టి

Q: మీ team ఏ branching strategy వాడుతుంది? A: Feature Branch workflow – main ని protected & always-deployable గా ఉంచుతాం, ప్రతి ticket కి main నుండి short-lived feature branch తీసి, PR raise చేస్తాం. PR కి 2 approvals + green CI అవసరం, తర్వాత squash-merge. ఇది isolation, review, CI ని natural గా enforce చేస్తుంది. Fast deploys కావాలంటే trunk-based + feature flags కూడా considered.

Q: Gitflow vs Trunk-based – ఎప్పుడు ఏది? A: Gitflow scheduled, versioned releases (mobile apps, libraries) కి – multiple long-lived branches, release/hotfix branches. కానీ web SaaS లో continuous deployment కి అది too slow, merge-heavy. Trunk-based అందరూ main మీద short branches తో పని చేసి, incomplete work ని feature flags వెనుక దాచి, రోజూ deploy చేయడానికి – కానీ strong CI/CD + test automation తప్పనిసరి. Velocity + CI maturity ఎక్కువ ఉంటే trunk-based, scheduled releases అయితే Gitflow.

Q: మంచి PR ఎలా ఉంటుంది? A: చిన్నది (200-400 lines, ఒక logical change), స్పష్టమైన description (ఏం/ఎందుకు/ఎలా test చేశా), green CI, raise చేసేముందు self-reviewed. పెద్ద PRs review చేయడం కష్టం, bugs దాటిపోతాయి. చిన్న PRs = వేగంగా review, తక్కువ bugs, సులభమైన rollback.

---

8. Clean Code & Code Quality

వివరణ

Topic 1 లో నేర్చుకున్న ఒక్క నిజం మళ్ళీ: **code write కంటే 10x ఎక్కువ read అవుతుంది.** అంటే code యొక్క అసలు audience compiler కాదు – మనుషులు (future నువ్వు, teammates). Clean code అంటే – పని చేసే code కాదు (అది minimum); **ఇంకో engineer సులభంగా చదివి, అర్థం చేసుకుని, safely మార్చగలిగే code.** ఇదే junior/senior మధ్య అతిపెద్ద visible తేడా.

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." – Martin Fowler. ఈ ఒక్క వాక్యం clean code యొక్క సారాంశం.

Real-life Scenario

> **Clean code = బాగా organize చేసిన వంటగది. Dirty code = గజిబిజి గది.**
>
> బాగా organize చేసిన kitchen లో – ప్రతి వస్తువుకి label, దగ్గర దగ్గర సంబంధిత వస్తువులు, కొత్త cook కూడా వెంటనే వంట మొదలుపెట్టగలడు. అదే గజిబిజి గదిలో – ఉప్పు ఎక్కడో, కారం ఇంకెక్కడో, ప్రతి వస్తువు వెతకడానికే గంట. వంట (feature) అదే – కానీ organization లేకపోతే ప్రతి పని 10x slow, error-prone.
>
> **నువ్వు రాసిన code ని maintain చేసేవాడు నీ address తెలిసిన violent psychopath అనుకుని రాయి** – అనేది ఒక పాత joke. అంటే: రేపు దీన్ని చదివేవాడికి (బహుశా నువ్వే, 6 నెలల తర్వాత, context మర్చిపోయి) కోపం రాకుండా, స్పష్టంగా రాయి.

Clean Code – practical rules

1. Meaningful names (అతి ముఖ్యం):

```
// చెడ్డది
const d = 86400000;
function calc(a, b) { return a * b * 0.05; }

// మంచిది
const MS_PER_DAY = 86400000;
function calculateTax(price, quantity) { return price * quantity * TAX_RATE; }
```

Names "ఏమిటి, ఎందుకు" చెప్పాలి. `data`, `temp`, `x`, `handleStuff` – అర్థం లేని names. Comment అవసరమైతే, బహుశా name సరిపోలేదు. Booleans కి `is/has/should` (isActive, hasPermission).

2. చిన్న functions, ఒకటే పని (Single Responsibility):

- ఒక function ఒకటే పని చేయాలి, ఒకటే abstraction level లో.
- 20-30 lines దాటితే – బహుశా అది చాలా పనులు చేస్తోంది, split చేయి.
- ఒక function పేరు "and" తో ఉంటే (`validateAndSave`) → అది రెండు functions అవ్వాలి.

3. Function arguments తక్కువ: 0-2 ideal, 3 border, 4+ అయితే object గా group చేయి. Boolean flag argument (`render(true)`) చెడ్డది – రెండు functions గా విడగొట్టు.

4. Comments – ఎప్పుడు, ఎప్పుడు కాదు:

- **మంచి comment:** *ఎందుకు* (why) explain చేస్తుంది – "ఈ retry 3 సార్లు ఎందుకంటే payment gateway తరచు timeout అవుతుంది."
- **చెడ్డ comment:** *ఏం* (what) చెప్తుంది – `i++; // increment i` (code నే చెప్తోంది, redundant).
- **అత్యంత చెడ్డ comment:** అబద్ధం చెప్పేది – code మారింది కానీ comment పాతది. Wrong comment > no comment.
- **లక్ష్యం:** code self-documenting గా ఉండాలి; comments *reasoning/context/warnings* కోసమే, code ని describe చేయడానికి కాదు.

5. Consistent formatting: ఒకే indentation, spacing, brace style. దీన్ని manually చేయకు – **Prettier (formatter) + ESLint (linter)** ని CI లో పెట్టు, arguments అవసరం లేదు.

మూడు మంత్రాలు – DRY, KISS, YAGNI

Principle	పూర్తి పేరు	అర్థం	ఉదాహరణ
DRY	Don't Repeat Yourself	ఒకే logic ని రెండు చోట్ల copy చేయకు – ఒక function/module గా extract చేయి. మారితే ఒకే చోట మారుస్తావు.	Validation logic 3 files లో copy – ఒక `validateEmail()` గా.
KISS	Keep It Simple, Stupid	సులభమైన solution ఎంచుకో. Clever, complex కంటే boring, obvious better.	3 design patterns కలిపి రాయడం కంటే, ఒక simple loop.
YAGNI	You Aren't Gonna Need It	ఇప్పుడు అవసరం లేని feature/flexibility ముందే build చేయకు.	"future లో 10 payment gateways కావచ్చు" అని ఇప్పుడే abstract factory రాయకు – ఇప్పుడు ఒకటే ఉంది.

సూక్ష్మం (nuance): ఈ మూడూ గుడ్డిగా apply చేయవద్దు. **DRY ని అతిగా చేస్తే** – పోలికున్న కానీ వేరే reasons ఉన్న రెండు codes ని బలవంతంగా కలిపేస్తావు, తర్వాత ఒకటి మారాలంటే రెండోది break అవుతుంది ("wrong abstraction is worse than duplication" – Sandi Metz). **YAGNI ని అతిగా చేస్తే** – అస్సలు extensibility లేని rigid code. Clean code అంటే rules ని blindly follow చేయడం కాదు – *judgment.*

SOLID – quick recap (deep dive `LLD_Telugu.md`)

Object-oriented design యొక్క 5 principles. ఇక్కడ ఒక్క line గుర్తు; పూర్తి `LLD_Telugu.md`, `LLD_Go_Telugu.md` లో:

అక్షరం	పేరు	ఒక్క line
S	Single Responsibility	ఒక class కి ఒకటే మారడానికి కారణం ఉండాలి.
O	Open/Closed	Extension కి open, modification కి closed (కొత్త behavior add, పాత code touch చేయకుండా).
L	Liskov Substitution	Subclass ని parent స్థానంలో పెట్టినా code break అవ్వకూడదు.
I	Interface Segregation	పెద్ద interface కంటే, చిన్న specific interfaces (అవసరం లేని methods force చేయకు).
D	Dependency Inversion	Concrete classes మీద కాదు, abstractions (interfaces) మీద depend అవ్వు.

SOLID = code ని **maintainable, extensible, testable** గా ఉంచే guidelines. ఇవి "law" కాదు, "guardrails."

Tech Debt – అర్థం, ఎప్పుడు OK, ఎలా manage

Technical Debt (tech debt) = ఇప్పుడు వేగంగా ship చేయడానికి తీసుకున్న shortcut, తర్వాత "వడ్డీ"తో తిరిగి చెల్లించాల్సిన compromise. Financial debt లాంటిదే – కొన్నిసార్లు అవసరం (deadline), కానీ

పేరుకుపోతే crippling.

```
Tech debt యొక్క "వడ్డీ":
  Shortcut తీసుకున్న రోజు:  ⚡ fast ship
  తర్వాత ప్రతి change:      🐌 slow (గజిబిజి code చుట్టూ జాగ్రత్త)
  పేరుకుపోతే:              🔥 ప్రతి feature painful, bugs, team demoralized
```

రకాలు:

- **Deliberate (ఉద్దేశపూర్వక):** "deadline కోసం ఇప్పుడు shortcut, తర్వాత fix చేద్దాం" – OK, *tracked* అయితే.
- **Accidental (అనుకోని):** knowledge లేక చెడ్డ design – తర్వాత refactor.
- **Bit rot:** code కాలంతో పాటు క్షీణించడం (dependencies పాతవి, patches మీద patches).

ఎలా manage: (1) tech debt ని *visible* చేయి – backlog లో tickets గా track చేయి, దాచొద్దు. (2) ప్రతి sprint లో కొంత % debt కి కేటాయించు ("boy scout rule" – touch చేసిన code ని కొంచెం clean గా వదులు). (3) Interest ఎక్కువ ఉన్న debt (తరచు touch అయ్యే hot code) ముందు fix. **గుర్తుంచుకో:** అన్ని debt చెడ్డది కాదు; *untracked, ignored* debt చెడ్డది.

Refactoring – behavior మార్చకుండా structure మెరుగుపరచడం

Refactoring = code యొక్క external behavior మార్చకుండా, internal structure ని మెరుగుపరచడం. Output అదే; code cleaner. (Feature add చేయడం కాదు – అది వేరే activity.)

- **ఎప్పుడు:** ఒక feature add చేసేముందు code ని refactor చేసి, తర్వాత feature add ("make the change easy, then make the easy change" – Kent Beck). Boy scout rule.
- **Safety net:** refactor చేసేముందు **tests ఉండాలి** – లేకపోతే accidentally behavior మార్చావని తెలియదు. Tests green ఉంటే, structure ఎంత మార్చినా safe.
- **చిన్న steps:** rename → extract function → move → simplify. ప్రతి step తర్వాత tests run.
- **Big rewrite trap:** "ఇదంతా చెత్త, from scratch రాద్దాం" – చాలావరకు disaster (Netscape rewrite famous failure). Incremental refactor > big rewrite.

Code Smells – "ఇక్కడ refactor అవసరం" అని చెప్పే signals

Code smell = అది bug కాదు, కానీ "ఇక్కడ design problem ఉంది" అని సూచించే లక్షణం (నిజమైన వాసన కాదు, metaphor). వీటిని గుర్తిస్తే refactor ఎప్పుడు అవసరమో తెలుస్తుంది:

Smell	ఏమిటి	ఎందుకు చెడ్డది
Long function	50+ lines, చాలా పనులు	అర్థం కష్టం, test కష్టం → split
Large class (God object)	ఒక class అంతా చేస్తుంది	SRP violation → responsibilities విడగొట్టు
Duplicate code	ఒకే logic చాలా చోట్ల	ఒకటి మారితే మిగతావి miss → DRY (judgment తో)
Long parameter list	4+ arguments	call చేయడం, గుర్తుంచుకోవడం కష్టం → object గా group

Smell	ఏమిటి	ఎందుకు చెడ్డది
Magic numbers	`if (status === 3)`	3 అంటే ఏమిటి? → named constant (`STATUS_SHIPPED`)
Deep nesting	4+ levels if/for	చదవడం కష్టం → early return, extract
Comments explaining bad code	comment లేకుండా అర్థంకాదు	code నే స్పష్టం చేయి, comment అవసరం రాకుండా

Boy Scout Rule (గుర్తుంచుకో): "Always leave the code cleaner than you found it." నువ్వు ఒక file touch చేసినప్పుడు, ఒక చిన్న smell fix చేసి వదులు (rename, extract, magic number). ఇలా చిన్న చిన్న cleanups కలిసి codebase ని కాలంతో పాటు మెరుగుపరుస్తాయి – పెద్ద "refactoring sprint" అవసరం రాదు.

Real MERN team లో

PR review లో reviewer "ఈ function 80 lines, split చేయవచ్చా? ఈ variable name `x` బదులు `userCount`? ఈ duplicate validation extract చేయవచ్చా?" అని అడుగుతాడు. CI లో ESLint + Prettier automatic గా style enforce చేస్తుంది. Backlog లో "Tech Debt: refactor auth middleware" tickets ఉంటాయి. కొత్త feature రాసేముందు "boy scout rule" – touch చేసిన file ని కొంచెం clean గా వదులు. ఇవన్నీ కలిసి codebase ని 2 సంవత్సరాల తర్వాత కూడా workable గా ఉంచుతాయి.

Key Points

- **Clean code = మనుషులు చదవగలిగే code.** Audience = future నువ్వు + teammates, compiler కాదు.
- Practical: **meaningful names, చిన్న single-purpose functions, తక్కువ args, why-comments (what కాదు), consistent formatting** (Prettier + ESLint).
- **DRY** (repeat చేయకు), **KISS** (simple గా ఉంచు), **YAGNI** (అవసరం లేనిది build చేయకు) – కానీ blindly కాదు, judgment తో (over-DRY = wrong abstraction).
- **SOLID** = OO maintainability guardrails (deep: `LLD_Telugu.md`).
- **Tech debt** = speed కోసం తీసుకున్న shortcut; tracked & deliberate అయితే OK, untracked/ignored అయితే crippling. Visible చేసి manage చేయి.
- **Refactoring** = behavior మార్చకుండా structure మెరుగుపరచడం; tests safety net తో, చిన్న steps లో, big rewrite కాదు.

Interview దృష్టి

Q: Clean code అంటే మీకు ఏమిటి? A: Compiler కి కాదు, మనుషులకి – ముఖ్యంగా future teammates కి – చదవడానికి, అర్థం చేసుకోవడానికి, safely మార్చడానికి సులభమైన code. Meaningful names, చిన్న single-purpose functions, why explain చేసే comments, consistency. Code write కంటే చాలా ఎక్కువ read అవుతుంది, కాబట్టి readability = long-term productivity. Clever one-liner కంటే boring-but-obvious code నేను prefer చేస్తాను.

Q: DRY ని ఎప్పుడైనా violate చేస్తారా? A: అవును, ఉద్దేశపూర్వకంగా. రెండు code blocks పైకి పోలికున్నా, వేర్వేరు business reasons ఉంటే – వాటిని బలవంతంగా కలపడం "wrong abstraction" సృష్టిస్తుంది;

తర్వాత ఒకటి మారాలంటే రెండోది break అవుతుంది. "Duplication is cheaper than the wrong abstraction." కాబట్టి DRY ని judgment తో apply చేస్తాను, గుడ్డిగా కాదు.

Q: Tech debt ని ఎలా handle చేస్తారు? A: మొదట దాన్ని visible చేస్తాను – backlog లో tickets గా track, దాచను. Deadline కోసం deliberate shortcut OK, కానీ "తర్వాత fix చేద్దాం" అనేది documented అవ్వాలి. తరచు touch అయ్యే hot code లో debt (ఎక్కువ "interest") ముందు fix చేస్తాను. ప్రతి sprint కొంత capacity debt కి కేటాయిస్తాం, boy scout rule follow చేస్తాం. అన్ని debt చెడ్డది కాదు – ignored, untracked debt మాత్రమే.

Q: పెద్ద legacy code refactor ఎలా? A: Big-bang rewrite avoid చేస్తాను (అది చాలావరకు fail అవుతుంది). ముందు refactor చేయబోయే code చుట్టూ tests రాస్తాను (safety net), తర్వాత చిన్న చిన్న steps (rename → extract → simplify), ప్రతి step తర్వాత tests green అని confirm. Behavior మారకూడదు, structure మెరుగవ్వాలి. Feature add చేయాల్సి వస్తే – ముందు refactor to make it easy, తర్వాత add.

Part 3 — Quality (నాణ్యత)

> Code రాశాం; అది సరిగ్గా పని చేస్తుందని ఎలా నమ్మకం? Break అవ్వకుండా ఎలా మారుస్తాం? ఈ Part లో: automated testing (unit/integration/e2e, test pyramid, TDD, mocking, Jest) మరియు code review — team గా quality ని కాపాడే రెండు అతి ముఖ్యమైన practices. Self-taught devs ఎక్కువగా skip చేసేవి ఇవే; SSE interviews & real teams అత్యధికంగా expect చేసేవి ఇవే.

---

9. Testing — Deep Dive

వివరణ

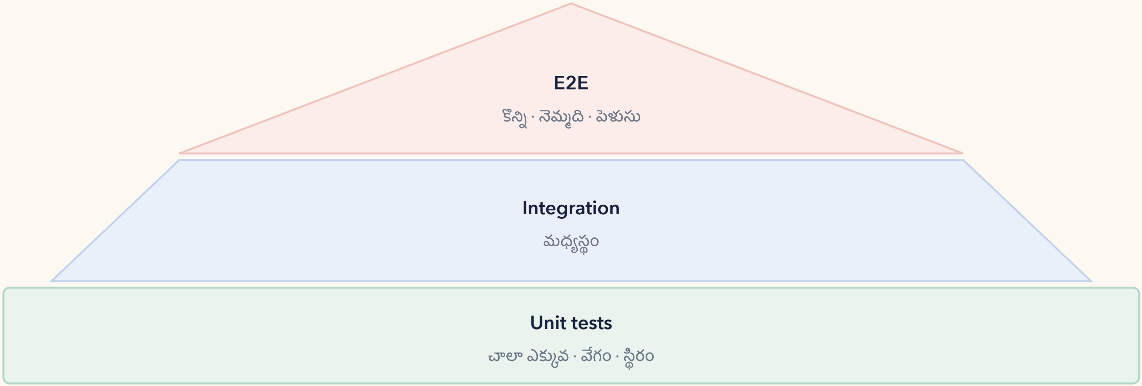
చాలామంది self-taught devs testing ని "అదనపు పని, deadline ఉంది skip చేద్దాం" అనుకుంటారు. కానీ real teams లో **tests లేని code = production కి రాకూడని code.** ఎందుకు? ఎందుకంటే tests రెండు అమూల్యమైన విషయాలు ఇస్తాయి: **(1) confidence** — నా code సరిగ్గా పని చేస్తోందని proof; **(2) safety net** — రేపు ఇంకొకడు (లేదా నేనే) ఈ code మార్చినప్పుడు, ఏదైనా break అయితే వెంటనే తెలుస్తుంది.

రెండో point అతి ముఖ్యం. Topic 8 లో చెప్పుకున్నాం — code ఎప్పుడూ మారుతుంది (refactor, feature add). Tests లేకపోతే ప్రతి change ఒక భయం ("ఇది ఏదైనా break చేసిందా?"). Tests ఉంటే — change చేసి, tests run చేసి, green అయితే confident. **Tests అనేది కేవలం bug-catching కాదు — అవి change ని fearless గా చేసే freedom.**

Manual testing (ప్రతిసారి UI లో click చేసి చూడటం) scale అవ్వదు — 500 features ఉన్న app ని ప్రతి deploy కి manually test చేయలేం. అందుకే **automated tests** — code రాసి, machine వాటిని seconds లో run చేస్తుంది.

TESTING PYRAMID · ఎన్ని, ఏ స్థాయిలో

TESTING PYRAMID



ఎందుకు ఈ ఆకారం

E2E tests నమ్మకం ఎక్కువ ఇస్తాయి కానీ — నెమ్మది, తరచుగా విఫలం (flaky), maintain కష్టం.
Unit tests వేగం, ఖచ్చితంగా ఎక్కడ విరిగిందో చెప్తాయి.
తలక్రిందుల pyramid (ఎక్కువ E2E) = నెమ్మది CI + ఎవరూ నమ్మని tests.

Real-life Scenario

> **Tests = ఇంటికి పెట్టిన smoke detectors + burglar alarms.**
>
> నువ్వు ఇల్లు కట్టేటప్పుడు smoke detectors పెట్టడం "అదనపు పని, ఖర్చు" అనిపిస్తుంది. కానీ ఒక రాత్రి, నువ్వు నిద్రలో ఉండగా, kitchen లో చిన్న మంట మొదలైతే – detector వెంటనే alarm మోగించి నిన్ను కాపాడుతుంది.
>
> Tests అలాంటివే. నువ్వు ఒక feature రాశాక "పని చేస్తోంది కదా, tests ఎందుకు?" అనిపిస్తుంది. కానీ 3 నెలల తర్వాత, ఇంకో engineer అస్సలు సంబంధం లేని ఒక file మార్చినప్పుడు, అనుకోకుండా నీ feature break అవుతుంది. **Test ఆ క్షణం alarm మోగిస్తుంది** – "ఈ change order-total calculation ని పాడు చేసింది!" – production కి వెళ్ళక ముందే. Detector లేకపోతే? మంట (bug) users ఇంట్లో (production లో) కనబడుతుంది.

ఎందుకు test – నాలుగు కారణాలు

1. **Confidence:** code సరిగ్గా పని చేస్తోందని proof (edge cases తో సహా).
2. **Regression safety:** పాత feature కొత్త change వల్ల break అవ్వలేదని guarantee. ("Regression" = పని చేసేది break అవ్వడం.)
3. **Living documentation:** tests చదివితే "ఈ function ఎలా వాడాలి, ఏం చేస్తుంది" అర్థమవుతుంది – code కి executable spec.
4. **Better design:** testable code రాయడానికి, code ని loosely-coupled గా (small functions, dependency injection) రాయాల్సి వస్తుంది → automatically better design.

The Test Pyramid (అతి ముఖ్యమైన mental model)

ఏ types tests ఎన్ని రాయాలి? దీన్ని **Test Pyramid** (Mike Cohn) చెప్తుంది:

```
              /\
             /  \            E2E Tests (కొన్ని)
            / E2E\           - full app, real browser
           /──────\          - slow, expensive, brittle
          /        \         - high confidence (user flow)
         /Integration\       Integration Tests (కొంత)
        /─────────────\      - modules కలిసి పని చేస్తాయా?
       /               \     - DB, API together
      /   Unit Tests    \    Unit Tests (చాలా – base)
     /───────────────────\   - ఒక function/module isolated
    /_____________________\  - fast, cheap, stable
```

నియమం: కింద ఎక్కువ (చాలా unit tests), పైకి వెళ్ళే కొద్దీ తక్కువ. ఎందుకు?

Type	ఏం test చేస్తుంది	వేగం	ఖరీదు	ఎన్ని	Confidence
Unit	ఒక function/module isolated	చాలా fast (ms)	చౌక	చాలా (70%)	ఒక unit correct
Integration	Modules కలిసి (API+DB)	medium	medium	కొంత (20%)	pieces కలిసి పని

Type	ఏం test చేస్తుంది	వేగం	ఖరీదు	ఎన్ని	Confidence
E2E	Full app, user దృష్టిలో	slow (seconds)	ఖరీదు, brittle	కొన్ని (10%)	పూర్తి flow పని

ఎందుకు pyramid, cone కాదు? E2E tests high confidence ఇస్తాయి (real user flow) కానీ slow, ఖరీదు, brittle (చిన్న UI మార్పుకి break). చాలా E2E tests ఉంటే – CI గంటలు పడుతుంది, tests అస్థిరం (flaky), team వాటిని నమ్మడం మానేస్తుంది. అందుకే: **fast, stable unit tests చాలా; slow, brittle E2E కొన్ని కీలక flows కి మాత్రమే.** (Anti-pattern: "ice cream cone" – ఎక్కువ manual/E2E, తక్కువ unit – slow & painful.)

Unit vs Integration vs E2E – MERN ఉదాహరణతో

- **Unit:** `calculateOrderTotal(items)` function సరిగ్గా total లెక్కిస్తుందా? (DB లేదు, network లేదు – pure logic.)
- **Integration:** `POST /orders` API నిజంగా MongoDB లో order save చేస్తుందా? (API + DB కలిసి – test DB వాడతారు.)
- **E2E:** User browser లో product → add to cart → checkout → order confirmation చూస్తాడా? (Cypress/Playwright తో నిజమైన browser drive చేస్తారు.)

TDD – Test-Driven Development (Red-Green-Refactor)

TDD = code రాసేముందు **test ముందు** రాయడం. మూడు steps loop:

```
   ┌──────────────────────────────────────────────┐
   │                                              │
   ▼                                              │
🔴 RED           →    🟢 GREEN          →    🔵 REFACTOR ┘
test రాయి,           test pass అయ్యే           code clean చేయి
అది fail అవుతుంది     minimum code రాయి        (tests green ఉంచుతూ)
(feature లేదు)       (అందంగా కాదు, just pass)
```

1. **Red:** ఇంకా లేని feature కి test రాయి. Run చేయి – fail (red) అవుతుంది (feature లేదు కాబట్టి).
2. **Green:** ఆ test pass అయ్యే కనీస code రాయి. అందంగా ఉండనవసరం లేదు – just green.
3. **Refactor:** ఇప్పుడు test safety net ఉంది కాబట్టి, code ని clean గా refactor చేయి. Tests green ఉంటే safe.
4. Repeat తర్వాతి requirement కి.

ఎందుకు TDD? (a) రాసేముందే "ఏం build చేస్తున్నా" clear అవుతుంది (test = spec). (b) 100% testable code (test-first కాబట్టి). (c) over-engineering ఆగుతుంది (test pass అయితే చాలు – YAGNI). (d) Refactor confidence. **అందరూ TDD strict గా follow చేయరు**, కానీ దాని philosophy (tests తో పాటు రాయడం, తర్వాత కాదు) విలువైనది.

Mocking & Stubbing – dependencies ని fake చేయడం

Unit test *isolated* గా ఉండాలి – కానీ నీ function ఒక database/API/email service ని call చేస్తే? ప్రతి test కి నిజమైన DB hit చేయడం slow + unreliable. పరిష్కారం – ఆ dependencies ని **fake (mock/stub)**

చేయడం:

- **Stub:** ఒక dependency కి fixed answer ఇచ్చే fake. ("`getUser()` ని call చేస్తే ఎప్పుడూ ఈ dummy user ఇవ్వు.")
- **Mock:** stub + అది *ఎలా called అయ్యిందో* verify చేసేది. ("`sendEmail()` సరిగ్గా ఒకసారి, ఈ arguments తో called అయ్యిందా?")
- **Fake:** simplified working implementation (ఉదా: real DB బదులు in-memory object).
- **Spy:** real function ని wrap చేసి, calls record చేసేది.

ఎందుకు? **(1) Speed** – network/DB లేదు. **(2) Isolation** – నా function లో bug ఉందా, DB లో bug ఉందా అని confuse అవ్వను. **(3) Control** – error scenario ని simulate చేయవచ్చు ("API 500 return చేస్తే నా code ఎలా handle చేస్తుంది?"). **Warning:** అతిగా mock చేస్తే – tests real behavior ని test చేయవు, mocks ని మాత్రమే test చేస్తాయి (false confidence). Balance ముఖ్యం.

Code Coverage – ఎంత code test అయ్యింది

Coverage = నీ tests ఎంత % code execute చేస్తాయి అనే metric (line, branch, function coverage). Tools: Jest `--coverage`, Istanbul.

- **ఉపయోగం:** test చేయని code (0% coverage areas) ని surface చేస్తుంది.
- **Trap (అతి ముఖ్యం): 100% coverage ≠ bug-free.** Coverage అంటే "ఈ line run అయ్యింది" మాత్రమే – "ఈ line సరిగ్గా పని చేస్తోంది" కాదు. Assertions లేని test కూడా coverage పెంచుతుంది కానీ ఏమీ verify చేయదు. Coverage ని *target* చేస్తే (mgmt "80% కావాలి" అంటే) – devs meaningless tests రాసి number పెంచుతారు (Goodhart's Law మళ్ళీ). Coverage = *guide,* goal కాదు. ~70-80% reasonable; 100% obsession waste.

Jest Example – ఒక Node function ని test చేయడం

ఒక real MERN backend function ని unit test చేద్దాం. **Jest** = JavaScript/Node లో అత్యంత popular test framework.

```
// ── orderUtils.js (test చేయాల్సిన code) ──────────────────────
function calculateOrderTotal(items, taxRate = 0.05) {
  if (!Array.isArray(items)) {
    throw new TypeError("items must be an array");
  }
  const subtotal = items.reduce((sum, item) => {
    return sum + item.price * item.quantity;
  }, 0);
  const tax = subtotal * taxRate;
  return Math.round((subtotal + tax) * 100) / 100; // 2 decimals
}

module.exports = { calculateOrderTotal };
```

```
// ── orderUtils.test.js (Jest tests) ──────────────────────────
const { calculateOrderTotal } = require("./orderUtils");

describe("calculateOrderTotal", () => {
```

```
  // 1. Happy path — సాధారణ case
  test("subtotal + tax ని సరిగ్గా లెక్కిస్తుంది", () => {
    const items = [
      { price: 100, quantity: 2 }, // 200
      { price: 50, quantity: 1 },  // 50
    ];
    // subtotal = 250, tax @5% = 12.5, total = 262.5
    expect(calculateOrderTotal(items)).toBe(262.5);
  });

  // 2. Edge case — ఖాళీ cart
  test("ఖాళీ array కి 0 return చేస్తుంది", () => {
    expect(calculateOrderTotal([])).toBe(0);
  });

  // 3. Custom tax rate
  test("custom tax rate ని గౌరవిస్తుంది", () => {
    const items = [{ price: 100, quantity: 1 }];
    expect(calculateOrderTotal(items, 0.18)).toBe(118); // 18% GST
  });

  // 4. Rounding — floating point జాగ్రత్త
  test("2 decimals కి round చేస్తుంది", () => {
    const items = [{ price: 9.99, quantity: 3 }]; // 29.97
    // total = 29.97 * 1.05 = 31.4685 → 31.47
    expect(calculateOrderTotal(items)).toBe(31.47);
  });

  // 5. Error case — invalid input
  test("array కానిదానికి TypeError throw చేస్తుంది", () => {
    expect(() => calculateOrderTotal("not-array")).toThrow(TypeError);
  });
});
```

గమనిక – మంచి test suite: **happy path + edge cases (empty) + boundary (rounding) + error cases** అన్నీ cover చేస్తుంది. `describe` group చేస్తుంది; `test` / `it` ఒక్క case; `expect(...).toBe(...)` assertion. **AAA pattern:** Arrange (data setup) → Act (function call) → Assert (verify). Run: `npx jest`.

FIRST – మంచి test యొక్క 5 లక్షణాలు

అన్ని tests సమానం కాదు. మంచి unit test **FIRST** గుణాలు కలిగి ఉంటుంది:

అక్షరం	అర్థం	ఎందుకు
F – Fast	milliseconds లో run అవ్వాలి	slow tests ని devs run చేయరు → useless
I – Isolated	ఒకదానిపై ఒకటి ఆధారపడకూడదు, order మారినా pass	flaky, debug కష్టం అవ్వకుండా
R – Repeatable	ఎప్పుడు, ఎక్కడ run చేసినా అదే result	"నా machine లో pass, CI లో fail" avoid

అక్షరం	అర్థం	ఎందుకు
S – Self-validating	pass/fail automatic (assert), human eye కాదు	manual check = scale అవ్వదు
T – Timely	code తో పాటు (లేదా ముందు – TDD) రాయాలి	తర్వాత రాస్తే ఎప్పటికీ రాయరు

Test naming: test పేరు "ఏ scenario, ఏ expected behavior" చెప్పాలి – `test("ఖాళీ array కి 0 return చేస్తుంది")` (మంచిది), `test("test1")` (చెడ్డది). Test fail అయినప్పుడు, పేరు చదివితేనే ఏం break అయ్యిందో తెలియాలి.

React Testing Library – MERN frontend test (ఉదా)

Backend logic ని Jest తో చూశాం. MERN frontend (React) కి **React Testing Library (RTL)** – user దృష్టిలో test చేస్తుంది ("implementation కాదు, behavior test చేయి"):

```
// LoginButton.test.jsx
import { render, screen, fireEvent } from "@testing-library/react";
import LoginButton from "./LoginButton";

test("click చేస్తే onLogin call అవుతుంది", () => {
  const onLogin = jest.fn();              // mock function
  render(<LoginButton onLogin={onLogin} />);

  // user చూసేలా element ని కనుక్కో (text/role తో, CSS class తో కాదు)
  const button = screen.getByRole("button", { name: /log in/i });
  fireEvent.click(button);                 // user action simulate

  expect(onLogin).toHaveBeenCalledTimes(1); // behavior verify
});
```

RTL philosophy: **"the more your tests resemble the way your software is used, the more confidence they give you."** అంటే – internal state/props కాదు, user ఏం చూస్తాడు/clicks చేస్తాడో test చేయి. దీనివల్ల refactor చేసినా (internal మారినా, behavior అదే) tests break అవ్వవు.

Flaky Tests – అస్థిర tests, testing యొక్క శత్రువు

Flaky test = ఏ code మార్పు లేకుండా, కొన్నిసార్లు pass, కొన్నిసార్లు fail అయ్యే test. ఇవి testing యొక్క అతిపెద్ద శత్రువు – ఎందుకంటే team వాటిని నమ్మడం మానేస్తుంది ("మళ్ళీ ఆ flaky test, re-run చేయి"), తర్వాత నిజమైన failures కూడా ignore చేస్తుంది. కారణాలు:

- **Timing/async:** hardcoded `sleep`, race conditions (fix: proper waits, `waitFor`).
- **Shared state:** ఒక test ఇంకో test కి affect (fix: isolation, cleanup).
- **External dependencies:** real API/network/time/random (fix: mock వాటిని).
- **Order dependence:** tests ఒక specific order లో మాత్రమే pass (fix: independent tests).

Flaky test ని వెంటనే fix చేయాలి లేదా quarantine చేయాలి – ignore చేస్తే మొత్తం test suite trust కోల్పోతుంది.

ఏం test చేయాలి, ఏం చేయకూడదు

అన్నిటినీ test చేయడం waste; ఏదీ test చేయకపోవడం risky. Balance:

- **Test చేయాలి:** business logic (calculations, validations, state transitions), edge cases, bug fixes (regression test – అదే bug మళ్ళీ రాకుండా), critical user flows (login, payment).
- **Test చేయనవసరం లేదు:** third-party libraries (అవి already tested – నీ Express, Mongoose ని test చేయకు), trivial getters/setters, framework internals, పదేపదే మారే UI pixel details.
- **Golden guide:** "ఇది break అయితే users/business కి బాధ కలిగిస్తుందా?" అవును → test. "ఇది ఎప్పటికీ break అవ్వదు, break అయినా ఎవరికీ పట్టదు" → skip. Test *risk* ని target చేయాలి, coverage number ని కాదు.

Real MERN team లో

ప్రతి PR లో: business logic కి **unit tests** (Jest), API routes కి **integration tests** (Jest + supertest + test MongoDB), critical user flows (login, checkout) కి **E2E** (Cypress/Playwright). CI ప్రతి commit కి tests run చేస్తుంది – fail అయితే merge block. Coverage report PR లో కనిపిస్తుంది (కొత్త code coverage తగ్గకూడదు). React components కి React Testing Library. Tests లేని PR ని reviewer reject చేస్తాడు.

Key Points

- **Tests = confidence + regression safety net + living docs + better design.** అవి change ని fearless చేసే freedom, కేవలం bug-catching కాదు.
- **Test Pyramid:** చాలా unit (fast, cheap, stable) → కొంత integration → కొన్ని E2E (slow, brittle, high-confidence). Ice-cream-cone anti-pattern (ఎక్కువ E2E) avoid చేయి.
- **TDD = Red (fail test) → Green (min code to pass) → Refactor.** Test-first → clear spec, testable code, no over-engineering.
- **Mock/stub** = dependencies (DB/API) ని fake చేసి unit tests ని fast & isolated గా ఉంచడం; అతిగా mock చేస్తే false confidence.
- **Coverage** = ఎంత code run అయ్యింది, **bug-free కాదు.** Guide గా వాడు, target గా కాదు (~70-80% reasonable).
- Jest: `describe` / `test` / `expect` , AAA pattern, happy + edge + error cases test చేయి.

Interview దృష్టి

Q: మీరు ఎందుకు tests రాస్తారు? Deadline ఉంటే skip చేయవచ్చా? A: Tests confidence + regression safety ఇస్తాయి – నా code సరిగ్గా ఉందని proof, పైగా future changes ఏదైనా break చేస్తే వెంటనే catch. Manual testing scale అవ్వదు. Deadline కోసం skip చేస్తే – short-term fast, కానీ ప్రతి తర్వాతి change భయంతో, slow, buggy అవుతుంది (tech debt). నేను కనీసం core business logic కి unit tests తప్పకుండా రాస్తాను; అవి time save చేస్తాయి, waste చేయవు.

Q: Test pyramid explain చేయండి. A: కింద చాలా unit tests (ఒక function isolated, fast, cheap, stable), మధ్యలో కొంత integration tests (modules/DB/API కలిసి), పైన కొన్ని E2E tests (full app, real browser – slow, brittle, ఖరీదు కానీ high confidence). ఎక్కువ E2E రాస్తే CI slow + flaky అవుతుంది

(ice-cream-cone anti-pattern). కాబట్టి fast unit tests ని foundation గా, E2E ని కీలక flows కి మాత్రమే వాడతాం.

Q: TDD అంటే? నిజంగా follow చేస్తారా? A: TDD = code కి ముందు test రాయడం: Red (test fail అవుతుంది) → Green (pass అయ్యే min code) → Refactor (clean, tests green). Benefits – test = clear spec, 100% testable code, no over-engineering, refactor confidence. నేను strict TDD ప్రతిసారి కాకపోయినా, tests ని code తో పాటు (తర్వాత కాదు) రాయడం అనే core discipline follow చేస్తాను, ముఖ్యంగా complex business logic కి.

Q: 100% code coverage ఉంటే bug-free అనొచ్చా? A: లేదు. Coverage అంటే "ఈ lines execute అయ్యాయి" మాత్రమే – "అవి సరిగ్గా పని చేస్తున్నాయి" కాదు. Assertions లేని test కూడా coverage పెంచుతుంది. Coverage ని target చేస్తే devs meaningless tests రాస్తారు. నేను coverage ని test చేయని areas ని కనుక్కునే guide గా వాడతా, edge/error cases ని consciously test చేస్తా – number ని target చేయను.

---

10. Code Review

CODE REVIEW · ప్రాధాన్యత క్రమం

CODE REVIEW – ఏం చూడాలి, ఏ క్రమంలో

1 · Correctness	ఇది నిజంగా పని చేస్తుందా · edge cases
2 · Design	ఇది సరైన చోట, సరైన abstraction నా
3 · Tests	కొత్త behaviour కి tests ఉన్నాయా
4 · Readability	6 నెలల తర్వాత అర్థమవుతుందా
5 · Style	linter పని – మనిషి పని కాదు

Review ని విషపూరితం చేసేవి

Style మీద పోరాటం – అది **linter/prettier** పని, CI lo ఆటోమేట్ చేయాలి.
"ఇది తప్పు" కాకుండా "ఇలా చేస్తే ఎలా ఉంటుంది?" – ప్రశ్నగా అడగడం.
Blocking comment మరియు nit ని స్పష్టంగా వేరు చేయడం (nit: అని prefix).

వివరణ

Code Review = నీ code ని merge చేసేముందు ఇంకో engineer చదివి, feedback ఇవ్వడం. ప్రతి modern team లో ఇది mandatory gate – PR (Topic 7) raise చేస్తే, 1-2 reviewers approve చేయందే main లో merge అవ్వదు. Self-taught devs కి ఇది కొత్త, కొంచెం భయపెట్టే experience – "నా code ని ఎవరో judge చేస్తారు." కానీ నిజానికి ఇది నీ అతిపెద్ద learning tool + team యొక్క quality safety net.

Code review ఎందుకు ఇంత విలువైనది? నాలుగు కారణాలు:

1. **Bugs early పట్టుకోవడం:** ఇంకో కంటికి నీకు కనబడని bug కనిపిస్తుంది (cost-of-change – cheap phase లో).

2. **Knowledge sharing:** ఒక feature గురించి కనీసం ఇద్దరికి తెలుస్తుంది (**bus factor** పెరుగుతుంది – ఒకరు leave అయినా knowledge పోదు).
3. **Consistency:** team అంతా ఒకే standards, patterns follow చేస్తుంది.
4. **Mentorship:** juniors seniors నుండి నేర్చుకుంటారు; seniors juniors నుండి fresh ideas పొందుతారు.

అతి ముఖ్యమైన mindset shift: code review అనేది "gatekeeping" (నిన్ను ఆపడం) కాదు – "collaboration" (కలిసి better code చేయడం). Reviewer నీ శత్రువు కాదు, నీ teammate. అలాగే, review అవుతున్నది **code, నువ్వు కాదు.** ఈ ఒక్క distinction – "my code" కాదు "the code" – feedback ని personal గా తీసుకోకుండా చేస్తుంది.

Real-life Scenario

> **Code review = ముఖ్యమైన email పంపేముందు colleague కి చూపించడం.**
>
> నువ్వు boss కి ఒక ముఖ్యమైన email రాశావ్. Send నొక్కేముందు, పక్క desk colleague కి "ఒకసారి చూడు" అని చూపిస్తావు. అతను – "ఇక్కడ tone కొంచెం rude గా ఉంది, ఈ figure తప్పు, ఈ attachment మర్చిపోయావ్" అని చెప్తాడు. నీకు కోపం రాదు – పైగా thanks చెప్తావు, ఎందుకంటే అతను నిన్ను embarrassment నుండి కాపాడాడు.
>
> Code review సరిగ్గా అదే – production (boss) కి పంపేముందు, teammate నీ code ని చూసి mistakes పట్టుకుంటాడు. **Fresh కళ్ళకి, రచయితకి కనబడని తప్పులు కనిపిస్తాయి.** నీ email ని నువ్వు 10 సార్లు చదివినా typo కనబడదు; colleague మొదటిసారే పట్టుకుంటాడు. అదే మనస్తత్వం.

Reviewer గా – ఏం చూడాలి (priority order)

Review చేసేటప్పుడు ఒక mental checklist. **ముఖ్యమైన order** – పైవి ముందు, కిందవి తర్వాత:

Priority	ఏం చూడాలి	ప్రశ్నలు
1. **Correctness**	Logic సరిగ్గా ఉందా?	Edge cases handle చేశారా? Off-by-one? Null checks? Race conditions?
2. **Design/Architecture**	ఇది సరైన approach నా?	Existing patterns కి fit అవుతుందా? SOLID? సరైన చోట ఉందా?
3. **Security**	ఏమైనా vulnerability?	Input validation? SQL/NoSQL injection? Secrets hardcode అయ్యాయా? Auth checks?
4. **Tests**	Adequate tests ఉన్నాయా?	Happy + edge + error cases? Tests meaningful గా ఉన్నాయా?
5. **Readability**	సులభంగా అర్థమవుతోందా?	Names స్పష్టమా? Functions చిన్నవా? Comments where needed?
6. **Performance**	Obvious inefficiency?	N+1 queries? Unnecessary loops? Big-O concern (real scale లో)?
7. **Style/Nits**	Formatting, conventions	(ఇది automate చేయాలి – linter, human కాదు)

కీలక insight: style/formatting (priority 7) ని **automate** చేయాలి (Prettier/ESLint) – human reviewers correctness, design, security (priority 1-4) మీద focus చేయాలి. Semicolon, indentation గురించి review comment రాయడం reviewer time waste. Machine చేయగలిగేది machine చేయాలి.

Reviewer గా – ఎలా feedback ఇవ్వాలి (అతి ముఖ్యం)

Feedback ఏం చెప్తావో కాదు, ఎలా చెప్తావో team culture ని నిర్ణయిస్తుంది. Rules:

- **Code ని criticize చేయి, మనిషిని కాదు.** "You wrote bad code" కాదు; "ఈ function ని split చేస్తే readability పెరుగుతుందేమో?"
- **Questions అడుగు, orders ఇవ్వకు.** "Change this to X" కాదు; "X approach ఏమైనా better అవుతుందా? ఎందుకు Y ఎంచుకున్నావ్?" (బహుశా వాళ్ళకి నీకు తెలియని reason ఉందేమో.)
- **"Why" explain చేయి.** "ఇది తప్పు" కాదు; "ఇది concurrent requests లో race condition సృష్టిస్తుంది ఎందుకంటే..." – reviewee నేర్చుకుంటాడు.
- **మంచిదాన్ని praise చేయి.** "ఈ error handling చాలా clean గా ఉంది" – reviews కేవలం criticism అవ్వకూడదు.
- **Nitpicks ని label చేయి.** ముఖ్యం కాని suggestion అయితే "nit:" prefix – "nit: ఈ variable పేరు..." (optional అని తెలుస్తుంది).
- **Blocking vs non-blocking స్పష్టం చేయి.** "ఇది merge కి ముందు fix చేయాలి" vs "ఇది తర్వాత చూద్దాం" – reviewee కి clarity.

Conventional Comments అనే convention కొన్ని teams వాడతాయి: `praise:`, `nit:`, `suggestion:`, `question:`, `issue:`, `blocking:` – comment యొక్క severity/intent స్పష్టం చేయడానికి.

Author (reviewee) గా – feedback ఎలా తీసుకోవాలి

- **Personal గా తీసుకోకు.** Review అవుతున్నది code, నువ్వు కాదు. ప్రతి comment నిన్ను better engineer చేస్తుంది.
- **Ego వదులు.** నీ మొదటి approach best కాకపోవచ్చు – అది normal. Best engineers కూడా reviews లో feedback పొందుతారు.
- **అర్థం కాకపోతే అడుగు.** "ఇది ఎందుకు problem?" అని అడగడం weakness కాదు – learning.
- **అంగీకరించకపోతే, respectfully discuss చేయి.** "నేను Y ఎంచుకున్నా ఎందుకంటే... నీ concern valid, కానీ..." – reviewer కూడా తప్పు కావచ్చు.
- **Feedback ని act చేయి** – changes చేసి, "done" అని reply చేయి, లేదా ఎందుకు చేయలేదో explain చేయి. Ignore చేయకు.

PR Etiquette – రెండువైపులా మర్యాద

Author responsibilities:

- చిన్న PR (Topic 7) – reviewer కి దయ.
- మంచి description – context ఇవ్వు, reviewer code చదవడం సులభం.
- Self-review ముందు – silly mistakes నువ్వే పట్టుకో.
- Green CI – broken PR review అడగకు.

Reviewer responsibilities:

- **వేగంగా respond చేయి** – PR days పాటు pending ఉంటే author blocked, momentum పోతుంది. (Many teams: "review within 1 business day.")
- **Thorough కానీ pragmatic** – perfect ని వెతకకు, "good enough + safe" చాలు. Bikeshedding (చిన్న విషయాల మీద అతి discussion) avoid చేయి.
- **గౌరవంగా** – నీ comment ఒక మనిషి చదువుతాడు.

Self-review Checklist – PR raise చేసేముందు (author)

Reviewers time విలువైనది. PR raise చేసేముందు నీ diff ని నువ్వే ఒకసారి review చేయి – ఇది silly comments ని ఆపి, review వేగం పెంచుతుంది:

- [ ] Diff మొత్తం చదివా, debug `console.log`, commented code తీసేశా.
- [ ] Tests జోడించా, అన్నీ green (CI pass).
- [ ] Meaningful commit messages + PR description (ఏం/ఎందుకు/ఎలా test చేశా).
- [ ] Edge cases, error handling ఉన్నాయి.
- [ ] Secrets, hardcoded values లేవు.
- [ ] Naming clear, functions చిన్నవి (Topic 8).
- [ ] PR చిన్నది (పెద్దదైతే split చేయగలనా?).
- [ ] UI change అయితే screenshots/video జోడించా.

Review metrics reality: research చెప్తుంది – reviewers 400 lines దాటితే bug-detection dramatically పడిపోతుంది, మరియు 60 నిమిషాల పైన review quality క్షీణిస్తుంది. అందుకే **చిన్న PRs = ఎక్కువ bugs caught.** ఇది "reviewer సోమరి" కాదు – human attention యొక్క limit. చిన్న PR = నీ code కి better review = fewer prod bugs.

Real MERN team లో

Author feature branch push చేసి, GitHub PR raise చేస్తాడు (Jira ticket link, description, screenshots). CODEOWNERS ఆధారంగా 2 reviewers auto-assign అవుతారు. CI (tests, lint, build) run అవుతుంది. Reviewers inline comments పెడతారు – కొన్ని "blocking: null check missing", కొన్ని "nit: rename this". Author changes చేసి, threads resolve చేసి, re-request review. 2 approvals + green CI అయ్యాక squash-merge. మొత్తం loop 1-2 రోజుల్లో. తరచుగా pending PRs Scrum standup లో "review కావాలి" అని surface అవుతాయి.

Key Points

- **Code review = merge ముందు peer feedback** – mandatory quality gate. Benefits: early bugs, knowledge sharing (bus factor↑), consistency, mentorship.
- Mindset: **collaboration, gatekeeping కాదు.** Review అవుతున్నది **code, మనిషి కాదు.**
- Reviewer priority: **correctness → design → security → tests → readability → performance → style.** Style ని automate (linter), human ని high-value మీద focus.
- Feedback: **code ని criticize, questions అడుగు, why explain, praise కూడా, blocking vs nit label.**

- Author: **personal గా తీసుకోకు, ego వదులు, అడుగు, respectfully discuss, act చేయి.**
- Etiquette: చిన్న PR + మంచి description + self-review (author); వేగంగా + pragmatic + గౌరవంగా (reviewer).

Interview దృష్టి

Q: Code review లో ఏం చూస్తారు? A: Priority order లో – correctness (logic, edge cases, null/race conditions) ముందు, తర్వాత design (సరైన approach, patterns fit), security (input validation, injection, secrets), tests (adequate + meaningful), readability (names, function size), performance (N+1 queries వంటివి). Style/formatting ని linter కి వదిలేస్తా – human గా correctness, design, security మీద focus చేస్తా. Machine చేయగలిగేది machine చేయాలి.

Q: Review లో కఠినమైన feedback వస్తే ఎలా react అవుతారు? A: Personal గా తీసుకోను – review అవుతున్నది code, నేను కాదు. ప్రతి comment నన్ను better చేస్తుంది. అర్థం కాకపోతే అడుగుతా, అంగీకరించకపోతే respectfully reasoning తో discuss చేస్తా (reviewer కూడా తప్పు కావచ్చు). చివరికి feedback ని act చేసి, threads resolve చేస్తా. Best engineers కూడా reviews పొందుతారు – ego వదలడం maturity.

Q: Junior code review చేస్తున్నారు, చాలా issues ఉన్నాయి. ఎలా feedback ఇస్తారు? A: మనిషిని కాదు, code ని address చేస్తా. Orders కాదు, questions అడుగుతా ("X approach better అవుతుందా?"). ప్రతి issue కి "why" explain చేస్తా – వాళ్ళు నేర్చుకోవాలి. మంచి భాగాలని praise కూడా చేస్తా – reviews demoralizing అవ్వకూడదు. అన్ని nitpicks blocking కాదని label చేస్తా. అతి ఎక్కువ issues ఉంటే, PR చాలా పెద్దదేమో – sync లో discuss చేసి, భవిష్యత్తులో చిన్న PRs encourage చేస్తా.

Part 4 – Ship & operate (deploy చేసి నడపడం)

> Code రాశాం, test చేశాం, review చేశాం – ఇప్పుడు దాన్ని users చేతికి *safely, repeatedly* ఎలా చేర్చాలి? Break అయితే ఎలా తెలుస్తుంది, ఎలా fix చేస్తాం? ఈ Part లో: CI/CD (automated build-test-deploy pipelines), DevOps basics (Docker, Kubernetes, IaC, monitoring), మరియు documentation/estimation/collaboration. ఇవి "code రాసేవాడు" నుండి "software ని own చేసి నడిపేవాడు" గా మారుస్తాయి – అదే SSE.

---

11. CI/CD – Continuous Integration / Delivery / Deployment

వివరణ

CI/CD అనేది modern software delivery యొక్క వెన్నెముక. SSE interview లో దాదాపు guaranteed question. Self-taught devs "code push చేస్తే auto deploy అవుతుంది" అని surface level తెలుసుకుంటారు – కానీ *ఎందుకు, ఎలా, ఏ stages* అనేది SSE level. దీన్ని మూడు వేర్వేరు (కానీ related) concepts గా అర్థం చేసుకుందాం:

CI – Continuous Integration: ప్రతి developer తన code ని రోజూ చాలాసార్లు shared main branch లో merge చేస్తాడు, మరియు ప్రతి merge/push కి **automated build + tests** run అవుతాయి. లక్ష్యం – integration problems (conflicts, broken tests) ని *వెంటనే, చిన్నగా* పట్టుకోవడం, వారాల తర్వాత పెద్దగా కాదు.

CD – Continuous Delivery: CI + code ఎప్పుడూ **deployable state** లో ఉంచడం. ప్రతి change automatically staging వరకు వెళ్తుంది, production కి deploy చేయడానికి ఒక **manual button** (human approval) మాత్రమే మిగిలి ఉంటుంది. అంటే – ఎప్పుడైనా, ఒక్క click తో, safely release చేయవచ్చు.

CD – Continuous Deployment: ఒక అడుగు ముందుకి – manual button కూడా లేదు. Tests pass అయితే, code **automatically production కి** వెళ్తుంది. Human intervention లేదు. (Delivery = button ఉంది; Deployment = button లేదు.)

```
Continuous Integration:   merge → build → test                          (auto)
Continuous Delivery:      merge → build → test → staging → [🔘 manual] → prod
Continuous Deployment:    merge → build → test → staging → prod         (fully auto)
                                                          ↑ button లేదు
```

CI/CD · COMMIT నుంచి PRODUCTION వరకు

CI / CD PIPELINE

Commit	Build	Test	Scan	Deploy	Release
git push	compile, bundle	unit + integration	lint, security	staging	production

CI – Continuous Integration

ప్రతి push కి: build + test ఆటోమేటిక్ గా
విరిగితే నిమిషాల్లో తెలుస్తుంది
లక్ష్యం: main ఎప్పుడూ పని చేసే స్థితిలో

CD – Delivery vs Deployment

Delivery = ఎప్పుడైనా release చేయగలిగే స్థితి
(చివరి బటన్ మనిషి నొక్కుతాడు)
Deployment = ఆ బటన్ కూడా ఆటోమేటిక్

ఎందుకు ఇది ముఖ్యం

చిన్న, తరచుగా వచ్చే మార్పులు = చిన్న ప్రమాదం. నెలకోసారి పెద్ద release = పెద్ద ప్రమాదం.
Rollback ని ముందే ఆలోచించాలి – deploy చేయడం కంటే వెనక్కి తీసుకోవడం ముఖ్యం.

Real-life Scenario

> **CI/CD = restaurant యొక్క automated kitchen assembly line.**
>
> పాత రోజుల్లో (CI/CD లేకుండా): ప్రతి chef తన dish విడిగా వండి, రోజు చివర్లో అందరూ కలిసి ఒక పెద్ద thali అమర్చుతారు (integration). అప్పుడు తెలుస్తుంది – ఒక curry మరో దానికి match అవ్వదు, ఒకటి చెడిపోయింది. అంతా మళ్ళీ. **నెలకోసారి "release day" = నరకం.**
>
> CI/CD ఉన్న kitchen: ప్రతి chef ఒక dish తయారుచేయగానే, వెంటనే **conveyor belt** మీద పెడతాడు. Belt మీద automatic quality check (tests) – ఉప్పు సరిపోయిందా, వేడిగా ఉందా. Fail అయితే వెంటనే belt ఆగి alarm (ఆ chef కి). Pass అయితే నేరుగా customer table కి (production). **ప్రతి dish చిన్నగా, తరచుగా, automatic గా, checked గా వెళ్తుంది – పెద్ద risky "release day" లేదు.**
>
> అసలు idea: **"if it hurts, do it more often."** Integration నొప్పి పెడితే, నెలకోసారి కాదు – రోజుకి 10 సార్లు చేయి; అప్పుడు ప్రతి integration చిన్నది, painless.

Pipeline Stages – build → test → deploy

ఒక typical CI/CD pipeline ఇలా ఉంటుంది (ప్రతి stage fail అయితే pipeline ఆగుతుంది – "fail fast"):

```
git push / PR
      │
      ▼
┌───────────┐   ┌───────────┐   ┌───────────┐   ┌────────────┐   ┌────────────┐   ┌───────────┐
│ 1.        │   │ 2.        │   │ 3.        │   │ 4.         │   │ 5.         │   │ 6.        │
│ Checkout  │──▶│ Build/    │──▶│ Lint +    │──▶│ Test       │──▶│ Package    │──▶│ Deploy    │
│ + deps    │   │ Compile   │   │ Static    │   │(unit+      │   │(Docker     │   │(staging   │
│           │   │           │   │ analysis  │   │ integ.)    │   │ image)     │   │ → prod)   │
└───────────┘   └───────────┘   └───────────┘   └────────────┘   └────────────┘   └───────────┘
  ✅ fast → slow (ముందు చౌక checks; ఖరీదైన deploy చివర్లో)
```

1. **Checkout + install:** code pull, dependencies install (`npm ci`).
2. **Build/Compile:** MERN లో – frontend build (`npm run build`), TypeScript compile.
3. **Lint + static analysis:** ESLint, Prettier check, type check, security scan (SAST).

4. **Test:** unit → integration → (కొన్ని) e2e. Coverage report.
5. **Package:** deployable artifact – సాధారణంగా **Docker image** build & registry కి push.
6. **Deploy:** staging కి auto, తర్వాత (approval తో/లేకుండా) production.

కీలక design principle: fast, cheap checks ముందు (lint seconds; unit tests minutes), **slow, expensive చివర్లో** (e2e, deploy). ఒక lint error ఉంటే, ఖరీదైన deploy దాకా వెళ్ళే ముందే fail – time & resources save.

GitHub Actions Example – MERN CI pipeline

GitHub Actions = GitHub లో అత్యంత popular CI/CD tool. `.github/workflows/*.yml` file రాస్తే, GitHub events (push, PR) కి workflow run అవుతుంది. ఒక real MERN CI:

```
# .github/workflows/ci.yml
name: CI Pipeline

on:                            # ఎప్పుడు run అవ్వాలి
  push:
    branches: [main]
  pull_request:                # ప్రతి PR కి కూడా

jobs:
  test:
    runs-on: ubuntu-latest     # GitHub ఇచ్చే Linux machine
    services:
      mongodb:                 # integration tests కి test DB
        image: mongo:6
        ports: ['27017:27017']
    steps:
      - name: Code checkout
        uses: actions/checkout@v4

      - name: Node.js setup
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'         # dependencies cache → వేగం

      - name: Dependencies install
        run: npm ci             # ci = clean, reproducible install

      - name: Lint (code style + errors)
        run: npm run lint

      - name: Unit + integration tests
        run: npm test -- --coverage
        env:
          MONGO_URL: mongodb://localhost:27017/test

      - name: Build (frontend + backend)
        run: npm run build

  deploy:
    needs: test                # test job pass అయ్యాకే deploy (gate)
    if: github.ref == 'refs/heads/main'   # main branch కి మాత్రమే
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - name: Production deploy
        run: ./scripts/deploy.sh
        env:
          DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}   # secrets safe గా
```

గమనిక: `needs: test` వల్ల deploy job, test pass అయ్యాకే run అవుతుంది (broken code deploy అవ్వదు). `if: github.ref == main` వల్ల PR లకి test మాత్రమే, main merge కి మాత్రమే deploy. `secrets.DEPLOY_TOKEN` – credentials code లో కాదు, GitHub Secrets లో (Topic 12 security). ఇది exactly ఒక real MERN team వాడే pattern.

Deployment Strategies – production కి safely ఎలా

Code ని production కి push చేయడం risky – bug ఉంటే అందరు users affected. దీన్ని safe చేయడానికి strategies (deep dive `HLD_Telugu.md`, `HLD_Go_Telugu.md`):

Strategy	ఎలా పని చేస్తుంది	మంచిది	చెడ్డది
Recreate	పాతది ఆపి, కొత్తది start (downtime ఉంటుంది)	సులభం, చౌక	Downtime, risky
Rolling	Servers ని కొన్ని కొన్ని గా update (batch by batch)	Downtime లేదు, resource-efficient	పాత+కొత్త versions కలిసి నడుస్తాయి (compat కావాలి), rollback slow
Blue-Green	రెండు identical environments (blue=old, green=new). Green ready అయ్యాక traffic switch.	Instant rollback (switch back), zero downtime	2x infrastructure ఖరీదు
Canary	కొత్త version ని ముందు కొద్దిమంది (5%) users కి, monitor, సరిగ్గా ఉంటే క్రమంగా 100%	Risk తక్కువ (bug 5% కే), real-user validation	Complex, monitoring కావాలి, slow rollout

```
Blue-Green:                          Canary:

  ┌────────┐    traffic               ┌────────────────────┐
  │ BLUE   │◄──────                   │ v1 (95% users)     │
  │ (v1)   │                          │ v2 (5% users) 🐤   │  → monitor
  └────────┘                          └────────────────────┘

  ┌────────┐    switch →              క్రమంగా: 5% → 25% → 50% → 100%
  │ GREEN  │◄══════                    bug కనిపిస్తే → rollback (5% కే affected)
  │ (v2)   │    (instant)
  └────────┘
```

ఎప్పుడు ఏది: downtime OK + simple → recreate/rolling. Zero downtime + instant rollback కావాలి → blue-green. అత్యధిక safety, పెద్ద user base, risky change → **canary** (Google/Netflix default). చాలా MERN teams – staging కి rolling, production కి canary లేదా blue-green.

CI/CD ఎందుకు game-changer – benefits

- **Bugs early:** ప్రతి commit tested → integration issues చిన్నగా, వెంటనే.

- **Fast, frequent releases:** రోజుకి multiple deploys సాధ్యం (Amazon సెకనుకి ఒక deploy చేస్తుంది!).
- **తక్కువ risk:** చిన్న changes = చిన్న blast radius; break అయితే ఏ commit అని easy.
- **Automation:** manual, error-prone deploy steps తొలగింపు; repeatable, reliable.
- **Faster feedback:** feature push చేసిన గంటలో users చేతిలో → వేగంగా నేర్చుకోవడం.

Environments – dev → staging → prod

Code నేరుగా production కి వెళ్ళదు – అది **environments** అనే stages గుండా promote అవుతుంది. ప్రతిదీ production కి దగ్గరవుతూ:

Environment	ఎవరు వాడతారు	ఎందుకు
Local/Dev	developer laptop	code రాయడం, quick test
CI	pipeline	automated tests ప్రతి commit కి
Staging	QA, team	production-లాంటి env లో final testing (real DB copy, integrations)
Production	real users	live traffic

కీలక principle – parity: staging ని production కి వీలైనంత దగ్గరగా (same config, data shape, versions) ఉంచాలి. లేకపోతే "staging లో pass, prod లో fail" – Docker (Topic 12) ఈ parity ని enforce చేస్తుంది. కొన్ని teams ephemeral "preview environments" (ప్రతి PR కి ఒక temporary env) కూడా వాడతాయి.

Rollback – వెనక్కి వెళ్ళగలగడం

CI/CD యొక్క unsung hero: **fast rollback.** కొత్త deploy break అయితే, ముందు version కి *వెంటనే* తిరిగి వెళ్ళగలగాలి (Topic 13 "stop the bleeding"). అందుకే immutable Docker images, versioned deploys, blue-green/canary (instant switch back). **"Roll forward" (కొత్త fix deploy)** vs **"roll back" (పాత version)** – emergency లో rollback safer, ఎందుకంటే పాత version already tested & working.

DORA Metrics – team performance ని measure చేయడం

Google DORA research ప్రకారం, high-performing teams ని 4 metrics చెప్తాయి – ఇవి SSE interview లో impressive:

Metric	ఏమిటి	Elite teams
Deployment Frequency	ఎంత తరచు deploy చేస్తారు	రోజుకి multiple
Lead Time for Changes	commit నుండి prod వరకు time	గంటల్లో
Change Failure Rate	ఎన్ని deploys break అవుతాయి	< 15%
MTTR (Mean Time To Recovery)	break అయ్యాక recover ఎంత time	గంటలోపు

మొదటి రెండు = **velocity** (వేగం); చివరి రెండు = **stability** (నాణ్యత). పాత అపోహ "fast అంటే unstable" – DORA దీన్ని తప్పని నిరూపించింది: elite teams రెండింటిలోనూ మెరుగు (strong CI/CD వల్ల). Speed vs

stability trade-off కాదు; మంచి practices రెంటినీ ఇస్తాయి.

Real MERN team లో

Developer PR raise చేస్తాడు → GitHub Actions CI (lint, test, build) run → green అయితే + 2 approvals → main merge → CD pipeline Docker image build → staging కి auto-deploy → automated smoke tests → production కి canary deploy (5% → monitor error rate/latency → 100%). Error rate పెరిగితే automatic rollback + team కి alert. ఇదంతా code push నుండి production వరకు 15-30 నిమిషాల్లో, human ఒక్క approval button తప్ప. ఇదే "code push చేస్తే auto deploy" వెనుక ఉన్న నిజమైన machinery.

Key Points

- **CI** = ప్రతి merge కి auto build+test (integration issues early). **CD (Delivery)** = ఎప్పుడూ deployable, prod కి manual button. **CD (Deployment)** = button కూడా లేదు, fully auto to prod.
- Pipeline stages: **checkout → build → lint → test → package (Docker) → deploy.** Fast/cheap checks ముందు, slow/expensive చివర ("fail fast").
- **GitHub Actions:** `.github/workflows/*.yml`, events (push/PR) trigger, jobs+steps, `needs` gates, secrets for credentials.
- **Deployment strategies:** recreate (downtime), rolling (batch, no downtime), blue-green (2 envs, instant rollback), **canary** (కొద్దిమందికి ముందు, safest for risky/large-scale).
- Benefits: early bugs, frequent low-risk releases, automation, fast feedback. Core mantra: **"if it hurts, do it more often."**

Interview దృష్టి

Q: CI vs CD – తేడా? A: CI (Continuous Integration) = developers రోజూ చాలాసార్లు main లో merge చేస్తారు, ప్రతి merge కి automated build+test run అవుతుంది – integration bugs early పట్టుకోవడానికి. CD రెండు అర్థాలు: Continuous Delivery = code ఎప్పుడూ deployable, production కి ఒక manual approval button; Continuous Deployment = ఆ button కూడా లేదు, tests pass అయితే automatically prod కి. తేడా – Delivery లో human clicks; Deployment లో fully automated.

Q: CI/CD pipeline stages చెప్పండి. A: Checkout + install deps → build/compile → lint & static analysis → test (unit + integration) → package (Docker image) → deploy (staging → prod). Fast, cheap checks (lint) ముందు, slow expensive (e2e, deploy) చివర్లో – fail fast కోసం. ప్రతి stage fail అయితే pipeline ఆగుతుంది, broken code ముందుకి వెళ్ళదు.

Q: Blue-green vs canary deployment? A: Blue-green = రెండు identical environments (blue=current, green=new); green ready అయ్యాక traffic ని పూర్తిగా switch – instant rollback (switch back), zero downtime, కానీ 2x infra. Canary = కొత్త version ని ముందు కొద్దిమంది (5%) users కి release, error rate/latency monitor, సరిగ్గా ఉంటే క్రమంగా 100% – risky changes కి safest ఎందుకంటే bug కేవలం 5% ని affect చేస్తుంది. పెద్ద user base + risky change → canary; instant full rollback కావాలంటే blue-green.

Q: "If it hurts, do it more often" – అర్థం? A: Integration/deployment నొప్పి పెడితే (conflicts, broken releases), దాన్ని తక్కువసార్లు కాదు – ఎక్కువసార్లు చేయాలి. నెలకోసారి పెద్ద risky release కంటే, రోజుకి

చిన్న చిన్న releases చాలా సురక్షితం – ప్రతిది చిన్నది, blast radius తక్కువ, break అయితే ఏ change అని easy. CI/CD యొక్క మూల philosophy ఇదే.

---

12. DevOps Basics

DEVOPS · సంస్కృతి మరియు DORA METRICS

DEVOPS – ఒక సంస్కృతి, ఒక పాత్ర కాదు

పాత విధానం

Dev code రాసి "throw over the wall"
Ops deploy చేసి, పడిపోతే వాళ్ళ సమస్య
ఒకరినొకరు నిందించుకోవడం

DevOps

"You build it, you run it"
Dev కి production visibility
Ops కి pipeline lo పాత్ర

నాలుగు కీలక metrics (DORA)

Deploy frequency	ఎంత తరచుగా release
Lead time	commit నుంచి production దాకా
MTTR	పడిపోతే ఎంతలో లేపుతారు
Change failure rate	ఎన్ని deploys విఫలం

ఈ నాలుగు ఎందుకు

మొదటి రెండు = వేగం · చివరి రెండు = స్థిరత్వం. మంచి teams **రెండింటిలోనూ** మెరుగవుతాయి.
"వేగం vs నాణ్యత" అనేది తప్పుడు ఎంపిక – మంచి pipeline రెండింటినీ ఇస్తుంది.

వివరణ

DevOps = Development + Operations కలయిక – ఒక culture + practices set. పాత రోజుల్లో రెండు వేర్వేరు teams: **Dev** (code రాసేవాళ్ళు) మరియు **Ops** (servers నడిపేవాళ్ళు). Dev code రాసి "ఇదిగో, deploy చేసుకోండి" అని Ops కి విసిరేవాళ్ళు (wall throw). Production లో break అయితే – Dev "నా machine లో పని చేసింది" (works on my machine), Ops "నీ code చెత్త." పరస్పర నింద, slow, painful.

DevOps ఈ wall ని కూల్చుతుంది: **"You build it, you run it"** (Amazon CTO Werner Vogels). అంటే – నువ్వు రాసిన code ని నువ్వే production లో నడుపుతావు, monitor చేస్తావు, break అయితే నువ్వే fix చేస్తావు. ఇది ownership + accountability సృష్టిస్తుంది – "నా code prod లో ఎలా ఉందో నాకు అనవసరం" అనే attitude పోతుంది.

DevOps ఒక tool కాదు – ఒక mindset. కానీ దాన్ని enable చేసే tools/practices ఉన్నాయి. SSE గా నీకు వీటి *concepts* తెలియాలి (deep expertise DevOps engineer specialty; deployment/infra deep dive `HLD_Go_Telugu.md`). ఐదు స్తంభాలు చూద్దాం.

Real-life Scenario

> **పాత విధానం = restaurant లో chef vs waiter యుద్ధం.** Chef వంట చేసి కిటికీలో పెడతాడు, "నా పని అయ్యింది." Waiter cold food, తప్పు table – "chef తప్పు." Customer అసంతృప్తి. ఇద్దరూ ఒకరినొకరు నిందించుకుంటారు, customer బాధపడతాడు.
>
> **DevOps = chef తన dish ని తానే table కి తీసుకెళ్ళి, customer తినడం చూసి, "ఉప్పు సరిపోయిందా?" అని అడిగే restaurant.** ఇప్పుడు chef కి dish యొక్క పూర్తి journey (kitchen → customer) బాధ్యత. Cold అయితే, customer నచ్చకపోతే – chef కే తెలుస్తుంది, chef కే బాధ్యత. **Quality అమాంతం పెరుగుతుంది,** ఎందుకంటే వండేవాడికే end result బాధ్యత. "You cook it, you serve it."

స్తంభం 1 – Containers (Docker)

సమస్య: "నా machine లో పని చేసింది, prod లో break అయ్యింది" – ఎందుకంటే environments వేరు (Node version, OS, env vars, dependencies). **పరిష్కారం – Container.**

Container = నీ app + దాని అన్ని dependencies (Node, libraries, config) ని ఒక్క standardized, portable box లో pack చేయడం. ఆ box ఎక్కడ run చేసినా (నీ laptop, staging, prod) – అచ్చం ఒకేలా నడుస్తుంది. **Docker** = అత్యంత popular container tool.

> **Container = shipping container.** పాత రోజుల్లో సరుకుని loose గా (బస్తాలు, పెట్టెలు) ships లో ఎక్కించేవాళ్ళు – ఒక్కోటి ఒక్కో shape, load చేయడం నరకం. Shipping container వచ్చాక – అన్నీ ఒకే standard box; ఏ ship అయినా, ఏ port అయినా, ఏ truck అయినా handle చేయగలదు. Docker software కి అదే చేసింది – app ని standard box లో pack చేస్తే, ఏ machine అయినా run చేయగలదు.

- `Dockerfile` = container ఎలా build చేయాలో recipe (base image, deps install, code copy, start command).
- **Image** = ఆ recipe నుండి build అయిన snapshot (blueprint). **Container** = ఆ image యొక్క running instance.
- **VM vs Container:** VM మొత్తం OS ని copy చేస్తుంది (heavy, GB, slow start). Container host OS kernel ని share చేస్తుంది, కేవలం app+deps (light, MB, seconds start). అందుకే containers ఎక్కువ efficient.

స్తంభం 2 – Orchestration (Kubernetes)

ఒక container సులభం. కానీ production లో 100 containers, 20 machines మీద, auto-scale, auto-restart-on-crash, load balance – ఇవన్నీ manually manage చేయలేం. అక్కడ **orchestration** – containers ని manage చేసే "conductor."

Kubernetes (K8s) = అత్యంత popular container orchestrator (Google నుండి, Go లో రాశారు). ఇది చేసేవి:

- **Scheduling:** ఏ container ఏ machine లో run చేయాలో నిర్ణయం.

- **Self-healing:** container crash అయితే auto-restart; machine చనిపోతే వేరే machine కి move.
- **Scaling:** load పెరిగితే auto ఎక్కువ containers (horizontal scaling), తగ్గితే తగ్గించు.
- **Load balancing:** traffic ని containers మధ్య పంచడం.
- **Rolling updates:** downtime లేకుండా కొత్త version deploy (Topic 11 strategies).

> **Kubernetes = ఒక పెద్ద orchestra conductor.** 100 musicians (containers) ఉన్నారు. Conductor ఎవరు ఎప్పుడు వాయించాలో, ఒకరు తప్పితే cover ఎలా, tempo (load) ప్రకారం ఎలా adjust – అన్నీ manage చేస్తాడు. నువ్వు "ఈ symphony ఇలా వినిపించాలి" (desired state) అని చెప్తే, conductor దాన్ని maintain చేస్తాడు.

Declarative model: K8s కి నువ్వు "నాకు 5 replicas కావాలి" అని *desired state* చెప్తావు (YAML). K8s ఆ state ని ఎప్పుడూ maintain చేస్తుంది – ఒకటి crash అయితే కొత్తది spin చేస్తుంది. (SSE కి K8s concepts చాలు; deep ops DevOps engineer.)

స్తంభం 3 – IaC (Infrastructure as Code)

సమస్య: servers, databases, networks ని manually (dashboard లో clicks) setup చేస్తే – reproducible కాదు, error-prone, "ఎవరు ఏం మార్చారో" తెలియదు, disaster recovery కష్టం.

IaC = infrastructure ని (servers, DBs, networks) code గా define చేయడం. ఒక file రాస్తే, tool ఆ infrastructure ని create చేస్తుంది. Tools: **Terraform** (cloud-agnostic), **AWS CloudFormation, Pulumi.**

Benefits: **(1) Reproducible** – అదే code → అదే infra (dev/staging/prod identical). **(2) Version controlled** – infra changes కూడా Git లో, reviewed, history. **(3) Automated** – manual clicks లేవు. **(4) Disaster recovery** – infra పోతే, code run చేస్తే మళ్ళీ వస్తుంది.

> **IaC = ఇంటి blueprint vs memory నుండి కట్టడం.** Blueprint లేకుండా ఇల్లు కడితే, రెండో ఇల్లు మొదటిదానిలా ఉండదు, పైగా కూలితే మళ్ళీ కట్టడం కష్టం. Blueprint (code) ఉంటే, ఎన్ని identical ఇళ్ళైనా, ఎప్పుడైనా కట్టవచ్చు. Infrastructure కి అదే.

స్తంభం 4 – Monitoring, Logging, Alerting (observability)

Production లో code నడుస్తోంది – కానీ అది *ఆరోగ్యంగా* ఉందా? Users కి error వస్తోందా? Slow అవుతోందా? దీన్ని తెలుసుకోవడమే **observability.** మూడు స్తంభాలు:

అంశం	ఏమిటి	ప్రశ్నకి జవాబు	Tools
Logging	Events యొక్క timestamped records	"ఏం జరిగింది? ఎందుకు fail అయ్యింది?"	ELK stack, Loki, CloudWatch
Metrics/Monitoring	Numbers over time (CPU, latency, error rate, requests/sec)	"System ఎంత ఆరోగ్యంగా ఉంది?"	Prometheus + Grafana, Datadog

అంశం	ఏమిటి	ప్రశ్నకి జవాబు	Tools
Tracing	ఒక request అన్ని services గుండా ప్రయాణం	"ఈ slow request ఎక్కడ time తీసుకుంది?"	Jaeger, OpenTelemetry
Alerting	Threshold దాటితే team కి notification	"సమస్య వచ్చింది, ఎవరైనా చూడండి!"	PagerDuty, Opsgenie

> **Observability = human body యొక్క vital signs.** Logging = doctor notes (ఏం జరిగింది). Metrics = heart rate, BP monitor (continuous health numbers). Tracing = ఒక్క blood cell ని follow చేయడం (request journey). Alerting = heart rate danger level దాటితే మోగే alarm. వీటి లేకుండా production నడపడం = కళ్ళు మూసుకుని car నడపడం – break అయ్యేదాకా తెలియదు.

కీలక metrics (Google SRE "Four Golden Signals"): Latency (ఎంత slow), Traffic (ఎంత load), Errors (ఎంత fail), Saturation (resources ఎంత నిండాయి). వీటిని monitor చేస్తే system ఆరోగ్యం తెలుస్తుంది. **SLI/SLO/SLA:** SLI = measured metric (99.95% uptime); SLO = internal target (99.9%); SLA = customer తో contract (99.5%, లేకపోతే penalty).

స్తంభం 5 – "You build it, you run it" culture + secrets

- **On-call:** teams తమ services కి on-call ఉంటారు – production alert వస్తే (అర్ధరాత్రి కూడా) respond చేస్తారు. ఇది ownership పెంచుతుంది – "నా code prod లో break అయితే నన్నే లేపుతుంది" → మంచి code, మంచి tests, మంచి monitoring రాస్తావు.
- **Blameless culture:** incident అయితే మనిషిని నిందించరు – system/process ని మెరుగుపరుస్తారు (Topic 13 postmortem).
- **Secrets management:** passwords, API keys, tokens ని **ఎప్పుడూ code/Git లో పెట్టకూడదు** (leak అయితే disaster). Environment variables, secret managers (AWS Secrets Manager, HashiCorp Vault, GitHub Secrets) వాడాలి. (ఇది security basic – Topic 11 GitHub Actions లో `secrets.` చూశాం.)

12-Factor App – cloud-native apps కి golden checklist

12-Factor App = Heroku engineers రాసిన, cloud లో బాగా నడిచే (scalable, portable, maintainable) apps ఎలా build చేయాలో చెప్పే methodology. MERN app cloud కి deploy చేసేటప్పుడు ఇవి కీలకం. అన్నీ గుర్తుంచుకోనవసరం లేదు, కానీ ముఖ్యమైనవి SSE కి తెలియాలి:

Factor	సూత్రం	MERN లో అర్థం
Codebase	ఒక codebase, version control లో, చాలా deploys	ఒక Git repo → dev/staging/prod
Dependencies	explicit గా declare (isolate)	`package.json` లో అన్నీ, global installs కాదు

Factor	సూత్రం	MERN లో అర్థం
Config	config ని environment లో (code లో కాదు)	DB URL, secrets → env vars, hardcode కాదు
Backing services	DB/cache/queue ని attached resources గా	MongoDB, Redis ని URL తో swap చేయగలగాలి
Build, release, run	ఈ 3 stages ని strictly వేరు	Docker build → tagged release → run
Processes	app ని stateless గా	session ని memory లో కాదు, Redis/DB లో (scale కి)
Port binding	app తనే port ద్వారా serve	Node `app.listen(PORT)`
Concurrency	processes జోడించి scale (horizontal)	ఎక్కువ containers, పెద్ద machine కాదు
Disposability	fast startup + graceful shutdown	container ఎప్పుడైనా kill/restart అవ్వచ్చు
Dev/prod parity	environments ని దగ్గరగా	Docker → local = prod
Logs	logs ని event stream గా (stdout కి)	file కాదు, stdout → aggregator
Admin processes	one-off tasks (migrations) ని same env లో	migration scripts, same config

అతి ముఖ్యమైనవి (interview): **config ని env లో ఉంచడం** (secrets code లో కాదు), **stateless processes** (horizontal scaling కి – session ని external store లో), **dev/prod parity** (Docker). ఈ మూడు MERN app ని "నా laptop లో నడిచే toy" నుండి "cloud లో scale అయ్యే production system" గా మారుస్తాయి.

Security Basics – ప్రతి engineer బాధ్యత (DevSecOps)

Security కేవలం "security team" పని కాదు – modern teams లో **"shift-left security"** (DevSecOps): ప్రతి engineer basic security ఆలోచించాలి. MERN dev గా నీకు కీలకంగా తెలియాల్సినవి (OWASP top risks నుండి):

Risk	ఏమిటి	MERN లో fix
Injection	untrusted input ని query లో నేరుగా	parameterized queries, Mongoose (raw `$where` కాదు), input validation
Broken auth	weak password/session handling	bcrypt password hashing, JWT proper expiry, HTTPS-only cookies
Sensitive data exposure	secrets/PII leak	HTTPS everywhere, secrets in vault, DB encryption
Broken access control	user వేరేవాడి data చూడగలడు	ప్రతి request కి authorization check (కేవలం authentication కాదు)

Risk	ఏమిటి	MERN లో fix
XSS	malicious script user browser లో	React auto-escapes; `dangerouslySetInnerHTML` avoid; sanitize
Vulnerable dependencies	పాత npm package లో known bug	`npm audit`, Dependabot, regular updates

మూడు golden rules: **(1) ఎప్పుడూ user input ని నమ్మకు** – validate & sanitize. **(2) Least privilege** – ప్రతిదానికీ కనీస access మాత్రమే. **(3) Secrets ని code లో కాదు** (మళ్ళీ). Security ని CI లో automate చేయాలి – SAST (static code scan), dependency scan, secret scan ప్రతి PR కి. "Security బగ్ ని production లో పట్టుకోవడం 100x ఖరీదు" – Topic 2 cost-of-change security కి కూడా వర్తిస్తుంది.

Real MERN team లో

MERN app ఒక **Docker container** గా pack అవుతుంది (Node backend + build). CI/CD (Topic 11) ఆ image ని build చేసి **Kubernetes** (లేదా simpler: AWS ECS/Fargate) కి deploy చేస్తుంది – auto-scaling, self-healing. Infrastructure (MongoDB Atlas, Redis, load balancer) **Terraform** తో define. Production లో **Datadog/Grafana** metrics, **CloudWatch/Loki** logs, **PagerDuty** alerts. Error rate spike అయితే on-call engineer కి page వస్తుంది. Secrets AWS Secrets Manager లో, code లో ఎప్పుడూ లేవు. SSE గా నువ్వు వీటన్నిటినీ *touch* చేస్తావు – expert కాకపోయినా, comfortable గా.

Key Points

- **DevOps = Dev + Ops culture:** wall కూల్చడం, **"you build it, you run it"** – code యొక్క full lifecycle (build → run → monitor → fix) ownership.
- **Docker/Containers:** app + deps ని portable box లో → "works on my machine" సమస్య పరిష్కారం. VM కంటే light (kernel share).
- **Kubernetes:** container orchestration – scheduling, self-healing, auto-scaling, rolling updates. Declarative (desired state).
- **IaC (Terraform):** infrastructure ని code గా → reproducible, version-controlled, automated.
- **Observability:** Logging (ఏం జరిగింది), Metrics (ఆరోగ్యం), Tracing (request journey), Alerting (సమస్య notify). Four Golden Signals; SLI/SLO/SLA.
- **Secrets ఎప్పుడూ code లో కాదు** – env vars/secret managers. On-call + blameless culture.

Interview దృష్టి

Q: DevOps అంటే ఏమిటి? ఒక tool నా? A: DevOps tool కాదు – Dev మరియు Ops మధ్య wall ని కూల్చే culture. Core idea "you build it, you run it" – నేను రాసిన code ని నేనే deploy, monitor, fix చేస్తాను. ఇది ownership పెంచుతుంది, quality మెరుగుపరుస్తుంది. దీన్ని enable చేసే practices – CI/CD, containers (Docker), orchestration (K8s), IaC (Terraform), observability (monitoring/logging/alerting).

Q: Docker ఎందుకు? Container vs VM? A: Docker "works on my machine but breaks in prod" సమస్యని పరిష్కరిస్తుంది – app + అన్ని dependencies ని ఒక portable container లో pack చేస్తే, ఏ environment లో అయినా అచ్చం ఒకేలా నడుస్తుంది. VM మొత్తం OS ని copy చేస్తుంది (heavy, GB, slow);

container host kernel ని share చేసి కేవలం app+deps carry చేస్తుంది (light, MB, seconds start) – అందుకే efficient, dense.

Q: Production issue ఎలా debug చేస్తారు? A: Observability tools మీద ఆధారపడతా – metrics (Grafana/Datadog) చూసి ఎప్పుడు, ఏ metric (latency/error rate) spike అయ్యిందో గుర్తిస్తా, logs లో ఆ time window లో errors వెతుకుతా, tracing తో slow request ఏ service లో ఆగిందో కనుక్కుంటా. Alert ఆ issue ని ముందే surface చేస్తుంది. వీటి లేకుండా production debug చేయడం కళ్ళు మూసుకుని – అందుకే monitoring/logging తప్పనిసరి.

Q: API keys, passwords ని code లో పెట్టవచ్చా? A: ఎప్పుడూ కాదు. Git history లో ఉంటే leak అయితే disaster (public repo, ex-employee). Secrets ని environment variables లేదా secret managers (AWS Secrets Manager, Vault, GitHub Secrets) లో ఉంచాలి, code runtime లో inject చేయాలి. CI/CD లో కూడా `secrets.` references వాడతాం, plain text కాదు.

---

13. Documentation, Estimation & Collaboration

వివరణ

SSE అంటే కేవలం మంచి code కాదు – **team ని effective గా పని చేయించే engineer.** ఈ topic మూడు "soft but critical" skills: **documentation** (knowledge ని preserve చేయడం), **estimation** (ఎంత time పడుతుందో చెప్పడం), మరియు **collaboration** (team గా, incidents గుండా పని చేయడం). Non-CS self-taught devs తరచు వీటిని underrate చేస్తారు – కానీ senior levels లో ఇవే నిన్ను వేరు చేస్తాయి. "Best coder" promotion తేదు; "team ని level-up చేసేవాడు" తెస్తుంది.

Real-life Scenario

> **Documentation లేని code = manual లేని powerful machine.** ఒక factory లో అద్భుతమైన machine ఉంది, కానీ దాన్ని operate చేయడం ఒక్క వ్యక్తికే తెలుసు. అతను సెలవు పెడితే? Leave అయితే? Machine నిరుపయోగం. ఆ knowledge అతని తలలోనే బంధించబడింది – **"bus factor = 1"** (ఆ ఒక్క వ్యక్తి bus కింద పడితే, project ఆగిపోతుంది). Documentation ఆ knowledge ని team అందరికీ అందుబాటులో పెడుతుంది – machine ఎప్పుడూ నడుస్తుంది.

Part A – Documentation

ఎందుకు: Code "ఏం చేస్తోందో" చెప్తుంది, కానీ "ఎలా వాడాలి, ఎందుకు ఇలా, ఎలా setup చేయాలి" చెప్పదు. Documentation ఆ gap పూడుస్తుంది. మంచి docs = knowledge preserved (bus factor↑), onboarding fast, తక్కువ "ఎలా చేయాలి?" interruptions.

రకాలు:

Doc రకం	ఎవరికోసం	ఏం ఉంటుంది
README	ఈ repo touch చేసే ఎవరైనా	Project ఏమిటి, ఎలా setup, ఎలా run, ఎలా test, ఎలా contribute

Doc రకం	ఎవరికోసం	ఏం ఉంటుంది
API docs	ఈ API వాడే devs (frontend, other teams)	Endpoints, request/response format, auth, error codes (OpenAPI/Swagger)
Design docs / ADR	Future engineers	ఎందుకు ఇలా design చేశాం, ఏ alternatives (Topic 6)
Runbooks	On-call engineers	"ఈ alert వస్తే ఏం చేయాలి" step-by-step
Inline comments	Code చదివేవాళ్ళు	ఎందుకు (why), ఏం కాదు (Topic 8)

మంచి README structure: Project title + one-line description → features → prerequisites → installation → running locally → running tests → environment variables → deployment → contributing → license. కొత్త dev README చదివి 15 నిమిషాల్లో project run చేయగలగాలి – అదే success metric.

API docs (MERN కి ముఖ్యం): REST API కి **OpenAPI/Swagger** spec రాస్తే – interactive docs auto-generate అవుతాయి, frontend devs endpoints ని self-serve గా అర్థం చేసుకుంటారు, "ఈ API ఎలా call చేయాలి?" అని అడగరు. Postman collections కూడా.

Golden rule: docs ని code దగ్గరే ఉంచు (same repo), లేకపోతే stale అవుతాయి. Outdated docs > no docs (misleading). Code మారితే docs కూడా PR లోనే మారాలి.

Part B – Estimation

Estimation = ఒక పని ఎంత time/effort పడుతుందో అంచనా వేయడం. Business planning కి కీలకం (ఎప్పుడు ship చేస్తాం?), కానీ engineers అత్యంత భయపడే skill. ఎందుకు కష్టం? **Software inherently unpredictable** – unknown unknowns, hidden complexity, dependencies.

Techniques (Topic 4 story points recap + more):

- **Story points (relative):** absolute hours కాదు, relative complexity (Fibonacci, planning poker). Topic 4 చూడు.
- **T-shirt sizing:** S/M/L/XL – rough, early estimation.
- **Three-point estimate:** Optimistic + Most-likely + Pessimistic → `(O + 4M + P)/6` (PERT). Uncertainty ని capture చేస్తుంది.
- **Break it down:** పెద్ద task ని చిన్న subtasks గా విడగొట్టు – ప్రతిది estimate సులభం + accurate.

Estimation traps & wisdom:

- **Planning Fallacy:** మనం ఎప్పుడూ underestimate చేస్తాం (optimism bias). "2 రోజులు" అంటే సాధారణంగా 4.
- **Add buffer for unknowns:** testing, code review, meetings, bugs, integration – వీటిని మర్చిపోతాం. "Coding time" ≠ "done time."
- **Estimates ≠ commitments/deadlines.** Estimate = best guess with uncertainty. Manager దాన్ని hard deadline గా మార్చడం anti-pattern. Ranges ఇవ్వు ("3-5 రోజులు"), single number కాదు.

- **Padding vs honesty balance:** అతిగా pad చేస్తే lazy అనిపిస్తుంది; తక్కువ చెప్తే miss అవుతావు. Honest, reasoned estimate + assumptions స్పష్టంగా.

> **Estimation = ప్రయాణ time అంచనా.** "Office కి ఎంత time?" – normal traffic లో 30 నిమిషాలు (most-likely). కానీ accident అయితే గంట (pessimistic), ఖాళీ road అయితే 20 (optimistic). అనుభవం ఉన్నవాడు "30-45 నిమిషాలు, traffic బట్టి" అని range + assumption చెప్తాడు. Junior "30 నిమిషాలు" అని single number చెప్పి, traffic లో ఇరుక్కుంటాడు. Software అలాంటిదే – hidden traffic (complexity) ఎప్పుడూ ఉంటుంది.

Part C – Collaboration & Incident/Postmortem culture

Teamwork basics:

- **Communication over-index:** async (Slack, PR comments, docs) + sync (standup, meetings). Remote teams లో writing skill = superpower.
- **Ask for help early:** 2 గంటలు stuck అయితే అడుగు – "30-min rule." Ego కోసం రోజు waste చేయకు.
- **Unblock others:** నీ review, నీ answer వల్ల ఇంకొకరు blocked అయితే – priority ఇవ్వు.
- **Disagree & commit:** discussion లో నీ view చెప్పు, కానీ team decision తీసుకున్నాక (నీది కాకపోయినా) దానికి commit అవ్వు.

RACI – ఎవరు దేనికి బాధ్యులు (పెద్ద tasks/projects లో): ఒక పని మీద గందరగోళం రాకుండా, roles ని స్పష్టం చేసే framework. **R**esponsible (పని చేసేవాడు), **A**ccountable (final బాధ్యత, ఒక్కరే), **C**onsulted (సలహా అడిగేవాళ్ళు), **I**nformed (తెలియజేయాల్సినవాళ్ళు). ఉదా: ఒక feature కి – dev = Responsible, tech lead = Accountable, security team = Consulted, PO = Informed. "అందరూ బాధ్యులు అంటే ఎవరూ బాధ్యులు కాదు" అనే trap ని ఇది ఆపుతుంది. చిన్న tasks కి overkill, కానీ cross-team work లో clarity ఇస్తుంది.

Async-first communication (remote/distributed teams): ముఖ్యమైన decisions ని writing లో (doc, PR, thread) పెట్టు – meetings కాదు. ఎందుకు? (1) time zones – అందరూ ఒకేసారి online ఉండరు. (2) record – తర్వాత reference. (3) clarity – రాసేటప్పుడు thinking sharp అవుతుంది. Meetings ని decisions కి కాదు, discussion/alignment కి మాత్రమే వాడు. Modern engineering లో **strong writing = strong engineering.**

Incident management (production break అయినప్పుడు):

1. **Detect** – alert/monitoring (Topic 12).
2. **Respond** – on-call engineer acknowledges, severity assess (SEV1 = అందరూ down; SEV3 = minor).
3. **Mitigate** – ముందు bleeding ఆపు (rollback, feature flag off) – root cause తర్వాత. **"Stop the bleeding first."**
4. **Resolve** – root cause fix.
5. **Postmortem** – తర్వాత ఏం జరిగింది, ఎందుకు, ఎలా ఆపాలో document.

Blameless Postmortem (అతి ముఖ్యమైన culture): incident తర్వాత "ఎవరు తప్పు చేశారు?" అని కాదు – **"ఏ system/process ఈ human error ని సాధ్యం చేసింది?"** అని అడగడం. ఎందుకు blameless?

- మనిషిని నిందిస్తే → జనం mistakes దాచుతారు → learning ఆగుతుంది → అదే error మళ్ళీ.
- System ని మెరుగుపరిస్తే → అదే mistake ఎవరూ చేయలేరు → నిజమైన fix.
- "ఒక junior prod DB delete చేశాడు" → తప్పు junior ది కాదు; *ఒక junior కి prod DB delete చేసే access ఇచ్చిన system* తప్పు. Fix = permissions, confirmations, backups – junior ని తిట్టడం కాదు.

> **Blameless postmortem = plane crash investigation.** Plane crash అయితే, pilot ని జైలుకి పంపడం లక్ష్యం కాదు – "ఏం system fail అయ్యింది, భవిష్యత్తులో ఎలా ఆపాలి" అని కనుక్కోవడం. అందుకే aviation అత్యంత safe industry – ప్రతి incident నుండి system నేర్చుకుంటుంది, మనిషిని నిందించదు. Software SRE culture దీన్నే copy చేసింది.

Postmortem doc: timeline (ఏం జరిగింది ఎప్పుడు), impact (ఎంతమంది users, ఎంత revenue), root cause (5 Whys), action items (ఎలా prevent – owners + deadlines తో).

Real MERN team లో

ప్రతి repo లో మంచి README + Swagger API docs. ముఖ్యమైన decisions ADRs లో. Sprint planning లో team story points తో estimate చేస్తుంది, ranges ఇస్తుంది. Production incident అయితే – on-call PagerDuty alert పొంది, war room (Slack channel), rollback తో mitigate, తర్వాత blameless postmortem doc (timeline, root cause, action items) రాసి team తో share. ఆ action items backlog లో tickets అవుతాయి. ఇదంతా "code రాయడం" కాదు – కానీ ఇదే SSE ని define చేస్తుంది.

Key Points

- **Documentation** = knowledge preserved, bus factor↑, fast onboarding. README (setup/run), API docs (Swagger), ADRs (why), runbooks (on-call). Docs ని code దగ్గరే ఉంచు.
- **Estimation** hard (software unpredictable). Story points/T-shirt/three-point, break down, buffer for unknowns. **Estimates ≠ commitments**; ranges ఇవ్వు, planning fallacy (underestimate) గుర్తుంచుకో.
- **Collaboration:** over-communicate, ask early (30-min rule), unblock others, disagree & commit.
- **Incident:** detect → respond → **mitigate (stop bleeding first)** → resolve → postmortem.
- **Blameless postmortem:** మనిషిని కాదు, system/process ని fix. Blame → hidden mistakes → repeat; blameless → real learning. Aviation-style.

Interview దృష్టి

Q: ఒక task estimate ఎలా చేస్తారు? A: ముందు task ని చిన్న subtasks గా విడగొట్టి, ప్రతిదాన్ని story points (relative complexity) తో estimate చేస్తా. Coding time మాత్రమే కాదు – testing, code review, integration, meetings కూడా లెక్కిస్తా (వీటిని జనం మర్చిపోతారు). Single number కాదు, range + assumptions ఇస్తా ("3-5 రోజులు, ఈ API ready అయితే"). Estimate ఒక informed guess with uncertainty – hard commitment కాదు. Software inherently unpredictable కాబట్టి planning fallacy (underestimation) కి buffer జోడిస్తా.

Q: Blameless postmortem అంటే? ఎందుకు? A: Incident తర్వాత "ఎవరు తప్పు చేశారు" కాదు, "ఏ system/process ఈ error ని సాధ్యం చేసింది" అని అడగడం. ఎందుకంటే మనిషిని నిందిస్తే జనం mistakes

దాచుతారు, learning ఆగుతుంది, అదే error మళ్ళీ జరుగుతుంది. System ని fix చేస్తే ఎవరూ ఆ mistake చేయలేరు. ఉదా: junior prod DB delete చేస్తే, తప్పు అతనిది కాదు – అతనికి ఆ access, safeguards లేకపోవడం system failure. Aviation crash investigation లాంటిది – blame కాదు, prevention.

Q: Production down అయ్యింది, మీరు on-call. మొదట ఏం చేస్తారు? A: మొదట severity assess (ఎంతమంది affected), తర్వాత **"stop the bleeding"** – root cause వెతకడం కంటే ముందు impact ఆపడం (last deploy rollback, లేదా feature flag off). Users కి service restore అయ్యాక, ప్రశాంతంగా root cause debug (metrics, logs, tracing). Fix deploy చేశాక, blameless postmortem రాసి – timeline, root cause (5 Whys), action items (prevention). Mitigation first, investigation later – ఇది కీలక principle.

Q: మంచి documentation యొక్క లక్షణం? A: కొత్త dev README చదివి 15 నిమిషాల్లో project setup + run చేయగలగాలి. Code దగ్గరే ఉండాలి (same repo, PR లోనే update – లేకపోతే stale). README (setup/run/test), API docs (Swagger – frontend self-serve), ADRs (ఎందుకు ఇలా), runbooks (on-call). Outdated docs no-docs కంటే ప్రమాదం (misleading), కాబట్టి code మారితే docs కూడా మారాలి.

Part 5 – Reference (రిఫరెన్స్)

> ఇప్పుడు అన్నీ నేర్చుకున్నావ్. ఈ చివరి Part = interview రోజు ముందు చదివే cheat-sheet. Process + behavioral Q&A, ప్రతి concept ని గుర్తుంచుకునే memory tips, మరియు self-taught devs చేసే common mistakes – అన్నీ ఒకే చోట. దీన్ని bookmark చేసి interview ముందు revise చెయి.

---

14. Interview Q&A + Memory Tips + Common Mistakes

వివరణ

SSE interview లో technical rounds (DSA, HLD, LLD) తో పాటు, ఒక **"process & behavioral" round** దాదాపు ఎప్పుడూ ఉంటుంది. ఇక్కడే self-taught devs తడబడతారు – code వచ్చు కానీ "మీ team ఎలా పని చేస్తుంది? Conflict ఎలా handle చేశారు? Production down అయితే?" వంటి questions కి fluent answers లేవు. ఈ topic ఆ gap ని పూర్తిగా పూడుస్తుంది. Behavioral questions కి **STAR method** వాడు: **S**ituation → **T**ask → **A**ction → **R**esult.

Real-life Scenario

> **Process round = driving test యొక్క "theory + attitude" భాగం.** Practical driving test (car నడపడం) = technical rounds (DSA/coding). కానీ examiner కేవలం "car నడపగలవా?" చూడడు – "traffic rules తెలుసా? accident అయితే ఏం చేస్తావ్? ఇతర drivers తో ఎలా behave చేస్తావ్?" కూడా చూస్తాడు. ఎందుకంటే road మీద గంటల తరబడి ఉండేవాడికి skill ఒక్కటే చాలదు – judgment + discipline + others తో cooperation కావాలి.
>
> SSE process round అదే – నీ code (driving skill) technical rounds లో చూశారు. ఇప్పుడు "team లో ఎలా పని చేస్తావ్? Production crash అయితే? Disagreement వస్తే? Deadline miss అవుతుంటే?" – నీ *engineering maturity* చూస్తారు. ఈ prep లేకపోతే, మంచి coder అయినా ఇక్కడ జారిపోతావు.

Rapid-fire Process Q&A (అన్ని topics కవర్)

Q: SDLC అంటే? Phases? A: Idea నుండి maintenance వరకు lifecycle – requirements → design → implement → test → deploy → maintain. Cycle, ఎందుకంటే maintenance feedback కొత్త requirements అవుతుంది.

Q: Agile అంటే? Waterfall కంటే ఎప్పుడు better? A: Agile = short iterations + constant feedback + welcome change. Requirements evolving అయిన products/web కి better. Waterfall frozen-scope, regulated projects కి.

Q: Scrum roles? A: PO (*what/why* – backlog), Scrum Master (*process*, servant leader, blockers), Dev Team (*how*, self-organizing).

Q: Sprint ceremonies? A: Planning (ఏం చేస్తాం), Daily standup (sync/blockers, 15min), Review (product demo to stakeholders), Retro (process improve).

Q: Story points ఎందుకు hours కాదు? A: Relative complexity – individual-independent, team-stable. Hours absolute + person-dependent (junior 12, senior 3 → unstable).

Q: Velocity ని performance metric గా వాడవచ్చా? A: వద్దు. అది planning tool. Teams మధ్య compare invalid; target చేస్తే points inflate (Goodhart's Law).

Q: Functional vs non-functional requirements? A: FR = ఏం చేయాలి (features); NFR = ఎంత బాగా (performance, scale, security, "-ilities"). NFRs architecture ని shape చేస్తాయి.

Q: Feature branch vs trunk-based? A: Feature branch = main stable + short branches + PR (default). Trunk-based = అందరూ main మీద, short branches, feature flags – high velocity + strong CI.

Q: DRY, KISS, YAGNI? A: DRY = repeat చేయకు; KISS = simple గా ఉంచు; YAGNI = అవసరం లేనిది build చేయకు. కానీ judgment తో (over-DRY = wrong abstraction).

Q: Tech debt? A: Speed కోసం తీసుకున్న shortcut, "వడ్డీ"తో తిరిగి చెల్లించాలి. Tracked + deliberate అయితే OK; untracked/ignored అయితే crippling.

Q: Test pyramid? A: చాలా unit (fast/cheap/stable) → కొంత integration → కొన్ని E2E (slow/brittle/high-confidence). Ice-cream-cone (ఎక్కువ E2E) anti-pattern.

Q: TDD? A: Red (fail test) → Green (min code to pass) → Refactor. Test-first → clear spec, testable code, no over-engineering.

Q: 100% coverage = bug-free? A: కాదు. Coverage = "lines run అయ్యాయి," "సరిగ్గా పని చేస్తున్నాయి" కాదు. Guide, target కాదు.

Q: CI vs CD? A: CI = ప్రతి merge auto build+test. CD (Delivery) = ఎప్పుడూ deployable + manual button; CD (Deployment) = button లేదు, fully auto to prod.

Q: Blue-green vs canary? A: Blue-green = 2 envs, traffic switch, instant rollback, 2x infra. Canary = కొద్దిమందికి ముందు (5%), monitor, క్రమంగా 100% – risky/large-scale కి safest.

Q: Docker ఎందుకు? A: "Works on my machine" fix – app+deps ని portable container లో. VM కంటే light (kernel share).

Q: Blameless postmortem? A: మనిషిని కాదు, system/process ని fix. Blame → hidden mistakes → repeat; blameless → learning.

Behavioral Questions (STAR method) – SSE కి common

Q: ఒక teammate తో technical disagreement – ఎలా handle చేశారు? A (STAR): *Situation* – REST vs GraphQL మీద ఒక senior తో నేను disagree అయ్యా. *Task* – team ఒక decision తీసుకోవాలి, relationship చెడగొట్టకుండా. *Action* – నా reasoning ని data తో (latency, over-fetching numbers) present చేశా, అతని concerns విన్నా, ఒక చిన్న POC చేసి compare చేశాం. *Result* – data GraphQL వైపు

చూపింది, team అంగీకరించింది; అంగీకరించకపోయినా "disagree & commit" చేసేవాడిని. కీలకం – ego కాదు, best outcome for team.

Q: ఒక mistake చేసి production ని affect చేసిన సందర్భం? A (STAR): *Situation* – నేను ఒక DB migration లో index మర్చిపోయా, prod slow అయ్యింది. *Task* – వెంటనే fix + future prevention. *Action* – ముందు rollback చేసి bleeding ఆపా (mitigate first), తర్వాత index జోడించి re-deploy చేశా, blameless postmortem రాశా. *Result* – 20 నిమిషాల్లో resolved; action item గా migration checklist + CI లో migration review step జోడించాం. అదే mistake మళ్ళీ జరగలేదు. (ఇది ownership + blameless learning చూపిస్తుంది.)

Q: Deadline miss అవుతుందని తెలిసిన సందర్భం – ఏం చేశారు? A (STAR): *Situation* – sprint మధ్యలో ఒక feature అనుకున్నదానికంటే complex అని తెలిసింది. *Task* – deadline vs quality balance. *Action* – వెంటనే (చివరి రోజు కాదు) PO కి చెప్పా, scope ని negotiate చేశా (core feature ship, nice-to-have తర్వాతి sprint), risks స్పష్టంగా. *Result* – core on-time ship అయ్యింది, quality compromise లేదు. కీలకం – bad news early, silent గా miss కాదు.

Q: ఒక juniorకి mentor చేసిన అనుభవం? A: Code review లో orders కాదు, questions + "why" explain చేసేవాడిని; pair programming చేసేవాడిని; చిన్న tasks నుండి పెద్దవి కి grow చేసేవాడిని. Result – అతను 3 నెలల్లో independent గా features ship చేయడం మొదలుపెట్టాడు. SSE = team ని level-up చేసేవాడు.

Q: Ambiguous requirement – ఎలా proceed అయ్యారు? A: Assumptions తో code చేయకుండా, clarifying questions అడిగా (users, edge cases, scale), mockup చేసి stakeholder feedback తీసుకున్నా, అప్పుడే build. తప్పు build చేయడం కంటే ఒక గంట clarify చౌక.

Q: కొత్త technology వేగంగా నేర్చుకోవాల్సిన సందర్భం? A (STAR): *Situation* – ఒక feature కి real-time updates కావాలి, team లో ఎవరికీ WebSockets అనుభవం లేదు. *Task* – వారంలో నేర్చుకుని ship చేయాలి. *Action* – official docs + ఒక small POC చేశా, edge cases (reconnection, scaling) గురించి seniors తో discuss చేశా, learnings ని team doc గా రాశా. *Result* – feature time కి ship అయ్యింది, ఆ doc వల్ల team మిగతావాళ్ళు కూడా త్వరగా onboard అయ్యారు. కీలకం – నేర్చుకోవడం మాత్రమే కాదు, knowledge ని team కి share చేయడం.

Q: Manager unrealistic deadline పెట్టారు. ఏం చేశారు? A (STAR): *Situation* – 3 వారాల పని కి 1 వారం deadline ఇచ్చారు. *Task* – realistic గా negotiate, silent గా burnout కాదు. *Action* – నా estimate ని breakdown తో (ఏ subtask ఎంత, ఎందుకు) present చేశా, options ఇచ్చా – "full scope 3 వారాలు, లేదా core (MVP) 1 వారంలో + మిగతావి తర్వాత." Data తో మాట్లాడా, emotion తో కాదు. *Result* – manager MVP-first approach ఎంచుకున్నాడు; on-time ship, quality కాపాడాం. Estimates ని defend చేయడం (respectfully, data తో) senior skill.

Memory Tips – గుర్తుంచుకోవడానికి

SDLC phases (6): "**R**equirements **D**esign **I**mplement **T**est **D**eploy **M**aintain" → *"Really Don't Implement Tests? Damn, Maintenance!"* (tests skip చేస్తే maintenance నరకం అని గుర్తు).

Agile Manifesto (4 values): *Individuals, Working software, Customer collaboration, Responding to change* → **"I W**ant **C**hange **R**esponsively" (I-W-C-R).

Scrum ceremonies (4): **P**lanning, **S**tandup, **R**eview, **R**etro → "**P**lease **S**tart **R**eviewing **R**egularly."

Review vs Retro: Re**v**iew = **v**isible product (stakeholders); Re**t**ro = **t**eam process (internal). "v = view the product, t = team talks."

SOLID: Single, Open/closed, Liskov, Interface segregation, Dependency inversion (deep `LLD_Telugu.md`).

Clean code mantras: DRY-KISS-YAGNI – "Don't Repeat, Keep Simple, You Ain't Gonna Need It."

Test pyramid: "**U**nit చాలా, **I**ntegration కొంత, **E**2E కొన్ని" – base నుండి top కి తక్కువ (**U**p = fewer). Cost & speed పైకి పెరుగుతాయి.

TDD loop: 🔴 **Red** → 🟢 **Green** → 🔵 **Refactor.** "Fail, Pass, Polish."

CI/CD: CI = **I**ntegrate often (build+test). Delivery = **button ఉంది.** Deployment = **button లేదు.**

Deployment strategies: "**R**ecreate, **R**olling, **B**lue-green, **C**anary" – risk↓ safety↑ ఆ order లో.

Incident response: "**Stop the bleeding first**" (mitigate) → then investigate (root cause).

DevOps mantra: "**You build it, you run it.**"

Four Golden Signals: **L**atency, **T**raffic, **E**rrors, **S**aturation – "**LTE-S**" (mobile network గుర్తు).

Common Mistakes – self-taught devs చేసేవి (జాగ్రత్త!)

తప్పు	ఎందుకు ప్రమాదం	సరైనది
Requirements clarify చేయకుండా నేరుగా code	తప్పు feature build → rework	ముందు clarifying questions, then code
Tests skip ("deadline ఉంది")	ప్రతి change భయం, regressions, slow	కనీసం core logic కి unit tests
పెద్ద PRs (2000 lines)	review కష్టం, bugs దాటిపోతాయి	చిన్న PRs (200-400 lines), ఒక logical change
Commit messages: "fix", "update", "wip"	history అర్థంకాదు, debug కష్టం	imperative + why ("Fix null check in cart total")
Clever/complex code (show off)	maintain కష్టం, bugs	boring, readable, simple
"Works on my machine"	prod లో break	Docker, env parity, CI
Secrets code/Git లో	leak = disaster	env vars, secret managers
Over-engineering (future కోసం abstract)	complexity, waste (YAGNI)	ఇప్పటి అవసరానికి build
Feedback ని personal గా తీసుకోవడం	learning ఆగుతుంది, defensive	code review = learning; ego వదులు

తప్పు	ఎందుకు ప్రమాదం	సరైనది
Velocity/coverage ని target చేయడం	gaming, meaningless numbers	వాటిని guides గా వాడు, goals కాదు
Silent గా deadline miss	trust పోతుంది	bad news early, scope negotiate
Documentation రాయకపోవడం	bus factor=1, onboarding నరకం	README + API docs + ADRs
Big-bang rewrite	చాలావరకు fail (Netscape)	incremental refactor, tests తో
Stuck అయి గంటలు silent	time waste	30-min rule – ask for help

Glossary – SSE jargon quick reference

Interview/team meetings లో ఈ terms రోజూ వినిపిస్తాయి. ఒక్క line definition – గబగబా revise చేయడానికి:

Term	ఒక్క-line అర్థం
SDLC	Software development life cycle – idea నుండి maintenance వరకు phases
Agile	Short iterations + feedback + welcome change (mindset)
Scrum	Sprint-based Agile framework (roles, ceremonies, artifacts)
Sprint	Fixed timebox (సాధారణంగా 2 వారాలు) పని చేసే period
Backlog	చేయాల్సిన అన్ని పనుల prioritized list
Epic / Story / Task	పెద్ద feature → చిన్న user stories → చిన్న tasks
Story Point	Story యొక్క relative effort/complexity (Fibonacci)
Velocity	Sprint కి average completed points (planning tool)
Standup	రోజువారీ 15-min sync (nిన్న/ఇవాళ/blockers)
PO / SM	Product Owner (what/why) / Scrum Master (process)
DoD / DoR	Definition of Done / Ready – quality gates
Kanban	Continuous-flow framework, WIP limits
WIP	Work In Progress – ఒకేసారి ఎన్ని పనులు
FR / NFR	Functional (ఏం) / Non-functional (ఎంత బాగా) requirements
HLD / LLD	High-level (architecture) / Low-level (classes) design
ADR	Architecture Decision Record – ఎందుకు ఇలా decide చేశాం
MVP	Minimum Viable Product – తక్కువ features తో మొదటి version

Term	ఒక్క-line అర్థం
PR / MR	Pull/Merge Request – code merge కి review request
Trunk-based	అందరూ main మీద, short branches, feature flags
Feature flag	Code deployed కానీ toggle తో on/off
Tech debt	Speed కోసం shortcut, తర్వాత "వడ్డీ"తో చెల్లించాలి
Refactoring	Behavior మార్చకుండా structure మెరుగుపరచడం
DRY / KISS / YAGNI	Repeat చేయకు / simple గా / అవసరం లేనిది build చేయకు
SOLID	5 OO design principles (maintainability)
Code smell	Design problem ని సూచించే లక్షణం
Unit / Integration / E2E	ఒక function / modules కలిసి / whole app tests
TDD	Test-Driven Dev – Red → Green → Refactor
Mock / Stub	Dependencies ని test కోసం fake చేయడం
Coverage	ఎంత % code tests execute చేస్తాయి
Flaky test	ఏ మార్పు లేకుండా కొన్నిసార్లు fail అయ్యే test
CI / CD	Continuous Integration / Delivery-Deployment
Pipeline	Automated build → test → deploy steps
Blue-green / Canary	Deployment strategies (safe release)
Rollback	పాత working version కి తిరిగి వెళ్ళడం
Docker / Container	App + deps ని portable box లో pack
Kubernetes (K8s)	Container orchestration (scale, self-heal)
IaC	Infrastructure as Code (Terraform)
Observability	Logs + metrics + traces + alerts
SLI / SLO / SLA	Measured / target / contracted reliability
MTTR	Mean Time To Recovery (break→fix time)
On-call	Production alerts కి respond చేసే duty
Postmortem	Incident తర్వాత blameless analysis
DORA metrics	Deploy freq, lead time, change fail rate, MTTR
Bus factor	ఎంతమంది leave అయితే project ఆగుతుంది

Term	ఒక్క-line అర్థం
12-Factor	Cloud-native app best-practices checklist
STAR	Behavioral answer format (Situation-Task-Action-Result)

చివరి మాట – SSE mindset

ఈ document మొత్తం ఒక్క idea చుట్టూ తిరిగింది: **software engineering అంటే code కాదు – code చుట్టూ ఉన్న discipline.** Bootcamp నీకు React/Node నేర్పింది; ఈ guide నీకు *ఒక engineer ఎలా ఆలోచిస్తాడో* నేర్పింది. SSE interview లో వాళ్ళు వెతికేది ఇదే:

- నువ్వు **ఒక feature ని idea నుండి production monitoring వరకు** own చేయగలవా?
- నువ్వు **trade-offs తో ఆలోచిస్తావా** (fast vs cheap vs reliable)?
- నువ్వు **team ని level-up చేస్తావా** (reviews, docs, mentorship)?
- నువ్వు **long-term maintainability** కోసం రాస్తావా, కేవలం "ఇది పని చేస్తోంది" కాదా?
- Break అయినప్పుడు **calm గా, blameless గా, systematically** handle చేస్తావా?

ఈ ఐదింటికీ "అవును" అంటే – నీ background CS అయినా కాకపోయినా, నువ్వు **Senior Software Engineer.** కేవలం code రాయడం నేర్చుకోవడం కాదు – *software ని engineer చేయడం* నేర్చుకున్నావ్. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" – ఇప్పుడు నువ్వు మర్చిపోవు.

Key Points

- Process round దాదాపు ప్రతి SSE interview లో – ఈ topic ని interview ముందు revise చేయి.
- Behavioral questions కి **STAR** (Situation-Task-Action-Result); ready examples: disagreement, mistake, missed deadline, mentorship, ambiguity.
- Memory mnemonics ప్రతి Part కి – RDITDM, DRY-KISS-YAGNI, Red-Green-Refactor, LTE-S, "stop the bleeding first."
- Common mistakes = self-taught pitfalls; వీటిని consciously avoid చేస్తే junior→senior signal.
- SSE = code కాదు, **ownership + trade-offs + team enablement + long-term thinking + calm incident handling.**

Interview దృష్టి

Q: Non-CS background నుండి SSE role కి ఎందుకు ready అని అనుకుంటున్నారు? A: నేను coding ని self-study తో నేర్చుకున్నా, కానీ దానితో ఆగలేదు – engineering process (SDLC, Agile, testing, CI/CD, code review, DevOps, incident management) ని consciously study చేసి, ప్రతి practice *ఎందుకు* ఉందో అర్థం చేసుకున్నా. ఇప్పుడు నేను ఒక feature ని requirements clarify చేయడం నుండి, design, test, review, safely deploy, production monitor, break అయితే blameless postmortem వరకు own చేయగలను. CS degree ఒక foundation ఇస్తుంది; నేను ఆ foundation ని deliberate practice + real-world discipline తో build చేసుకున్నా. Interview లో నా code మాత్రమే కాదు, నా *engineering judgment* చూడండి.

Q: మీకు ఏ SSE skill ఇంకా బలహీనం, ఎలా మెరుగుపరుస్తున్నారు? A: (Honest answer maturity చూపిస్తుంది.) ఉదా – "పెద్ద-scale distributed systems design లో నా hands-on అనుభవం limited. దీన్ని

`HLD_Telugu.md` వంటి resources తో study చేస్తూ, side projects లో apply చేస్తూ, seniors design reviews లో active గా participate చేస్తూ మెరుగుపరుస్తున్నా." – self-awareness + growth mindset = senior signal.

🎯 Interview రోజు ముందు – 2-నిమిషాల final revision

interview కి బయలుదేరేముందు ఈ 15 lines ఒక్కసారి చదువు – మొత్తం doc యొక్క సారాంశం:

1. **Coding ≠ engineering** – engineering = time + scale + trade-offs. Code read 10x > written.
2. **SDLC** = requirements → design → implement → test → deploy → maintain (cycle). Cost of change late = 100x.
3. **Agile** = short iterations + feedback + welcome change. Waterfall = frozen scope, regulated.
4. **Scrum** = PO (what) + SM (process) + team (how); planning/standup/review/retro; story points (relative) + velocity (planning only).
5. **Kanban** = flow + WIP limits; interrupt-driven work కి.
6. **FR** = ఏం; **NFR** = ఎంత బాగా (NFRs architecture ని shape చేస్తాయి). Ambiguity → clarify, code కాదు.
7. **Feature branch + PR** = default; PR చిన్నది + green CI + self-reviewed.
8. **Clean code** = మనుషులకి readable. DRY/KISS/YAGNI (judgment తో). Tech debt = tracked shortcut.
9. **Test pyramid** = unit చాలా → integration → E2E కొన్ని. TDD = Red-Green-Refactor. Coverage ≠ bug-free.
10. **Code review** = collaboration, code ని (మనిషిని కాదు) review; correctness > style (style = automate).
11. **CI** = auto build+test; **CD** = deployable/auto-deploy. Canary/blue-green = safe release.
12. **DevOps** = "you build it, you run it." Docker (parity), K8s (orchestrate), IaC, observability. Secrets code లో కాదు.
13. **Estimation** = ranges, not commitments. **Incident** = stop bleeding first → blameless postmortem.
14. **Behavioral** = STAR. Ready examples: disagreement, mistake, deadline, mentorship, new tech.
15. **SSE mindset** = ownership + trade-offs + team enablement + long-term thinking. నీ *judgment* చూపించు.

> **All the best! 🚀 నీ background CS కాకపోయినా – నువ్వు ఇప్పుడు ఒక engineer లా ఆలోచిస్తావు. అదే వాళ్ళు వెతికేది. వెళ్ళి crack చెయ్యి.**